Stacklyst

Casos de uso e modelos do sistema

Laboratório de Engenharia de Software
Revisão de 3 de novembro de 2026
Base de código 6a2bfce e documentação local revisada

Pedro Antonio Silvestre Tescaro
Guilherme Alves de Campos
Paulo Henrique Leal dos Santos

Esta edição atualiza os modelos e a especificação do sistema. Os diagramas de atividades do negócio aprovados permanecem no documento 02, preservado integralmente. Os modelos revisados são editáveis no Astah UML. Cadastro e acesso usam o ator Usuário e declaram as pré-condições de sessão em cada caso.

As afirmações descrevem a fonte de implementação. Credenciais, infraestrutura externa, dispositivos e migrations dependem das verificações de ambiente indicadas. Esta revisão documental não representa execução de todos os fluxos em produção.

Roteiro de leitura

Roteiro de leitura 2
Referência de UML e critérios de modelagem 4
Casos de uso e sua especificação 5
Herança dos atores 5
Inclusão, extensão e pontos de extensão 5
Classes e persistência 6
Sequências e ligação com o código 6
Uso no seminário 6
Catálogo acadêmico de casos de uso 6
UC001 — Cadastrar conta 8
UC002 — Fazer login 8
UC003 — Gerenciar perfil e preferências 10
UC004 — Consultar e selecionar trilha 14
UC005 — Estudar lição 15
UC006 — Resolver exercício 16
UC007 — Consultar evolução e reconhecimento 18
UC008 — Disputar duelo de programação 21
UC010 — Enviar solução de duelo 24
UC012 — Consultar ajuda do exercício 24
UC013 — Gerenciar publicação própria 25
UC014 — Consultar e interagir com a comunidade 27
UC015 — Gerenciar eventos e desafios 30
UC016 — Avaliar duelo autorizado 33
UC017 — Gerenciar empresa e vagas 34
UC019 — Administrar usuários e recursos 36
UC021 — Acompanhar notificações 39
UC022 — Candidatar-se a vaga 40
UC023 — Participar de evento 41
UC024 — Solicitar credenciamento de avaliador 42

UC026 — Gerenciar candidaturas 43
UC028 — Avaliar resposta 45
UC029 — Executar código e testes 46
UC038 — Trocar mensagens privadas 47
Consolidação e redução do catálogo 49
Equivalência com os cenários de implementação 50
Casos de uso gerais 53
Classes conceituais gerais 1
Visão relacional geral reduzida 2
Casos de uso de aprendizagem 1
Classes de aprendizagem 1
Dados de aprendizagem 2
Casos de uso de comunidade 1
Classes de comunidade 1
Dados de comunidade 2
Casos de uso de duelos 1
Classes de duelos 1
Dados de duelos 2
Classes de recrutamento 3
Dados de recrutamento 4
Classes de eventos 5
Dados de eventos 6
Classes de identidade 7
Dados de identidade 8
Sequência completa 1 1
Sequência completa 2 2
Sequência completa 3 3
Sequência completa 4 4
Sequência completa 5 5
Sequência completa 9 6
Sequência completa 10 7

Sequência completa 11 8
Sequência completa 12 9
Sequência completa 13 10
Receber e avaliar resposta 11
Separar prévia de submissão 12
Creditar primeira conclusão 13
Autorizar publicação de vaga 14
Gravar vaga e etapas 15
Confirmar publicação 16
Registrar candidatura 17
Conferir gestão da candidatura 18
Atualizar e notificar 19
Correspondência dos modelos com a implementação 20
Tabelas relacionais 21
Uso da documentação no seminário 21
O que foi aproveitado do material fornecido 21
Ordem lógica sugerida 22
Roteiro de cada funcionalidade 22
Diagramas gerais e específicos 23
Recortes de sequência e código 23
Dados preparados 23
Tempo e divisão 24
Checklist de entrega 24

O sumário é atualizado na exportação pelo Word. As fontes editáveis e a matriz de rastreabilidade acompanham o pacote.

Referência de UML e critérios de modelagem

Referência principal: GUEDES, Gilleanes T. A. UML 2: uma abordagem prática. 3. ed. São Paulo: Novatec, 2018. ISBN 978-85-7522-644-5. Consultado no PDF fornecido pelo grupo. Os números de página abaixo são a posição no PDF de 657 páginas; podem diferir da paginação impressa.

O livro orienta a notação e a organização da documentação. As regras da Stacklyst são verificadas no código do projeto; os exemplos bancários e demais sistemas do livro não são requisitos desta plataforma. Os textos e diagramas desta revisão são próprios do projeto.

Casos de uso e sua especificação

Conforme a seção 3.4 (PDF, pp. 65-67), um caso de uso deve explicar o objetivo do ator e a resposta do sistema em linguagem compreensível. O catálogo identifica nome e ID, ator principal, pré-condições, fluxo principal, resultado esperado e alternativas. Referências de código e regras de negócio complementam a especificação e permitem verificar a implementação. A UML admite formatos diferentes para essa documentação; a estrutura adotada é uma adaptação ao projeto.

Cada passo do catálogo deve ser lido como interação entre ator e sistema. No diagrama, as elipses representam objetivos ou comportamentos reutilizados. Tabelas, componentes React, rotas e consultas são detalhados nos modelos estruturais e nas sequências, sem transformá-los em atores humanos.

Herança dos atores

A generalização/especialização segue a seção 3.7 (PDF, pp. 70-73). Avaliador, Recrutador e Administrador especializam Usuário. A linha contínua termina em triângulo vazio voltado para Usuário, o ator geral. Um papel especializado participa das interações comuns de Usuário e acrescenta as interações autorizadas ao seu papel.

Essa generalização representa papéis de interação. Não afirma que existam subclasses TypeScript desses nomes nem concede permissões em execução: o servidor continua validando sessão, papel, propriedade e escopo. Uma mesma pessoa pode exercer mais de um papel. Cadastro, recuperação e verificação pública são interações de Usuário cujas pré-condições permitem ausência de sessão.

Inclusão, extensão e pontos de extensão

Na inclusão, o comportamento chamado é obrigatório no contexto do caso que o inclui (seção 3.8; PDF, pp. 74-75). A dependência tracejada com «include» aponta do caso base para o incluído. Na visão geral, Resolver exercícios inclui Avaliar resposta; Submeter solução de duelo inclui Executar código. A validação das entradas e a autorização antecedem a execução efetiva, conforme os fluxos alternativos documentados.

Na extensão, o comportamento adicional ocorre sob uma condição, em um ponto identificável do caso base (seções 3.9 a 3.11; PDF, pp. 76-80). A dependência tracejada com «extend» aponta da extensão para o caso base. Executar código estende Avaliar resposta quando a atividade é de código, no ponto avaliar código. Consultar ajuda estende Resolver exercícios se o usuário solicitar e o modo permitir, no ponto consultar ajuda. Atividades objetivas podem ser avaliadas sem execução de código; a ajuda não é uma etapa obrigatória de todos os exercícios.

Relações de inclusão e extensão descrevem dependência comportamental. Não indicam ordem cronológica de navegação. Autenticação é pré-condição dos fluxos protegidos; não se repete um «include Fazer login» para toda operação.

Classes e persistência

Os capítulos 2 e 4 fundamentam classes, atributos, operações e multiplicidades. A seção 4.15 (PDF, pp. 213-218) distingue classes de tabelas e discute o mapeamento para persistência. As vistas CL desta revisão são conceituais, com nomes em português e responsabilidades do domínio. A visão geral tem oito classes; as vistas específicas têm quatro a seis. Uma classe conceitual não afirma que exista uma declaração TypeScript class com o mesmo nome.

As vistas DER são recortes relacionais em notação de classes UML: dez tabelas na visão geral e quatro ou cinco nas vistas específicas. CP significa chave primária e CE, chave estrangeira. Os nomes visíveis são traduções; a tabela de correspondência identifica as tabelas e campos reais. Campos e relações auxiliares permanecem no dicionário completo do banco. O Astah UML instalado não oferece o editor ER da edição Professional; a notação utilizada está explícita. A redução é de apresentação, sem alteração do schema.

Sequências e ligação com o código

O capítulo 7 orienta participantes, linhas de vida, mensagens, retornos e condições. As seções 7.3.3 e 7.8 (PDF, pp. 272-273 e 286-293) apoiam a leitura de respostas e alternativas. A ordem vertical mostra a progressão da interação; a posição horizontal separa responsabilidades.

Nas vistas nativas desta revisão, chamadas são mensagens UML editáveis e condições são guardas nas mensagens e notas de resposta. As respostas podem ser expressas nos valores de retorno das chamadas e em notas de resultado, dispensando setas de retorno redundantes. As sequências completas devem ser lidas junto ao fluxo textual e suas alternativas. Os recortes SQ01-A a SQ01-C e SQ11-A a SQ11-F relacionam uma parte da interação ao arquivo e às linhas reais da implementação.

O cenário principal, os caminhos alternativos e as falhas previstas precisam concordar com a especificação. Um retorno de erro não comprova sucesso, uma prévia não concede recompensa de submissão e uma repetição não deve ser documentada como novo crédito de XP. Essas são regras da implementação conferidas separadamente da referência bibliográfica.

Uso no seminário

A sequência de exposição segue a orientação da professora: protótipo, RF/RNF/RN, modelos estruturais, sequência completa, recortes com código e demonstração prática. A referência de Guedes fundamenta os modelos; a ordem, o tempo e a divisão dos integrantes provêm da orientação do seminário. Os diagramas de atividades previamente aprovados permanecem preservados.

Catálogo acadêmico de casos de uso

Esta é a numeração canônica dos diagramas e do seminário: 24 casos de uso, todos presentes na visão geral. Os números remanescentes da entrega acadêmica foram preservados para manter as referências, por isso não formam uma sequência sem lacunas. Operações relacionadas foram reunidas em objetivos e variantes; números de casos absorvidos não representam casos adicionais. A tabela de consolidação permite rastrear a alteração.

Critério baseado em Guedes, seção 3.4 e capítulo 3: caso de uso representa um objetivo ou um comportamento reutilizado relevante; rotas, telas, validação de sessão, persistência, sincronização e execução de um cliente mobile não geram automaticamente casos independentes. UC012 é uma

extensão opcional solicitada pelo usuário; UC028 e UC029 são comportamentos reutilizados da avaliação, mantidos para explicitar include e extend. Os três são contabilizados nos 24 casos.

Os cenários Cxx do caderno de implementação continuam como recortes técnicos e não são contados como casos de uso acadêmicos. O escopo desta modelagem exclui o Mundinho por solicitação do grupo e não inclui comunidades/guilds. O jogo permanece explicado no manual técnico, sem introduzir casos no diagrama geral. Cadastro, login e recuperação usam Usuário e podem ocorrer sem sessão; as ações protegidas exigem autenticação segundo suas pré-condições.

ID	Caso de uso	Área
UC001	Cadastrar conta	Conta e acesso
UC002	Fazer login	Conta e acesso
UC003	Gerenciar perfil e preferências	Perfil
UC004	Consultar e selecionar trilha	Trilhas
UC005	Estudar lição	Trilhas
UC006	Resolver exercício	Trilhas
UC007	Consultar evolução e reconhecimento	Trilhas
UC008	Disputar duelo de programação	Duelos
UC010	Enviar solução de duelo	Duelos
UC012	Consultar ajuda do exercício	Trilhas
UC013	Gerenciar publicação própria	Comunidade
UC014	Consultar e interagir com a comunidade	Comunidade
UC015	Gerenciar eventos e desafios	Eventos
UC016	Avaliar duelo autorizado	Avaliadores
UC017	Gerenciar empresa e vagas	Recrutamento
UC019	Administrar usuários e recursos	Administração
UC021	Acompanhar notificações	Recursos pessoais
UC022	Candidatar-se a vaga	Recrutamento
UC023	Participar de evento	Eventos
UC024	Solicitar credenciamento de avaliador	Avaliadores
UC026	Gerenciar candidaturas	Recrutamento
UC028	Avaliar resposta	Trilhas
UC029	Executar código e testes	Trilhas

ID	Caso de uso	Área
UC038	Trocar mensagens privadas	Recursos pessoais

UC001 — Cadastrar conta

Objetivo: Cadastrar conta.

Ator principal: Usuário.

Fluxo principal

Descrição: Criar uma conta individual para acessar funções autenticadas.

Tipo: Condução

Ator iniciador ou caso chamador: Usuário

Atores secundários: Provedor de identidade

Pré-condições: Usuário sem sessão autenticada. A disponibilidade do e-mail e do nome de usuário é verificada durante o fluxo.

Pós-condições: Identidade e perfil inicial criados; o usuário pode fazer login.

Entradas: Username, e-mail, senha e campos opcionais permitidos.

Saídas: Confirmação de cadastro ou erros por campo.

Fluxo Principal

1. O usuário abre a tela de cadastro e informa os dados solicitados.

2. O Stacklyst valida formato, força da senha e unicidade.

3. O sistema solicita ao serviço de autenticação a criação da identidade.

4. Após confirmação, o Stacklyst cria o registro local e perfil inicial.

5. O sistema confirma o cadastro e direciona o usuário ao acesso apropriado.

Fluxos Alternativos

• A1 — Dado inválido: o sistema destaca o campo sem criar a conta.

• A2 — E-mail ou username existente: o sistema recusa duplicidade.

• A3 — Serviço indisponível: o sistema informa falha temporária sem expor detalhes.

Implementação: src/app/api/auth/register/route.ts.

Cenários técnicos relacionados: C01.

Regras relacionadas: R01, R02.

UC002 — Fazer login

Objetivo: Fazer login.

Ator principal: Usuário.

Fluxo principal

Descrição: Permitir que o usuário faça login e inicie uma sessão com as permissões da sua conta.

Tipo: Condução

Ator iniciador ou caso chamador: Usuário

Atores secundários: Provedor de identidade

Pré-condições: Conta existente e não bloqueada.

Pós-condições: Sessão válida criada; rotas autorizadas ficam disponíveis.

Entradas: Credencial ou provedor OAuth.

Saídas: Sessão, perfil básico e redirecionamento.

Fluxo Principal

1. O usuário abre a tela de login e informa suas credenciais ou seleciona um provedor de identidade.

2. O Stacklyst envia a solicitação ao serviço de autenticação.

3. O serviço verifica a identidade e retorna o resultado.

4. O Stacklyst cria/atualiza cookies de sessão e carrega o papel.

5. O sistema redireciona para a área permitida.

Fluxos Alternativos

• A1 — Credencial inválida: o sistema mantém a sessão encerrada.

• A2 — Provedor OAuth não configurado: o sistema informa indisponibilidade.

• A3 — Conexão temporária: o sistema permite nova tentativa sem revelar segredo.

Variante 1 — Encerrar sessão

Descrição: Encerrar a sessão atual e retornar à área pública.

Tipo: Condução

Ator iniciador: Usuário autenticado, incluindo Avaliador, Administrador e Recrutador.

Atores secundários: Provedor de identidade.

Pré-condições: Usuário com sessão ativa.

Pós-condições: Sessão encerrada e estado local de autenticação removido; novas operações protegidas exigem autenticação.

Entradas: Solicitação de saída e sessão atual.

Saídas: Confirmação de encerramento ou informação de falha.

Fluxo Principal

1. O usuário solicita sair da conta.

2. O Stacklyst solicita o encerramento da sessão ao provedor de identidade.

3. Após confirmação, o sistema remove o estado local de autenticação.

4. O sistema apresenta a área pública e exige novo login para as funções protegidas.

Fluxos Alternativos

• A1 — Sessão já encerrada: o sistema apresenta a área pública.

• A2 — Falha ao encerrar a sessão: o sistema informa a falha, sem confirmar sucesso, e permite nova tentativa.

Rastreabilidade: RF003; RN017 e RN025. Referência de interface: ação Sair da conta já existente. Representado nas vistas UC00 e UC01 do projeto Astah.

Variante 2 — Confirmar e recuperar acesso

Descrição: Confirmar e recuperar acesso.

Ator principal: Titular da conta

Pré-condições: Token ou pedido de recuperação correspondente à própria conta

Pós-condições: Titular recebe resultado do fluxo, sem expor senha anterior

Fluxo principal

1. Solicitar confirmação ou recuperação

2. Processar o link no fluxo autorizado

3. Conferir token e dados necessários

4. Concluir acesso ou alteração de senha conforme resultado

Fluxos alternativos: Token inválido/expirado e falha do provedor devem ter tratamento próprio

Implementação: src/app/api/auth/reset-password/route.ts; src/lib/auth.ts.

Cenários técnicos relacionados: C02, C03.

Regras relacionadas: R01, R02, R03.

UC003 — Gerenciar perfil e preferências

Objetivo: Gerenciar perfil e preferências.

Ator principal: Usuário.

Fluxo principal

Descrição: Consultar e atualizar dados permitidos do próprio perfil.

Tipo: Configuração

Ator iniciador ou caso chamador: Usuário autenticado

Atores secundários: Nenhum.

Pré-condições: Usuário autenticado.

Pós-condições: Campos válidos persistidos e perfil público atualizado.

Entradas: Bio, avatar, banner, pronomes e demais campos suportados.

Saídas: Perfil atualizado ou erros de validação.

Fluxo Principal

1. O usuário acessa o próprio perfil ou as configurações.

2. O Stacklyst exibe os dados editáveis e a visualização atual.

3. O usuário altera campos e confirma.

4. O sistema valida propriedade, formato e limites e persiste os dados.

5. O Stacklyst recarrega o perfil e confirma a atualização.

Fluxos Alternativos

• A1 — Usuário sem permissão: o sistema nega alteração em perfil alheio.

• A2 — Campo inválido: o sistema preserva dados anteriores.

• A3 — Upload indisponível: o sistema mantém a imagem anterior.

Variante 1 — Consultar perfil público

Descrição: Consultar perfil público.

Ator iniciador: Usuário autenticado.

Pré-condições e autorização: Dados privados de conta não compõem a resposta pública.

Fluxo Principal

1. Selecionar usuário.

2. Consultar perfil e indicadores públicos.

Pós-condições: Resultado da consulta apresentado ou alteração autorizada confirmada ao usuário, conforme a operação solicitada.

Fluxos Alternativos

• A1 — Sessão ou permissão insuficiente: recusar acesso à operação protegida e informar a condição, sem aplicar a alteração solicitada.

• A2 — Dados inválidos, recurso indisponível ou falha de processamento: informar o resultado e permitir a correção ou nova tentativa; não apresentar sucesso sem confirmação.

Rastreabilidade: RF037, RF038.

Referência de implementação: src/app/profile/[username]; src/app/api/profile/[username].

Variante 2 — Seguir ou deixar de seguir usuário

Descrição: Seguir ou deixar de seguir usuário.

Ator iniciador: Usuário autenticado.

Pré-condições e autorização: A ação usa a identidade autenticada e as regras de FollowService.

Fluxo Principal

1. Selecionar usuário.

2. Alternar acompanhamento.

3. Confirmar estado.

Pós-condições: Resultado da consulta apresentado ou alteração autorizada confirmada ao usuário, conforme a operação solicitada.

Fluxos Alternativos

• A1 — Sessão ou permissão insuficiente: recusar acesso à operação protegida e informar a condição, sem aplicar a alteração solicitada.

• A2 — Dados inválidos, recurso indisponível ou falha de processamento: informar o resultado e permitir a correção ou nova tentativa; não apresentar sucesso sem confirmação.

Rastreabilidade: Ampliação: relações sociais.

Referência de implementação: src/app/explore/page.tsx; src/app/api/users/[id]/follow.

Variante 3 — Configurar preferências pessoais

Descrição: Configurar preferências pessoais.

Ator iniciador: Usuário autenticado.

Pré-condições e autorização: Preferências locais não representam mudança de permissão no servidor.

Fluxo Principal

1. Abrir configurações.

2. Alterar aparência, sons ou idioma suportado.

3. Aplicar preferência.

Pós-condições: Resultado da consulta apresentado ou alteração autorizada confirmada ao usuário, conforme a operação solicitada.

Fluxos Alternativos

• A1 — Sessão ou permissão insuficiente: recusar acesso à operação protegida e informar a condição, sem aplicar a alteração solicitada.

• A2 — Dados inválidos, recurso indisponível ou falha de processamento: informar o resultado e permitir a correção ou nova tentativa; não apresentar sucesso sem confirmação.

Rastreabilidade: Ampliação: preferências.

Referência de implementação: src/app/settings/page.tsx.

Variante 4 — Personalizar avatar

Descrição: Personalizar avatar.

Ator iniciador: Usuário autenticado.

Pré-condições e autorização: A alteração pertence à própria conta.

Fluxo Principal

1. Escolher características do avatar.

2. Salvar configuração.

3. Visualizar perfil atualizado.

Pós-condições: Resultado da consulta apresentado ou alteração autorizada confirmada ao usuário, conforme a operação solicitada.

Fluxos Alternativos

• A1 — Sessão ou permissão insuficiente: recusar acesso à operação protegida e informar a condição, sem aplicar a alteração solicitada.

• A2 — Dados inválidos, recurso indisponível ou falha de processamento: informar o resultado e permitir a correção ou nova tentativa; não apresentar sucesso sem confirmação.

Rastreabilidade: RF004.

Referência de implementação: src/app/avatar/AvatarCustomizer.tsx; src/app/api/avatar.

Variante 5 — Bloquear conta

Descrição: Bloquear conta.

Ator principal: Usuário

Pré-condições: Identidade do alvo e sessão válidas

Pós-condições: Bloqueio influencia consultas e interações

Fluxo principal

1. Solicitar bloqueio

2. Aplicar regra de interação

3. Persistir relação de bloqueio

4. Refletir restrições nos fluxos afetados

Fluxos alternativos: Ocultar um botão sem filtrar as APIs não implementa a regra

Variante 6 — Enviar imagem compatível

Descrição: Enviar imagem compatível.

Ator principal: Usuário

Pré-condições: Sessão e arquivo aceito pela rota de upload

Pós-condições: Referência de mídia após envio processado

Fluxo principal

1. Enviar multipart

2. Conferir tamanho e tipo

3. Validar assinatura do arquivo e processamento

4. Persistir no armazenamento e devolver referência

Fluxos alternativos: MIME declarado não basta; esta rota aceita raster e limite próprio, não todo tipo de anexo

Implementação: src/app/api/upload/route.ts; src/app/settings/page.tsx; src/services/block.service.ts; src/services/follow.service.ts; src/services/profile-customization.service.ts.

Cenários técnicos relacionados: C22, C23, C24, C28, C54.

Regras relacionadas: R01, R02, R03, R05, R06, R28, R93.

UC004 — Consultar e selecionar trilha

Objetivo: Consultar e selecionar trilha.

Ator principal: Usuário.

Fluxo principal

Descrição: Selecionar uma trilha e iniciar ou retomar o percurso.

Tipo: Condução

Ator iniciador ou caso chamador: Usuário autenticado

Atores secundários: Nenhum.

Pré-condições: Usuário autenticado; trilha disponível.

Pós-condições: Preferência registrada e lição elegível apresentada.

Entradas: Linguagem/trilha e etapa selecionada.

Saídas: Mapa atualizado e acesso à próxima lição.

Fluxo Principal

1. O usuário abre Trilhas e consulta as opções.

2. O Stacklyst carrega progresso, níveis e bloqueios.

3. O usuário seleciona a trilha desejada.

4. O sistema registra a preferência e determina a etapa elegível.

5. O Stacklyst abre a lição ou apresenta o próximo passo.

Fluxos Alternativos

• A1 — Trilha indisponível: o sistema impede o início e informa o motivo.

• A2 — Pré-requisito não cumprido: o sistema mantém a etapa bloqueada.

• A3 — Progresso não carregado: o sistema não sobrescreve dados existentes.

Variante 1 — Consultar disponibilidade de conhecimentos

Descrição: Consultar disponibilidade de conhecimentos.

Ator principal: Estudante

Pré-condições: Nós, caminhos e relações carregados

Pós-condições: Estados concluídos legítimos são preservados

Fluxo principal

1. Carregar estados persistidos

2. Consultar requisitos obrigatórios

3. Derivar disponibilidade/recomendação

4. Calcular percentual e próximo nó

Fluxos alternativos: Caminho vazio retorna percentual seguro; requisito ausente não é ignorado por recomendação

Implementação: src/app/trails/TrailsContent.tsx; src/lib/learning/progress.ts.

Cenários técnicos relacionados: C04, C05.

Regras relacionadas: R08, R09, R10.

UC005 — Estudar lição

Objetivo: Estudar lição.

Ator principal: Usuário.

Fluxo principal

Descrição: Carregar uma lição, validar o acesso e apresentar seu conteúdo, suas etapas interativas e o progresso à interface.

Tipo: Condução

Ator iniciador ou caso chamador: Usuário autenticado

Atores secundários: Nenhum.

Pré-condições: Usuário autenticado; lição existente e elegível.

Pós-condições: Conteúdo e progresso da lição apresentados após a validação do acesso; exercícios podem ser realizados.

Entradas: Identificador da lição.

Saídas: Conteúdo, etapas e progresso da sessão.

Fluxo Principal

1. O usuário escolhe uma lição liberada na trilha.

2. O Stacklyst resolve o identificador no registro de lições.

3. O sistema verifica conteúdo e estado da etapa.

4. A página apresenta explicação, exemplos e interação correspondente.

5. O usuário navega pelas etapas e decide resolver o exercício ou pedir ajuda opcional.

Fluxos Alternativos

• A1 — Lição inexistente: o sistema apresenta estado de não encontrada.

• A2 — Lição bloqueada: o sistema orienta o pré-requisito.

• A3 — Falha ao consultar progresso: informar indisponibilidade sem confirmar conclusão.

Variante 1 — Resgatar etapa especial da trilha

Descrição: Resgatar etapa especial da trilha.

Ator principal: Estudante

Pré-condições: Requisitos do resgate correspondente satisfeitos

Pós-condições: Resgate associado à regra específica

Fluxo principal

1. Solicitar checkpoint, baú ou salto pela rota própria

2. Conferir evidências exigidas

3. Registrar recibo idempotente

4. Retornar benefício confirmado

Fluxos alternativos: Não usar validação de quiz genérico para responder um recibo de recompensa

Implementação: src/app/api/trails/chest/route.ts; src/lib/learning/lesson-progress.ts.

Cenários técnicos relacionados: C06, C13.

Regras relacionadas: R11, R12, R14, R20, R21.

UC006 — Resolver exercício

Objetivo: Resolver exercício.

Ator principal: Usuário.

Fluxo principal

Descrição: Elaborar e submeter uma solução para avaliação no servidor, incluindo UC028 Avaliar resposta. Ponto de extensão consultar ajuda: durante a elaboração, UC012 pode ser executado se o usuário solicitar ajuda permitida pelo modo. Testar código antes da submissão, em UC006, não concede XP nem conclui a etapa.

Tipo: Condução

Ator iniciador ou caso chamador: Usuário autenticado

Atores secundários: Executor de código, por meio de UC029 quando a atividade exigir programação.

Pré-condições: Exercício disponível, pré-requisitos atendidos e usuário autenticado para registrar a tentativa.

Pós-condições: Tentativa e feedback registrados quando a gravação for concluída; XP e progresso atualizados somente se elegíveis. Em falha técnica, nenhum novo acerto é confirmado.

Entradas: Resposta, código, pares, ordem, lacunas, comando ou blocos.

Saídas: Resultado, feedback, saída e XP quando aplicável.

Fluxo Principal

1. O usuário elabora a solução. Durante esta etapa, pode consultar ajuda pelo ponto de extensão consultar ajuda.

2. O usuário submete a resposta; o Stacklyst verifica completude, acesso e pré-requisitos.

3. O Stacklyst executa UC028 Avaliar resposta, que trata o tipo da atividade e retorna o resultado.

4. O sistema registra a tentativa e, em acerto inédito elegível, a recompensa e o progresso na mesma operação.

5. O Stacklyst apresenta feedback e progresso confirmado; o usuário decide prosseguir ou realizar nova tentativa.

Fluxos Alternativos

• A1 — Dado inválido: o sistema solicita correção antes da avaliação.

• A2 — Executor indisponível ou timeout: o sistema não marca acerto nem concede XP.

• A3 — Resposta incorreta: o sistema exibe feedback e permite nova tentativa.

• A4 — Falha ao salvar: o sistema informa indisponibilidade, preserva a solução e permite reenvio sem duplicar recompensa.

Variante 1 — Testar código antes da submissão

Descrição: Testar código antes da submissão.

Ator iniciador: Usuário autenticado.

Pré-condições e autorização: Execução de teste não implica conclusão nem concessão de XP.
Revisão de 3 de novembro de 2026 17

Fluxo Principal

1. Preparar código.

2. Solicitar execução de teste.

3. Analisar saída e ajustar solução.

Pós-condições: Resultado da consulta apresentado ou alteração autorizada confirmada ao usuário, conforme a operação solicitada.

Fluxos Alternativos

• A1 — Sessão ou permissão insuficiente: recusar acesso à operação protegida e informar a condição, sem aplicar a alteração solicitada.

• A2 — Dados inválidos, recurso indisponível ou falha de processamento: informar o resultado e permitir a correção ou nova tentativa; não apresentar sucesso sem confirmação.

Rastreabilidade: RF008, RF019.

Referência de implementação: src/app/api/exercises/[exerciseId]/run; src/app/api/duels/[id]/run.

Implementação: src/app/api/lessons/[lessonId]/attempt/route.ts; src/lib/duels/judge.ts; src/lib/exercises/repository.ts; src/lib/learning/assessment.ts; src/lib/learning/lesson-progress.ts.

Cenários técnicos relacionados: C07, C08, C09, C10, C11, C37.

Regras relacionadas: R12, R13, R14, R15, R16, R17, R18, R19, R25, R63, R64.

UC007 — Consultar evolução e reconhecimento

Objetivo: Consultar evolução e reconhecimento.

Ator principal: Usuário.

Fluxo principal

Descrição: Apresentar XP, nível, sequência, trilhas, tentativas e conquistas permitidas.

Tipo: Análise

Ator iniciador ou caso chamador: Usuário autenticado

Atores secundários: Nenhum.

Pré-condições: Sessão válida para consultar o próprio progresso. A consulta do progresso de outro usuário limita-se às informações de seu perfil público.

Pós-condições: Nenhuma alteração obrigatória; visão consolidada exibida.

Entradas: Usuário, período e trilha opcionais.

Saídas: Indicadores e histórico permitido.

Fluxo Principal

1. O usuário abre perfil, trilha ou visão de atividade.

2. O Stacklyst identifica o titular e as permissões de visualização.

3. O sistema consulta XP, trilhas, sequência, tentativas e badges.

4. Os dados são consolidados sem expor campos privados.

5. O Stacklyst apresenta evolução atual e próximos marcos.

Fluxos Alternativos

• A1 — Perfil privado ou dado não permitido: o sistema oculta o campo.

• A2 — Histórico indisponível: o sistema mostra o último estado confirmado.

• A3 — Período sem atividade: o sistema retorna estado vazio claro.

Variante 1 — Consultar ranking de XP

Descrição: Exibir classificação global ou por linguagem.

Tipo: Análise

Ator iniciador ou caso chamador: Usuário autenticado

Atores secundários: Nenhum.

Pré-condições: Dados de XP disponíveis.

Pós-condições: Ranking exibido sem alterar pontuação.

Entradas: Filtro de linguagem opcional.

Saídas: Posição, usuário, XP, nível e divisão quando aplicável.

Fluxo Principal

1. O usuário abre o ranking.

2. O Stacklyst recebe o filtro global ou por linguagem.

3. O sistema consulta os maiores valores de XP no escopo.

4. A posição e o nível exibido são calculados.

5. O Stacklyst apresenta a classificação e a posição disponível.

Fluxos Alternativos

• A1 — Filtro inválido: o sistema usa opção válida ou informa erro.

• A2 — Ranking vazio: o sistema exibe estado sem participantes.

• A3 — Empate em XP: ordenar por username e id crescentes.

Variante 2 — Consultar e emitir credencial elegível

Descrição: Consultar e emitir credencial elegível.

Ator principal: Estudante

Pré-condições: Evidências pedagógicas registradas

Pós-condições: Credencial vinculada às evidências e versão

Fluxo principal

1. Consultar metas
2. Reconciliar requisitos em contexto autorizado
3. Emitir snapshot elegível uma vez
4. Mostrar coleção e validade

Fluxos alternativos: XP ou estudo declarado não substitui requisito; leitura pública não emite

Variante 3 — Verificar credencial pública

Descrição: Verificar credencial pública.

Ator principal: Usuário

Pré-condições: Identificador de credencial

Pós-condições: Verificação mostra o estado do registro

Fluxo principal

1. Consultar identificador
2. Carregar registro e snapshot
3. Conferir integridade e revogação
4. Apresentar validade

Fluxos alternativos: ID inválido, ausência, revogação e falha de integridade não são certificados válidos

Variante 4 — Gerar PDF de credencial

Descrição: Gerar PDF de credencial.

Ator principal: Titular ou usuário sem sessão autorizado pelo fluxo

Pré-condições: Credencial disponível para consulta/geração

Pós-condições: PDF corresponde ao registro consultável

Fluxo principal

1. Solicitar o documento
2. Carregar definição e dados válidos
3. Compor PDF, assinatura visual e QR
4. Devolver documento

Fluxos alternativos: A aparência da assinatura não é uma assinatura criptográfica nem mede horas reais

Implementação: src/lib/credentials/pdf.ts; src/lib/credentials/repository.ts; src/lib/learning/rewards.ts.

Cenários técnicos relacionados: C14, C15, C16, C25.

Regras relacionadas: R22, R23, R24, R25, R26, R27, R28.

UC008 — Disputar duelo de programação

Objetivo: Disputar duelo de programação.

Ator principal: Usuário.

Fluxo principal

Descrição: Disponibilizar um duelo aberto com uma linguagem e um desafio do catálogo, aguardando a entrada de outro usuário. Se já houver uma vaga compatível, o sistema pode iniciar o confronto nessa vaga.

Tipo: Condução

Ator iniciador ou caso chamador: Usuário autenticado

Atores secundários: Nenhum. O segundo participante entra no duelo por meio de UC008.

Pré-condições: Usuário autenticado; linguagem suportada e desafio verificável do catálogo.

Pós-condições: Duelo aberto no estado PENDING, vinculado ao criador e sem oponente, ou duelo compatível iniciado no estado ACTIVE.

Entradas: Linguagem e desafio do catálogo. Não há destinatário a selecionar.

Saídas: Identificador do duelo, estado e prazo de espera por participante.

Fluxo Principal

1. O usuário abre a área de duelos, solicita criar um duelo e escolhe linguagem e desafio.

2. O sistema verifica a sessão, a linguagem suportada e a existência do desafio no catálogo.

3. O sistema consulta se há uma vaga aberta de outro usuário na mesma linguagem; se houver, segue a alternativa A1.

4. Sem vaga compatível, o sistema registra o usuário como criador de um duelo PENDING, sem oponente, com prazo de espera de até 24 horas.

5. O sistema exibe o duelo aberto. O confronto começa quando outro usuário ocupa a vaga por UC008, com duração de duas horas.

Fluxos Alternativos

• A1 — Vaga compatível encontrada: o sistema reserva a vaga, registra o usuário como oponente e inicia o duelo existente, utilizando o desafio dessa vaga.

• A2 — Dados ou desafio inválidos: o sistema informa o erro e solicita correção antes de criar o duelo.

• A3 — Sessão ausente ou expirada: o usuário precisa fazer login para concluir a solicitação.

• A4 — Espera expirada: a vaga deixa de ser elegível para pareamento; a manutenção atualiza o estado do duelo conforme sua execução.

Variante 1 — Entrar em duelo

Descrição: Entrar voluntariamente em um duelo aberto ou usar a fila rápida para localizar uma vaga na linguagem selecionada.

Tipo: Condução

Ator iniciador ou caso chamador: Usuário autenticado

Atores secundários: Usuário autenticado que criou o duelo, no contexto de outro participante.

Pré-condições: Usuário autenticado; para entrada direta, duelo pendente com vaga de oponente disponível e criado por outro usuário.

Pós-condições: Vaga ocupada uma única vez e duelo ACTIVE, ou nova espera PENDING quando a fila rápida não encontra oponente.

Entradas: Duelo escolhido na entrada direta ou linguagem selecionada na fila rápida.

Saídas: Duelo ativo com participantes e prazo, ou confirmação de espera por oponente.

Fluxo Principal

1. O usuário seleciona um duelo aberto e solicita entrar.

2. O sistema verifica a sessão e confirma que o usuário não é o criador desse duelo.

3. O sistema verifica se o duelo continua pendente e com a vaga de oponente disponível.

4. O sistema reserva a vaga de forma condicional para impedir que dois usuários a ocupem ao mesmo tempo.

5. O sistema registra o oponente, ativa o duelo e apresenta o desafio e o prazo de duas horas aos participantes.

Fluxos Alternativos

• A1 — Fila rápida: o usuário escolhe JavaScript, TypeScript ou Python; o sistema busca a vaga pendente mais antiga e disponível na mesma linguagem, criada por outro usuário. Se não houver vaga, cria uma espera com desafio do catálogo e prazo de até 24 horas.

• A2 — Vaga ocupada ou duelo indisponível: o sistema informa que a entrada não pode ser concluída. Na fila rápida, pode buscar outra vaga ou criar uma espera.

• A3 — Próprio duelo, sessão ausente ou linguagem não suportada: o sistema recusa a entrada e informa a condição a corrigir.

Variante 2 — Consultar resultado de duelo

Descrição: Consultar resultado de duelo.

Ator iniciador: Usuário autenticado.

Pré-condições e autorização: Duelo ativo restringe acesso a participantes, avaliadores e administradores.

Fluxo Principal

1. Abrir duelo permitido.

2. Consultar estado, envios próprios e parecer disponível.

Pós-condições: Resultado da consulta apresentado ou alteração autorizada confirmada ao usuário, conforme a operação solicitada.

Fluxos Alternativos

• A1 — Sessão ou permissão insuficiente: recusar acesso à operação protegida e informar a condição, sem aplicar a alteração solicitada.

• A2 — Dados inválidos, recurso indisponível ou falha de processamento: informar o resultado e permitir a correção ou nova tentativa; não apresentar sucesso sem confirmação.

Rastreabilidade: RF020, RF021.

Referência de implementação: src/app/api/duels/[id]/route.ts; src/app/duels/[id].

Variante 3 — Excluir duelo elegível

Descrição: Excluir duelo elegível.

Ator iniciador: Criador do duelo ou Administrador autorizado.

Pré-condições e autorização: Somente criador ou Administrador. Estados permitidos: PENDING, EXPIRED e CLOSED.

Fluxo Principal

1. Criador seleciona duelo.

2. Solicita exclusão.

3. Servidor valida estado e propriedade.

Pós-condições: Resultado da consulta apresentado ou alteração autorizada confirmada ao usuário, conforme a operação solicitada.

Fluxos Alternativos

• A1 — Sessão ou permissão insuficiente: recusar acesso à operação protegida e informar a condição, sem aplicar a alteração solicitada.

• A2 — Dados inválidos, recurso indisponível ou falha de processamento: informar o resultado e permitir a correção ou nova tentativa; não apresentar sucesso sem confirmação.

Rastreabilidade: Ampliação: manutenção de duelo.

Referência de implementação: src/app/api/duels/[id]/route.ts.

Implementação: src/lib/duels/resolution-policy.ts; src/services/duel.service.ts.

Cenários técnicos relacionados: C36, C38.

Regras relacionadas: R61, R62, R64, R65, R66, R67, R68, R69, R70, R71.

UC010 — Enviar solução de duelo

Objetivo: Enviar solução de duelo.

Ator principal: Usuário.

Fluxo principal

Descrição: Executar e registrar solução para resolução do duelo.

Tipo: Condução

Ator iniciador ou caso chamador: Participante do duelo

Atores secundários: Executor de código

Pré-condições: Participante autenticado, duelo ativo e dentro do prazo.

Pós-condições: Solução registrada e resolução do duelo processada quando houver elementos suficientes; empate técnico permanece pendente de avaliação humana.

Entradas: Código e duelo.

Saídas: Resultados de testes e estado do duelo.

Fluxo Principal

1. Enviar código.

2. Verificar sessão, participante, estado e prazo.

3. O Stacklyst executa UC029 Executar código e testes com os testes públicos e privados do desafio.

4. Salvar submissão e aplicar política de resolução.

5. Retornar resultado sem revelar testes privados.

Fluxos Alternativos

• A1 — Fora do prazo ou não participante: negar submissão.

• A2 — Executor indisponível: não conceder vitória.

• A3 — Empate técnico: encaminhar para revisão humana.

Implementação: src/lib/duels/judge.ts.

Cenários técnicos relacionados: C37.

Regras relacionadas: R63, R64.

UC012 — Consultar ajuda do exercício

Objetivo: Consultar ajuda do exercício.

Ator principal: Usuário.

Fluxo principal

Descrição: Consultar material de apoio durante UC006, no ponto de extensão consultar ajuda, quando houver solicitação do usuário e o modo permitir.

Tipo: Análise

Ator iniciador ou caso chamador: Usuário autenticado

Atores secundários: Nenhum.

Pré-condições: Exercício carregado.

Pós-condições: Material exibido sem recompensa adicional.

Entradas: Modo e tipo de ajuda.

Saídas: Dica ou documentação.

Fluxo Principal

1. Abrir um exercício e selecionar o modo de assistência.

2. Verificar se o modo permite a ajuda solicitada.

3. Abrir dica ou documentação editorial.

4. O Stacklyst apresenta o material editorial correspondente à solicitação.

5. O usuário interpreta a orientação e retorna à elaboração da solução em UC006.

Fluxos Alternativos

• A1 — Modo bloqueia ajuda: trocar modo ou continuar sem ajuda.

Implementação: src/lib/exercises/repository.ts.

Cenários técnicos relacionados: C12.

Regras relacionadas: R18, R19.

UC013 — Gerenciar publicação própria

Objetivo: Gerenciar publicação própria.

Ator principal: Usuário.

Fluxo principal

Descrição: Publicar dúvida, conteúdo ou código no feed, gravando a publicação e a recompensa de XP na mesma transação.

Tipo: Condução

Ator iniciador ou caso chamador: Usuário autenticado

Atores secundários: Nenhum.

Pré-condições: Usuário autenticado e dentro do limite de publicação.

Pós-condições: Publicação e recompensa de XP persistidas na mesma transação; publicação exibida no feed após a confirmação.

Entradas: Título, corpo, linguagem, código e imagem opcional.

Saídas: Publicação criada ou erros de validação.

Fluxo Principal

1. O usuário abre o compositor do feed.

2. O usuário informa o conteúdo e confirma a publicação.

3. O Stacklyst valida autenticação, tamanho, formato e limite.

4. O sistema persiste a publicação, seus metadados e a recompensa de XP na mesma transação.

5. O feed é atualizado e a publicação fica disponível para interação.

Fluxos Alternativos

• A1 — Conteúdo inválido ou vazio: o sistema solicita correção.

• A2 — Limite excedido: o sistema recusa temporariamente.

• A3 — Upload falha: o usuário pode corrigir ou publicar sem a mídia, se permitido.

Variante 1 — Editar ou excluir publicação própria

Descrição: Editar ou excluir publicação própria.

Ator iniciador: Usuário autenticado.

Pré-condições e autorização: O servidor valida author_id; edição exige conteúdo com pelo menos 10 caracteres.

Fluxo Principal

1. Selecionar publicação própria.

2. Solicitar edição ou exclusão.

3. Confirmar resultado.

Pós-condições: Resultado da consulta apresentado ou alteração autorizada confirmada ao usuário, conforme a operação solicitada.

Fluxos Alternativos

• A1 — Sessão ou permissão insuficiente: recusar acesso à operação protegida e informar a condição, sem aplicar a alteração solicitada.

• A2 — Dados inválidos, recurso indisponível ou falha de processamento: informar o resultado e permitir a correção ou nova tentativa; não apresentar sucesso sem confirmação.

Rastreabilidade: Ampliação: manutenção de publicação.

Referência de implementação: src/app/api/posts/[id]/route.ts.

Implementação: src/services/post.service.ts.

Cenários técnicos relacionados: C17.

Regras relacionadas: R29, R30.

UC014 — Consultar e interagir com a comunidade

Objetivo: Consultar e interagir com a comunidade.

Ator principal: Usuário.

Fluxo principal

Descrição: Reagir, votar, responder, salvar ou denunciar uma publicação.

Tipo: Condução

Ator iniciador ou caso chamador: Usuário autenticado

Atores secundários: Autor da publicação

Pré-condições: Usuário autenticado; publicação existente.

Pós-condições: Interação registrada e contadores/notificações atualizados.

Entradas: Publicação, ação, reação ou resposta.

Saídas: Novo estado da interação e eventual notificação.

Fluxo Principal

1. O usuário seleciona uma publicação no feed.

2. O usuário escolhe a interação permitida e informa dados adicionais quando necessário.

3. O Stacklyst valida alvo, sessão e duplicidade.

4. O sistema registra ou alterna a interação e atualiza contadores.

5. O autor recebe notificação quando a regra prevê.

Fluxos Alternativos

• A1 — Publicação removida: o sistema interrompe a ação.

• A2 — Ação duplicada: o sistema mantém unicidade ou alterna o estado.

• A3 — Conteúdo denunciado: o sistema cria item para moderação.

Variante 1 — Consultar e pesquisar feed

Descrição: Consultar e pesquisar feed.

Ator iniciador: Usuário autenticado.

Pré-condições e autorização: Exibir somente o conteúdo autorizado.

Fluxo Principal

1. Informar filtro ou termo.

2. Consultar publicações e abrir conteúdo de interesse.

Pós-condições: Resultado da consulta apresentado ou alteração autorizada confirmada ao usuário, conforme a operação solicitada.

Fluxos Alternativos

• A1 — Sessão ou permissão insuficiente: recusar acesso à operação protegida e informar a condição, sem aplicar a alteração solicitada.

• A2 — Dados inválidos, recurso indisponível ou falha de processamento: informar o resultado e permitir a correção ou nova tentativa; não apresentar sucesso sem confirmação.

Rastreabilidade: RF031.

Referência de implementação: src/app/feed; src/app/api/posts; src/app/api/search.

Variante 2 — Consultar publicações salvas

Descrição: Consultar publicações salvas.

Ator iniciador: Usuário autenticado.

Pré-condições e autorização: A coleção pertence ao usuário autenticado.

Fluxo Principal

1. Abrir itens salvos.

2. Selecionar publicação.

3. Remover marcação se desejado.

Pós-condições: Resultado da consulta apresentado ou alteração autorizada confirmada ao usuário, conforme a operação solicitada.

Fluxos Alternativos

• A1 — Sessão ou permissão insuficiente: recusar acesso à operação protegida e informar a condição, sem aplicar a alteração solicitada.

• A2 — Dados inválidos, recurso indisponível ou falha de processamento: informar o resultado e permitir a correção ou nova tentativa; não apresentar sucesso sem confirmação.

Rastreabilidade: RF032.

Referência de implementação: src/app/bookmarks/page.tsx; src/app/api/posts/[id]/bookmark.

Variante 3 — Aceitar resposta de uma publicação

Descrição: Aceitar resposta de uma publicação.

Ator iniciador: Usuário autenticado.

Pré-condições e autorização: Somente autor da publicação; transação impede recompensa duplicada do mesmo aceite.

Fluxo Principal

1. Autor seleciona resposta.

2. Confirma aceite.

3. Sistema registra aceite e recompensa elegível.

Pós-condições: Resultado da consulta apresentado ou alteração autorizada confirmada ao usuário, conforme a operação solicitada.

Fluxos Alternativos

• A1 — Sessão ou permissão insuficiente: recusar acesso à operação protegida e informar a condição, sem aplicar a alteração solicitada.

• A2 — Dados inválidos, recurso indisponível ou falha de processamento: informar o resultado e permitir a correção ou nova tentativa; não apresentar sucesso sem confirmação.

Rastreabilidade: Ampliação: resposta aceita.

Referência de implementação: src/app/api/answers/[id]/accept/route.ts.

Variante 4 — Responder quiz diário

Descrição: Responder quiz diário.

Ator iniciador: Usuário autenticado.

Pré-condições e autorização: A correção ocorre no servidor; repetição não duplica a primeira recompensa.

Fluxo Principal

1. Consultar quiz disponível.

2. Selecionar resposta.

3. Enviar e analisar resultado.

Pós-condições: Resultado da consulta apresentado ou alteração autorizada confirmada ao usuário, conforme a operação solicitada.

Fluxos Alternativos

• A1 — Sessão ou permissão insuficiente: recusar acesso à operação protegida e informar a condição, sem aplicar a alteração solicitada.

• A2 — Dados inválidos, recurso indisponível ou falha de processamento: informar o resultado e permitir a correção ou nova tentativa; não apresentar sucesso sem confirmação.

Rastreabilidade: RF008, RF009, RF011.

Referência de implementação: src/app/api/quiz/daily; src/app/api/quiz/[id]/attempt.

Variante 5 — Votar em enquete

Descrição: Votar em enquete.

Ator principal: Usuário

Pré-condições: Post acessível e enquete válida

Pós-condições: Voto pertence à opção correta e ao votante autorizado

Fluxo principal

1. Abrir enquete

2. Escolher opção

3. Validar voto e vínculo da opção

4. Persistir e apresentar contagem

Fluxos alternativos: Reações do post não são votos de enquete; consultar a rota para mudança/fechamento

Variante 6 — Denunciar publicação

Descrição: Denunciar publicação.

Ator principal: Usuário

Pré-condições: Post e motivo admitidos pelo fluxo

Pós-condições: Denúncia disponível para a gestão autorizada

Fluxo principal

1. Identificar conteúdo

2. Enviar denúncia

3. Conferir contrato e identidade

4. Registrar para consulta administrativa

Fluxos alternativos: Denunciar não concede ao denunciante direito de apagar a publicação

Implementação: src/app/api/posts/[id]/answer/route.ts; src/app/api/posts/[id]/poll/route.ts; src/app/api/posts/[id]/report/route.ts; src/app/bookmarks/page.tsx; src/services/daily-quiz.service.ts; src/services/post.service.ts; src/services/reaction.service.ts.

Cenários técnicos relacionados: C18, C19, C20, C21, C50, C52, C53.

Regras relacionadas: R02, R07, R12, R14, R29, R31, R32, R33, R34, R89, R91, R92.

UC015 — Gerenciar eventos e desafios

Objetivo: Gerenciar eventos e desafios.

Ator principal: Usuário.

Fluxo principal

Descrição: Permitir que qualquer usuário autenticado crie um evento comunitário com descrição, período e requisitos de participação, tornando-se o organizador responsável pelo cadastro.

Tipo: Configuração

Ator iniciador ou caso chamador: Usuário autenticado, no contexto de organizador do evento.

Atores secundários: Nenhum.

Pré-condições: Usuário autenticado e dados obrigatórios do evento preenchidos. A criação não exige o papel de Administrador ou Recrutador.

Pós-condições: Evento criado no estado UPCOMING e vinculado à conta que o cadastrou como criadora.

Entradas: Título, descrição, tipo, datas de início e término, nível mínimo, capacidade opcional, recompensa de XP e imagem opcional.

Saídas: Confirmação de criação e evento disponível na listagem com seu estado e requisitos.

Fluxo Principal

1. O usuário acessa Eventos e seleciona Criar evento.

2. O sistema verifica se há uma sessão válida e apresenta o formulário.

3. O usuário preenche a descrição, o tipo, o período e os requisitos de participação.

4. O formulário verifica os dados obrigatórios e a ordem das datas; o servidor valida os dados recebidos para a criação.

5. O sistema registra o evento com o usuário como criador, define o estado inicial UPCOMING e confirma a criação.

Fluxos Alternativos

• A1 — Sessão ausente ou expirada: o sistema solicita que o usuário faça login antes de concluir a criação.

• A2 — Campos obrigatórios ou datas inválidos no formulário: o sistema solicita a correção e mantém os dados para nova tentativa.

• A3 — Falha ao salvar o evento: o sistema informa a falha e não apresenta confirmação de criação.

Variante 1 — Consultar eventos e desafios

Descrição: Consultar eventos e desafios.

Ator iniciador: Usuário autenticado.

Pré-condições e autorização: Respostas esperadas dos desafios ficam restritas ao gestor autorizado.

Fluxo Principal

1. Consultar eventos.

2. Abrir detalhes e desafios.

3. Analisar datas e condições.

Pós-condições: Resultado da consulta apresentado ou alteração autorizada confirmada ao usuário, conforme a operação solicitada.

Fluxos Alternativos

• A1 — Sessão ou permissão insuficiente: recusar acesso à operação protegida e informar a condição, sem aplicar a alteração solicitada.

• A2 — Dados inválidos, recurso indisponível ou falha de processamento: informar o resultado e permitir a correção ou nova tentativa; não apresentar sucesso sem confirmação.

Rastreabilidade: RF041, RF049.

Referência de implementação: src/app/events; src/app/api/events; src/app/api/events/[id]/challenges.

Variante 2 — Excluir evento sob responsabilidade

Descrição: Excluir evento sob responsabilidade.

Ator iniciador: Criador do evento ou Administrador autorizado.

Pré-condições e autorização: Somente criador ou Administrador, conforme regras de EventService.

Fluxo Principal

1. Selecionar evento.

2. Solicitar exclusão.

3. Validar responsabilidade.

4. Confirmar remoção.

Pós-condições: Resultado da consulta apresentado ou alteração autorizada confirmada ao usuário, conforme a operação solicitada.

Fluxos Alternativos

• A1 — Sessão ou permissão insuficiente: recusar acesso à operação protegida e informar a condição, sem aplicar a alteração solicitada.

• A2 — Dados inválidos, recurso indisponível ou falha de processamento: informar o resultado e permitir a correção ou nova tentativa; não apresentar sucesso sem confirmação.

Rastreabilidade: RF041.

Referência de implementação: src/app/api/events/[id]/route.ts; src/services/event.service.ts.

Variante 3 — Gerenciar desafios de evento

Descrição: Gerenciar desafios de evento.

Ator iniciador: Criador do evento ou Administrador autorizado.

Pré-condições e autorização: Criador ou Administrador; evento concluído não admite alteração; respostas privadas não são públicas.

Fluxo Principal

1. Criador define conjunto de desafios.

2. Valida dados.

3. Salva e consulta conjunto.

Pós-condições: Resultado da consulta apresentado ou alteração autorizada confirmada ao usuário, conforme a operação solicitada.

Fluxos Alternativos

• A1 — Sessão ou permissão insuficiente: recusar acesso à operação protegida e informar a condição, sem aplicar a alteração solicitada.

• A2 — Dados inválidos, recurso indisponível ou falha de processamento: informar o resultado e permitir a correção ou nova tentativa; não apresentar sucesso sem confirmação.

Rastreabilidade: Ampliação: desafios de evento.

Referência de implementação: src/app/api/events/[id]/challenges; src/services/event-challenge.service.ts.

Implementação: src/services/event-challenge.service.ts; src/services/event.service.ts.

Cenários técnicos relacionados: C44, C46.

Regras relacionadas: R83, R84, R85, R86, R87.

UC016 — Avaliar duelo autorizado

Objetivo: Avaliar duelo autorizado.

Ator principal: Avaliador.

Fluxo principal

Descrição: Emitir parecer humano sobre soluções de duelo.

Tipo: Análise

Ator iniciador ou caso chamador: Avaliador ou Administrador.

Atores secundários: Nenhum.

Pré-condições: Avaliador aprovado ou Administrador autenticado, que não participou do duelo; duelo no estado REVIEW_PENDING, com soluções disponíveis para análise.

Pós-condições: Parecer registrado e duelo concluído; a notificação é solicitada. Falha de notificação não desfaz a decisão, conforme RN022.

Entradas: Pontuações de 0 a 1.000, vencedor obrigatório, feedback com pelo menos 5 caracteres, pontos fortes e melhorias.

Saídas: Parecer persistido e resultado.

Fluxo Principal

1. O avaliador ou administrador abre a fila de avaliações pendentes.

2. O Stacklyst autoriza o papel e apresenta as soluções.

3. O avaliador compara código e resultados dos testes.

4. O avaliador informa pontuações e feedback explicativo.

5. O sistema valida, registra o parecer, fecha o duelo e notifica.

Fluxos Alternativos

• A1 — Papel ausente ou tentativa de avaliar o próprio duelo: o sistema nega a avaliação.

• A2 — Duelo já fechado: o sistema impede nova decisão conflitante.

• A3 — Feedback/pontuação inválidos: o sistema solicita correção.

Implementação: src/services/evaluator.service.ts.

Cenários técnicos relacionados: C40.

Regras relacionadas: R73, R74, R75.

UC017 — Gerenciar empresa e vagas

Objetivo: Gerenciar empresa e vagas.

Ator principal: Recrutador.

Fluxo principal

Descrição: Criar uma oportunidade vinculada a empresa autorizada.

Tipo: Configuração

Ator iniciador ou caso chamador: Recrutador; Administrador

Atores secundários: Nenhum.

Pré-condições: Ator autenticado com papel RECRUITER ou ADMIN e acesso autorizado à empresa selecionada. A verificação requireCompanyAccess valida o vínculo e as permissões antes da criação da vaga.

Pós-condições: Vaga aberta com etapas de recrutamento.

Entradas: Empresa, título, descrição, nível, tecnologias, modalidade, contrato e etapas.

Saídas: Vaga publicada ou erros por campo.

Fluxo Principal

1. O recrutador seleciona a empresa sob sua responsabilidade.

2. O ator informa dados da vaga e etapas.

3. O Stacklyst valida o papel de recrutador ou administrador, o acesso autorizado à empresa e os campos obrigatórios.

4. O sistema cria a vaga e as etapas em ordem.

5. A oportunidade é disponibilizada para consulta e candidatura.

Fluxos Alternativos

• A1 — Sem papel ou vínculo autorizado: o servidor nega a operação. A rota de vagas verifica o papel e aplica requireCompanyAccess para a empresa selecionada.

• A2 — Vaga inválida: o sistema destaca campos e não publica.

• A3 — Salário omitido: o sistema aplica a política definida sem inventar valor.

Variante 1 — Consultar vagas

Descrição: Listar e detalhar oportunidades disponíveis.

Tipo: Análise

Ator iniciador ou caso chamador: Usuário autenticado; Recrutador

Atores secundários: Nenhum.

Pré-condições: Acesso ao módulo; vagas cadastradas.

Pós-condições: Lista/detalhe exibido sem alteração obrigatória.

Entradas: Busca, tecnologia, nível, modalidade e contrato.

Saídas: Vagas filtradas, empresa, requisitos e etapas.

Fluxo Principal

1. O ator abre a listagem de vagas.

2. O Stacklyst carrega vagas abertas e filtros.

3. O ator informa critérios ou seleciona uma oportunidade.

4. O sistema aplica filtros e carrega os detalhes permitidos.

5. A vaga, empresa e opção de candidatura são apresentadas.

Fluxos Alternativos

• A1 — Nenhuma vaga: o sistema exibe estado vazio.

• A2 — Vaga encerrada: o sistema exibe status e desabilita candidatura.

• A3 — Filtro inválido: o sistema solicita correção.

Variante 2 — Cadastrar empresa

Descrição: Criar perfil de empresa vinculado a responsável autorizado.

Tipo: Configuração

Ator iniciador ou caso chamador: Usuário autenticado

Atores secundários: Nenhum.

Pré-condições: Usuário autenticado; dados da empresa disponíveis para preenchimento e validação.

Pós-condições: Empresa criada e vinculada ao responsável; conta USER passa a RECRUITER. A criação de eventos já é permitida a qualquer usuário autenticado.

Entradas: Nome, slug, descrição, localização, site e imagens opcionais.

Saídas: Cadastro da empresa.

Fluxo Principal

1. O usuário abre o cadastro de empresa e informa os dados solicitados.

2. O Stacklyst valida a sessão, os campos obrigatórios e a unicidade aplicável.

3. O sistema cria a empresa e vincula o usuário como proprietário.

4. O sistema atribui o papel RECRUITER à conta USER, preservando os demais papéis existentes.

5. O Stacklyst confirma o cadastro e apresenta a empresa ao responsável.

Fluxos Alternativos

• A1 — Sem permissão: o sistema nega o cadastro.

• A2 — Slug/nome conflitante: o sistema solicita alteração.

• A3 — Verificação oficial: não disponível; a empresa permanece com is_verified=false.

Implementação: src/services/job.service.ts.

Cenários técnicos relacionados: C41.

Regras relacionadas: R04, R79.

UC019 — Administrar usuários e recursos

Objetivo: Administrar usuários e recursos.

Ator principal: Administrador.

Fluxo principal

Descrição: Pesquisar usuários e alterar papéis autorizados.

Tipo: Configuração

Ator iniciador ou caso chamador: Administrador

Atores secundários: Nenhum.

Pré-condições: Administrador autenticado.

Pós-condições: Papel atualizado ou operação recusada; o serviço registra log da alteração.

Entradas: Busca, paginação, usuário e novo papel.

Saídas: Lista de usuários ou registro atualizado.

Fluxo Principal

1. O administrador abre o painel de usuários.

2. O Stacklyst valida a sessão administrativa e lista os registros.

3. O administrador pesquisa e seleciona o usuário.

4. O administrador escolhe o novo papel e confirma.

5. O sistema valida a transição, atualiza e registra a ação.

Fluxos Alternativos

• A1 — Sem permissão: o sistema nega acesso.

• A2 — Papel inválido: o sistema não altera o usuário.

• A3 — Alteração que removeria o último administrador: recusar a operação.

Variante 1 — Tratar denúncias

Descrição: Analisar denúncia e aplicar ação de moderação permitida.

Tipo: Análise

Ator iniciador ou caso chamador: Administrador

Atores secundários: Usuário denunciante; Autor do conteúdo

Pré-condições: Administrador autenticado; denúncia registrada.

Pós-condições: Denúncia tratada e ação registrada.

Entradas: Denúncia, conteúdo, contexto, decisão e justificativa.

Saídas: Resultado de moderação e eventual notificação.

Fluxo Principal

1. O administrador abre a fila de denúncias.

2. O Stacklyst apresenta denúncia, conteúdo e dados mínimos necessários.

3. O administrador analisa motivo e contexto.

4. O administrador seleciona decisão permitida e registra justificativa.

5. Aplicar a operação suportada e atualizar o registro da denúncia; não há trilha universal de auditoria.

Fluxos Alternativos

• A1 — Denúncia improcedente: o sistema encerra sem remover conteúdo.

• A2 — Conteúdo já removido: o sistema registra o estado encontrado.

• A3 — Sanção adicional ou recurso: fora do fluxo atual; registrar a solicitação para tratamento administrativo.

Variante 2 — Aprovar ou rejeitar candidatura de avaliador

Descrição: Aprovar ou rejeitar candidatura.

Tipo: Configuração

Ator iniciador ou caso chamador: Administrador

Atores secundários: Candidato

Pré-condições: Administrador autenticado; candidatura pendente.

Pós-condições: Decisão registrada; aprovação atribui papel.

Entradas: Candidatura e decisão.

Saídas: Status.

Fluxo Principal

1. O administrador seleciona uma candidatura pendente.

2. O administrador analisa os critérios de elegibilidade e as informações do candidato.

3. O administrador informa a aprovação ou rejeição.

4. O Stacklyst valida a autorização e o estado da candidatura, registra a decisão e atribui o papel de avaliador quando aprovada.

Fluxos Alternativos

• A1 — Candidatura já decidida: informar estado.

• A2 — Sem papel ADMIN: negar.

Variante 3 — Consultar métricas administrativas

Descrição: Consultar métricas administrativas.

Ator iniciador: Administrador.

Pré-condições e autorização: Exige Administrador; indicadores não equivalem a prova de segurança ou aceitação.

Fluxo Principal

1. Abrir painel.

2. Solicitar métricas.

3. Analisar indicadores consolidados.

Pós-condições: Resultado da consulta apresentado ou alteração autorizada confirmada ao usuário, conforme a operação solicitada.

Fluxos Alternativos

• A1 — Sessão ou permissão insuficiente: recusar acesso à operação protegida e informar a condição, sem aplicar a alteração solicitada.

• A2 — Dados inválidos, recurso indisponível ou falha de processamento: informar o resultado e permitir a correção ou nova tentativa; não apresentar sucesso sem confirmação.

Rastreabilidade: Ampliação: indicadores administrativos.

Referência de implementação: src/app/api/admin/metrics/route.ts; src/services/admin.service.ts.

Variante 4 — Importar perfis e threads editoriais

Descrição: Importar perfis e threads editoriais.

Ator principal: Operador autorizado

Pré-condições: Credenciais locais e nomes reservados sem conflito

Pós-condições: Coleção atualizada sem duplicar ou apagar comunidade

Fluxo principal

1. Consultar plano

2. Provisionar só identidades faltantes

3. Aplicar coleção em transação com IDs estáveis

4. Conferir quantidades e visibilidade

Fluxos alternativos: Colisão com conta comum interrompe; seed destrutivo é outro comando

Implementação: prisma/demo-community/publish.ts; src/services/admin.service.ts; src/services/evaluator.service.ts.

Cenários técnicos relacionados: C39, C47, C49.

Regras relacionadas: R02, R03, R72, R75, R88.

UC021 — Acompanhar notificações

Objetivo: Acompanhar notificações.

Ator principal: Usuário.

Fluxo principal

Descrição: Consultar notificações destinadas à própria conta e marcá-las como lidas.

Tipo: Análise

Ator iniciador ou caso chamador: Usuário autenticado, incluindo Avaliador, Administrador e Recrutador.

Atores secundários: Nenhum.

Pré-condições: Ator autenticado.

Pós-condições: Notificações exibidas; estado de leitura atualizado apenas quando solicitado.

Entradas: Cursor, limite e ação de leitura.

Saídas: Lista, próximo cursor e contagem não lida.

Fluxo Principal

1. O ator abre a central de notificações.

2. O Stacklyst identifica o usuário da sessão.

3. O sistema busca somente notificações do destinatário com paginação.

4. O ator abre itens ou solicita marcar como lidos.

5. O Stacklyst atualiza leitura e a contagem não lida.

Fluxos Alternativos

• A1 — Conexão indisponível: o sistema preserva o estado e permite nova tentativa.

• A2 — Notificação removida: o sistema ignora o item sem acessar recurso inexistente.

• A3 — Fora da aplicação web: o usuário deve acessar a central de notificações pelo navegador.

Variante 1 — Marcar notificações como lidas

Descrição: Marcar notificações como lidas.

Ator iniciador: Usuário autenticado.

Pré-condições e autorização: Operação afeta apenas as notificações do usuário autenticado.

Fluxo Principal

1. Abrir notificações.

2. Solicitar marcação.

3. Atualizar estado de leitura.

Pós-condições: Resultado da consulta apresentado ou alteração autorizada confirmada ao usuário, conforme a operação solicitada.

Fluxos Alternativos

• A1 — Sessão ou permissão insuficiente: recusar acesso à operação protegida e informar a condição, sem aplicar a alteração solicitada.

• A2 — Dados inválidos, recurso indisponível ou falha de processamento: informar o resultado e permitir a correção ou nova tentativa; não apresentar sucesso sem confirmação.

Rastreabilidade: RF046.

Referência de implementação: src/app/api/notifications/route.ts.

Implementação: src/services/notification.service.ts.

Cenários técnicos relacionados: C26.

Regras relacionadas: R35, R38.

UC022 — Candidatar-se a vaga

Objetivo: Candidatar-se a vaga.

Ator principal: Usuário.

Fluxo principal

Descrição: Registrar candidatura de usuário em vaga aberta.

Tipo: Condução

Ator iniciador ou caso chamador: Usuário autenticado

Atores secundários: Recrutador

Pré-condições: Usuário autenticado e vaga identificada. A abertura da vaga e a ausência de candidatura anterior são verificadas durante o fluxo.

Pós-condições: Candidatura criada, vinculada ao usuário, à vaga aberta e à etapa inicial do processo.

Entradas: Identificador da vaga e sessão.

Saídas: Confirmação e situação da candidatura.

Fluxo Principal

1. O usuário consulta o detalhe de uma vaga aberta.

2. O usuário seleciona a opção de candidatura.

3. O Stacklyst verifica sessão, status da vaga e duplicidade.

4. O sistema cria a candidatura vinculada ao usuário, à vaga e à etapa inicial do processo de seleção.

5. O Stacklyst confirma e passa a exibir a situação no processo.

Fluxos Alternativos

• A1 — Vaga encerrada: o sistema recusa candidatura.

• A2 — Candidatura duplicada: o sistema informa o registro existente.

• A3 — Perfil incompleto: permitir candidatura com os dados obrigatórios da vaga e indicar os campos públicos ausentes.

Implementação: src/services/job.service.ts.

Cenários técnicos relacionados: C42.

Regras relacionadas: R76, R77, R78.

UC023 — Participar de evento

Objetivo: Participar de evento.

Ator principal: Usuário.

Fluxo principal

Descrição: Inscrever o usuário em um evento elegível, cancelar sua própria participação e consultar o estado da inscrição.

Tipo: Condução

Ator iniciador ou caso chamador: Usuário autenticado

Atores secundários: Nenhum.

Pré-condições: Usuário autenticado e evento identificado. Encerramento, capacidade, nível e participação prévia são verificados durante o fluxo.

Pós-condições: Participação registrada uma única vez na inscrição ou vínculo de participação removido no cancelamento.

Entradas: Identificador do evento, sessão e ação de inscrição ou cancelamento.

Saídas: Confirmação, regras e estado da inscrição.

Fluxo Principal

1. O usuário abre a listagem e seleciona um evento.

2. O Stacklyst apresenta período, nível e capacidade.

3. O usuário solicita participação.

4. O sistema verifica se o evento não está encerrado, confere nível e capacidade e mantém a unicidade da participação.

5. O Stacklyst registra o participante pela operação POST e confirma a inscrição.

Fluxos Alternativos

• A1 — Nível insuficiente: o sistema informa o requisito.

• A2 — Evento lotado ou encerrado: o sistema recusa a inscrição.

• A3 — Inscrição já registrada: o sistema não cria duplicidade; apresenta a participação existente quando o evento continua elegível.

• A4 — Cancelar participação: antes do encerramento do evento, o usuário solicita retirar a própria inscrição; a operação DELETE remove o vínculo e atualiza o estado exibido.

Implementação: src/services/event.service.ts.

Cenários técnicos relacionados: C45.

Regras relacionadas: R81, R82, R83, R84.

UC024 — Solicitar credenciamento de avaliador

Objetivo: Solicitar credenciamento de avaliador.

Ator principal: Usuário.

Fluxo principal

Descrição: Verificar elegibilidade e enviar candidatura para papel de avaliador.

Tipo: Condução

Ator iniciador ou caso chamador: Usuário autenticado

Atores secundários: Administrador

Pré-condições: Usuário autenticado. Elegibilidade, papel atual e existência de candidatura pendente são verificados durante o fluxo.

Pós-condições: Candidatura pendente criada ou inelegibilidade informada.

Entradas: Motivação e tecnologias de domínio.

Saídas: Elegibilidade e status da candidatura.

Fluxo Principal

1. O usuário abre a área de candidatura a avaliador.

2. O Stacklyst calcula trilhas completas e XP.

3. Se elegível, o usuário informa motivação e tecnologias.

4. O sistema valida e registra a candidatura pendente.

5. O administrador passa a visualizar a candidatura para decisão.

Fluxos Alternativos

• A1 — Critério não atendido: o sistema mostra requisitos atuais.

• A2 — Candidatura já pendente: o sistema não duplica.

• A3 — Usuário já é avaliador/administrador: o sistema informa o papel existente.

Implementação: src/services/evaluator.service.ts.

Cenários técnicos relacionados: C39.

Regras relacionadas: R03, R72, R75.

UC026 — Gerenciar candidaturas

Objetivo: Gerenciar candidaturas.

Ator principal: Recrutador.

Fluxo principal

Descrição: Atualizar a etapa de uma candidatura autorizada e solicitar automaticamente a notificação ao candidato associado.

Tipo: Configuração

Ator iniciador: Recrutador responsável ou Administrador autorizado.

Atores secundários: Candidato

Pré-condições: Ator autenticado e autorizado; candidatura existente, vinculada à vaga da empresa sob sua responsabilidade; etapa de destino pertencente ao processo da vaga.

Pós-condições: Etapa persistida e notificação automática solicitada somente ao candidato associado à candidatura. Falha de notificação não desfaz a atualização.

Entradas: Candidatura e etapa.

Saídas: Novo estado.

Fluxo Principal

1. O recrutador seleciona a candidatura e informa a etapa desejada.

2. O Stacklyst confere o acesso à empresa e o vínculo da candidatura com a vaga sob responsabilidade do ator.

3. O sistema valida a etapa de destino e os dados informados.

4. O sistema salva a alteração, confirma o resultado e solicita a notificação JOB_APPLICATION_UPDATE para o usuário associado à candidatura, como comportamento interno de UC026.

Fluxos Alternativos

• A1 — Sem vínculo ou etapa inválida: recusar.

• A2 — Notificação falha: registrar erro; a etapa permanece salva.

Variante 1 — Consultar candidaturas de uma vaga

Descrição: Consultar as candidaturas de uma vaga sob gestão, após validar o papel autorizado e a responsabilidade pela empresa.

Ator iniciador: Recrutador responsável ou Administrador autorizado.

Pré-condições e autorização: Recrutador responsável ou Administrador autenticado. A rota exige papel autorizado e JobService valida a responsabilidade pela empresa da vaga.

Fluxo Principal

1. Selecionar vaga sob gestão.

2. Listar candidaturas.

3. Analisar informações autorizadas.

Pós-condições: Resultado da consulta apresentado ou alteração autorizada confirmada ao usuário, conforme a operação solicitada.

Fluxos Alternativos

• A1 — Sessão ou permissão insuficiente: recusar acesso à operação protegida e informar a condição, sem aplicar a alteração solicitada.

• A2 — Dados inválidos, recurso indisponível ou falha de processamento: informar o resultado e permitir a correção ou nova tentativa; não apresentar sucesso sem confirmação.

Rastreabilidade: RF035, RF038, RF048.

Referência de implementação: src/app/api/jobs/[id]/applications; src/services/job.service.ts.

Implementação: src/services/job.service.ts.

Cenários técnicos relacionados: C41, C43.

Regras relacionadas: R04, R79, R80.

UC028 — Avaliar resposta

Objetivo: Avaliar resposta.

Ator principal: Usuário.

Fluxo principal

Descrição: Avaliar a resposta conforme o conteúdo e os critérios canônicos da atividade. A comparação de respostas conceituais é completa por si. O ponto de extensão avaliar código é alcançado quando o tipo exige execução de programa.

Tipo: Análise; comportamento reutilizado por UC006.

Ator iniciador ou caso chamador: UC006 Resolver exercício

Atores secundários: Executor de código, por meio de UC029 nas atividades de programação.

Pré-condições: Resposta válida e acesso autorizado.

Pós-condições: Resultado da avaliação e feedback retornados, ou falha técnica informada sem aprovação. O caso chamador é responsável pela gravação.

Entradas: Resposta e atividade canônica.

Saídas: Avaliação e feedback.

Fluxo Principal

1. O Stacklyst identifica o tipo da atividade e os critérios de avaliação aplicáveis.

2. Para respostas conceituais, compara a escolha, as lacunas, a ordem, os pares, os blocos ou o comando com o conteúdo canônico.

3. Para atividades de programação, no ponto avaliar código, executa UC029 e utiliza os resultados dos testes para determinar acerto ou erro. Falha técnica não equivale a resposta correta.

4. O Stacklyst retorna a avaliação e o feedback a UC006, sem conceder XP diretamente.

Fluxos Alternativos

• A1 — Configuração de avaliação ausente: não aprovar.

• A2 — Executor indisponível: retornar falha controlada.

Sem tela própria: comportamento reutilizado pelo caso chamador; o resultado é apresentado no editor ou no feedback da atividade.

Implementação: src/lib/learning/assessment.ts.

Cenários técnicos relacionados: C07.

Regras relacionadas: R12, R13, R17.

UC029 — Executar código e testes

Objetivo: Executar código e testes.

Ator principal: Usuário.

Fluxo principal

Descrição: Executar código em ambiente isolado, usando o sandbox interno ou um provedor externo conforme a linguagem, com testes definidos no servidor.

Tipo: Análise; comportamento reutilizado por UC010 e UC028.

Ator iniciador ou caso chamador: UC010, por inclusão; UC028, pela extensão no ponto avaliar código.

Atores secundários: Executor de código

Pré-condições: Código e linguagem suportados; contrato de teste carregado.

Pós-condições: Saída e resultado de execução retornados.

Entradas: Código, linguagem e testes aplicáveis.

Saídas: Saída, erros e tempo de execução.

Fluxo Principal

1. Preparar código e testes.

2. Selecionar o executor suportado pela linguagem. JavaScript/TypeScript podem usar QuickJS interno; outras linguagens usam o provedor configurado.

3. Tratar saída, compilação ou tempo limite.

4. Retornar resultados sem expor testes ocultos.

Fluxos Alternativos

• A1 — Provedor indisponível: falhar sem confirmar acerto.

• A2 — Código inválido: retornar erro de compilação ou execução.

Sem tela própria: comportamento reutilizado pelo caso chamador; o resultado é apresentado no editor ou no feedback da atividade.

Implementação: src/app/api/lessons/[lessonId]/attempt/route.ts; src/lib/duels/judge.ts; src/lib/exercises/repository.ts.

Cenários técnicos relacionados: C08, C10, C37.

Regras relacionadas: R12, R13, R16, R17, R63, R64.

UC038 — Trocar mensagens privadas

Objetivo: Trocar mensagens privadas.

Ator principal: Usuário.

Fluxo principal

Descrição: Consultar conversas e mensagens.

Ator iniciador: Usuário autenticado.

Pré-condições e autorização: O histórico envolve o usuário autenticado como remetente ou destinatário.

Fluxo Principal

1. Abrir conversas.

2. Selecionar interlocutor.

3. Consultar histórico permitido.

Pós-condições: Resultado da consulta apresentado ou alteração autorizada confirmada ao usuário, conforme a operação solicitada.

Fluxos Alternativos

• A1 — Sessão ou permissão insuficiente: recusar acesso à operação protegida e informar a condição, sem aplicar a alteração solicitada.

• A2 — Dados inválidos, recurso indisponível ou falha de processamento: informar o resultado e permitir a correção ou nova tentativa; não apresentar sucesso sem confirmação.

Rastreabilidade: Ampliação: mensagens.

Referência de implementação: src/app/messages/page.tsx; src/app/api/messages/chats; src/app/api/messages.

Variante 1 — Enviar mensagem privada

Descrição: Enviar mensagem privada.

Ator iniciador: Usuário autenticado.

Pré-condições e autorização: Sessão válida e destinatário informado; envio sujeito a limite de frequência.

Fluxo Principal

1. Selecionar interlocutor.

2. Escrever texto ou anexar imagem.

3. Enviar e consultar retorno.

Pós-condições: Resultado da consulta apresentado ou alteração autorizada confirmada ao usuário, conforme a operação solicitada.

Fluxos Alternativos

• A1 — Sessão ou permissão insuficiente: recusar acesso à operação protegida e informar a condição, sem aplicar a alteração solicitada.

• A2 — Dados inválidos, recurso indisponível ou falha de processamento: informar o resultado e permitir a correção ou nova tentativa; não apresentar sucesso sem confirmação.

Rastreabilidade: Ampliação: mensagens.

Referência de implementação: src/app/messages/page.tsx; src/app/api/messages/route.ts.

Variante 2 — Editar ou excluir mensagem própria

Descrição: Editar ou excluir mensagem própria.

Ator iniciador: Usuário autenticado.

Pré-condições e autorização: Somente remetente; edição não admite texto vazio.

Fluxo Principal

1. Selecionar mensagem enviada.

2. Editar conteúdo ou excluir.

3. Atualizar conversa.

Pós-condições: Resultado da consulta apresentado ou alteração autorizada confirmada ao usuário, conforme a operação solicitada.

Fluxos Alternativos

• A1 — Sessão ou permissão insuficiente: recusar acesso à operação protegida e informar a condição, sem aplicar a alteração solicitada.

• A2 — Dados inválidos, recurso indisponível ou falha de processamento: informar o resultado e permitir a correção ou nova tentativa; não apresentar sucesso sem confirmação.

Rastreabilidade: Ampliação: mensagens.

Referência de implementação: src/app/api/messages/[id]/route.ts.

Variante 3 — Reagir a mensagem

Descrição: Reagir a mensagem.

Ator iniciador: Usuário autenticado.

Pré-condições e autorização: Somente remetente ou destinatário da mensagem.

Fluxo Principal

1. Selecionar mensagem.

2. Escolher emoji.

3. Adicionar ou remover reação.

Pós-condições: Resultado da consulta apresentado ou alteração autorizada confirmada ao usuário, conforme a operação solicitada.

Fluxos Alternativos

• A1 — Sessão ou permissão insuficiente: recusar acesso à operação protegida e informar a condição, sem aplicar a alteração solicitada.

• A2 — Dados inválidos, recurso indisponível ou falha de processamento: informar o resultado e permitir a correção ou nova tentativa; não apresentar sucesso sem confirmação.

Rastreabilidade: Ampliação: mensagens.

Referência de implementação: src/app/api/messages/[id]/react/route.ts.

Implementação: src/app/api/messages/route.ts.

Cenários técnicos relacionados: C27.

Regras relacionadas: R06, R36, R37, R38.

Consolidação e redução do catálogo

Caso mantido	Operações reunidas como fluxos e variantes
UC001 — Cadastrar conta	Cadastrar conta
UC002 — Fazer login	Fazer login; Encerrar sessão; Confirmar e recuperar acesso
UC003 — Gerenciar perfil e preferências	Editar perfil; Consultar perfil público; Seguir ou deixar de seguir usuário; Configurar preferências pessoais; Personalizar avatar; Bloquear conta; Enviar imagem compatível
UC004 — Consultar e selecionar trilha	Selecionar trilha; Consultar disponibilidade de conhecimentos
UC005 — Estudar lição	Estudar lição; Resgatar etapa especial da trilha
UC006 — Resolver exercício	Resolver exercício; Testar código antes da submissão
UC007 — Consultar evolução e reconhecimento	Consultar progresso; Consultar ranking de XP; Consultar e emitir credencial elegível; Verificar credencial pública; Gerar PDF de credencial
UC008 — Disputar duelo de programação	Criar duelo aberto; Entrar em duelo; Consultar resultado de duelo; Excluir duelo elegível
UC010 — Enviar solução de duelo	Enviar solução de duelo
UC012 — Consultar ajuda do exercício	Consultar ajuda do exercício

Caso mantido	Operações reunidas como fluxos e variantes
UC013 — Gerenciar publicação própria	Criar publicação; Editar ou excluir publicação própria
UC014 — Consultar e interagir com a comunidade	Interagir com publicação; Consultar e pesquisar feed; Consultar publicações salvas; Aceitar resposta de uma publicação; Responder quiz diário; Votar em enquete; Denunciar publicação
UC015 — Gerenciar eventos e desafios	Criar evento; Consultar eventos e desafios; Excluir evento sob responsabilidade; Gerenciar desafios de evento
UC016 — Avaliar duelo autorizado	Avaliar empate técnico
UC017 — Gerenciar empresa e vagas	Publicar vaga; Consultar vagas; Cadastrar empresa
UC019 — Administrar usuários e recursos	Gerenciar permissões de usuários; Tratar denúncias; Aprovar ou rejeitar candidatura de avaliador; Consultar métricas administrativas; Importar perfis e threads editoriais
UC021 — Acompanhar notificações	Consultar notificações; Marcar notificações como lidas
UC022 — Candidatar-se a vaga	Candidatar-se a vaga
UC023 — Participar de evento	Inscrever-se ou cancelar participação em evento
UC024 — Solicitar credenciamento de avaliador	Candidatar-se a avaliador
UC026 — Gerenciar candidaturas	Atualizar etapa de candidatura; Consultar candidaturas de uma vaga
UC028 — Avaliar resposta	Avaliar resposta
UC029 — Executar código e testes	Executar código e testes
UC038 — Trocar mensagens privadas	Consultar conversas e mensagens; Enviar mensagem privada; Editar ou excluir mensagem própria; Reagir a mensagem

Equivalência com os cenários de implementação

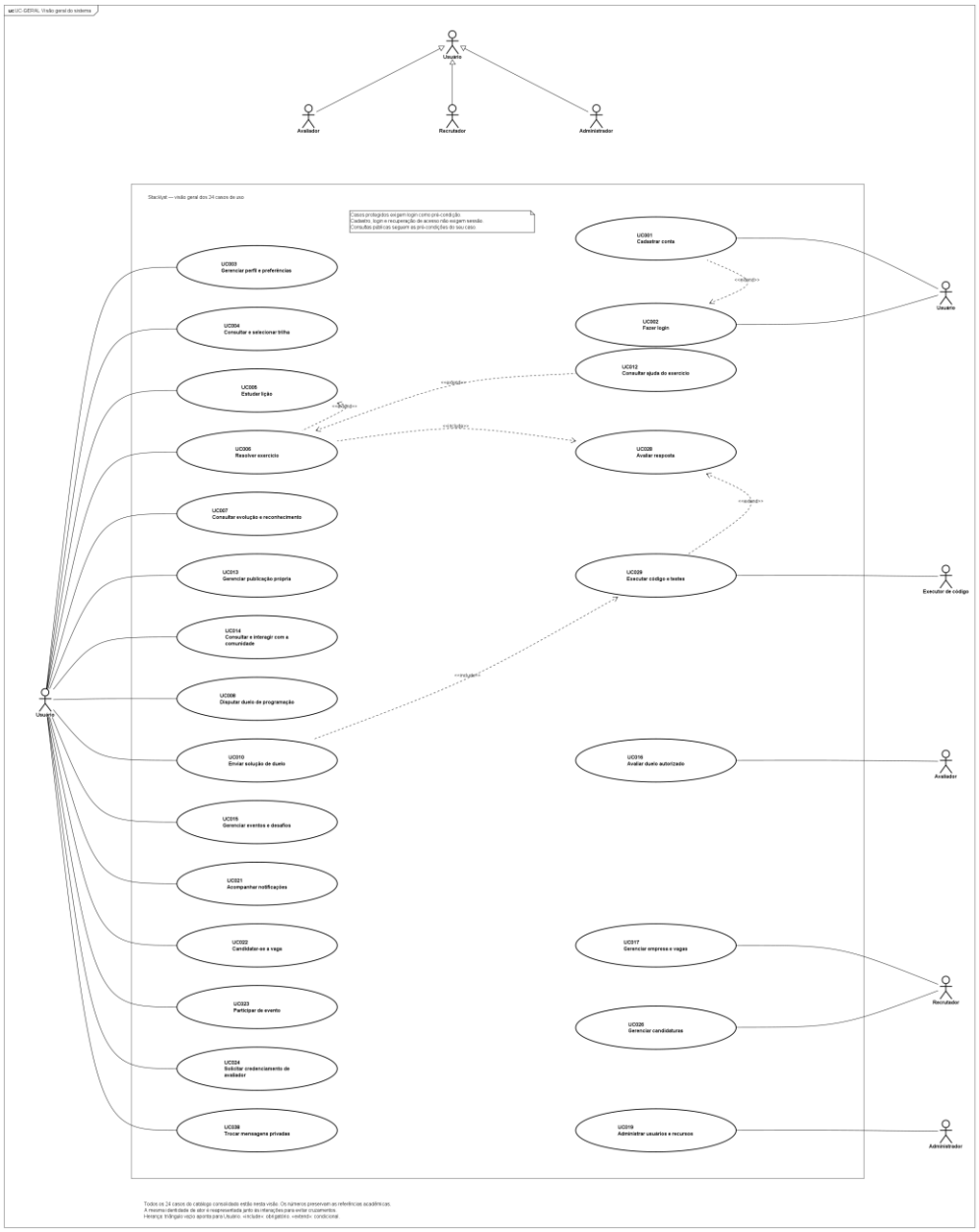
Cenário técnico	Casos acadêmicos relacionados
C01	UC001
C02	UC002
C03	UC002
C04	UC004
C05	UC004
C06	UC005

Cenário técnico	Casos acadêmicos relacionados
C07	UC006, UC028
C08	UC006, UC029
C09	UC006
C10	UC006, UC029
C11	UC006
C12	UC012
C13	UC005
C14	UC007
C15	UC007
C16	UC007
C17	UC013
C18	UC014
C19	UC014
C20	UC014
C21	UC014
C22	UC003
C23	UC003
C24	UC003
C25	UC007
C26	UC021
C27	UC038
C28	UC003
C29	
C30	
C31	
C32	
C33	
C34	
C35	
C36	UC008

Cenário técnico	Casos acadêmicos relacionados
C37	UC006, UC010, UC029
C38	UC008
C39	UC019, UC024
C40	UC016
C41	UC017, UC026
C42	UC022
C43	UC026
C44	UC015
C45	UC023
C46	UC015
C47	UC019
C48	
C49	UC019
C50	UC014
C52	UC014
C53	UC014
C54	UC003

Casos de uso gerais

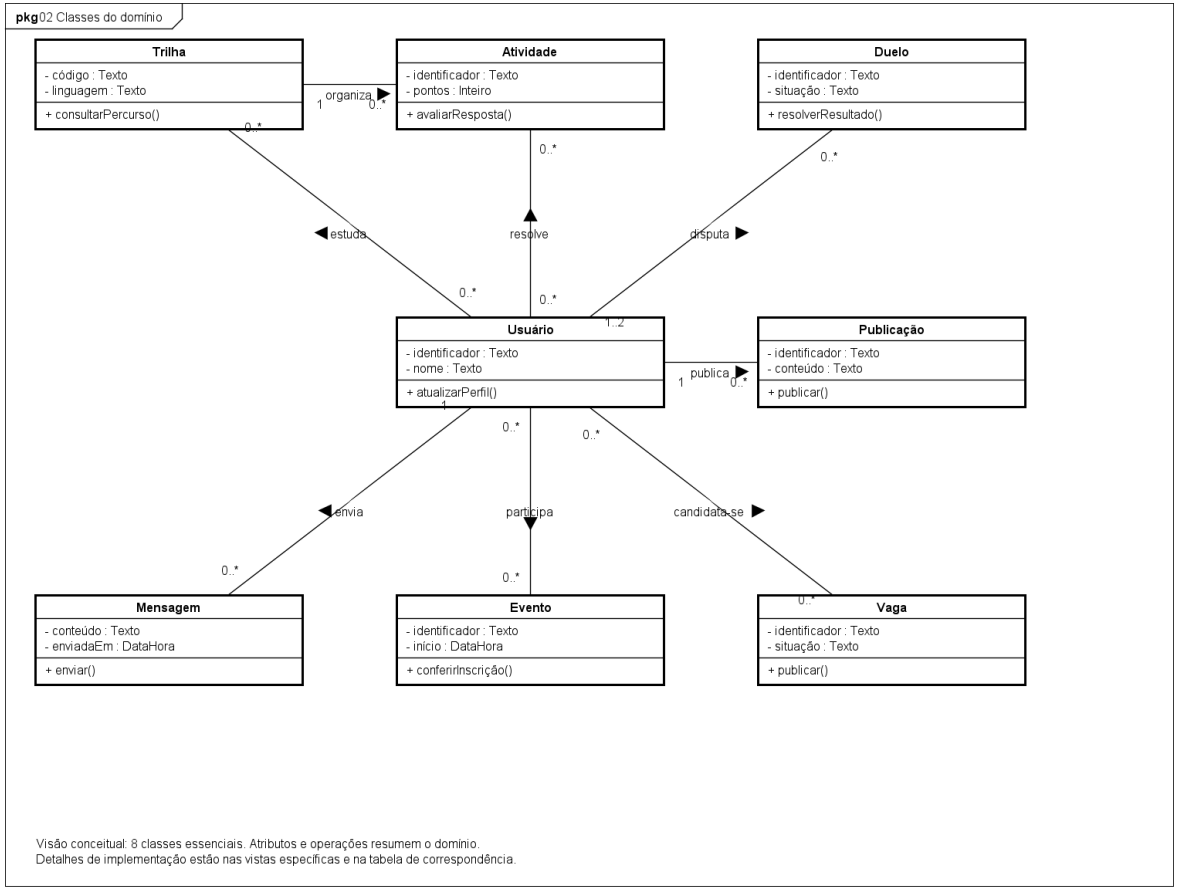
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



UC-GERAL Casos de uso gerais

Classes conceituais gerais

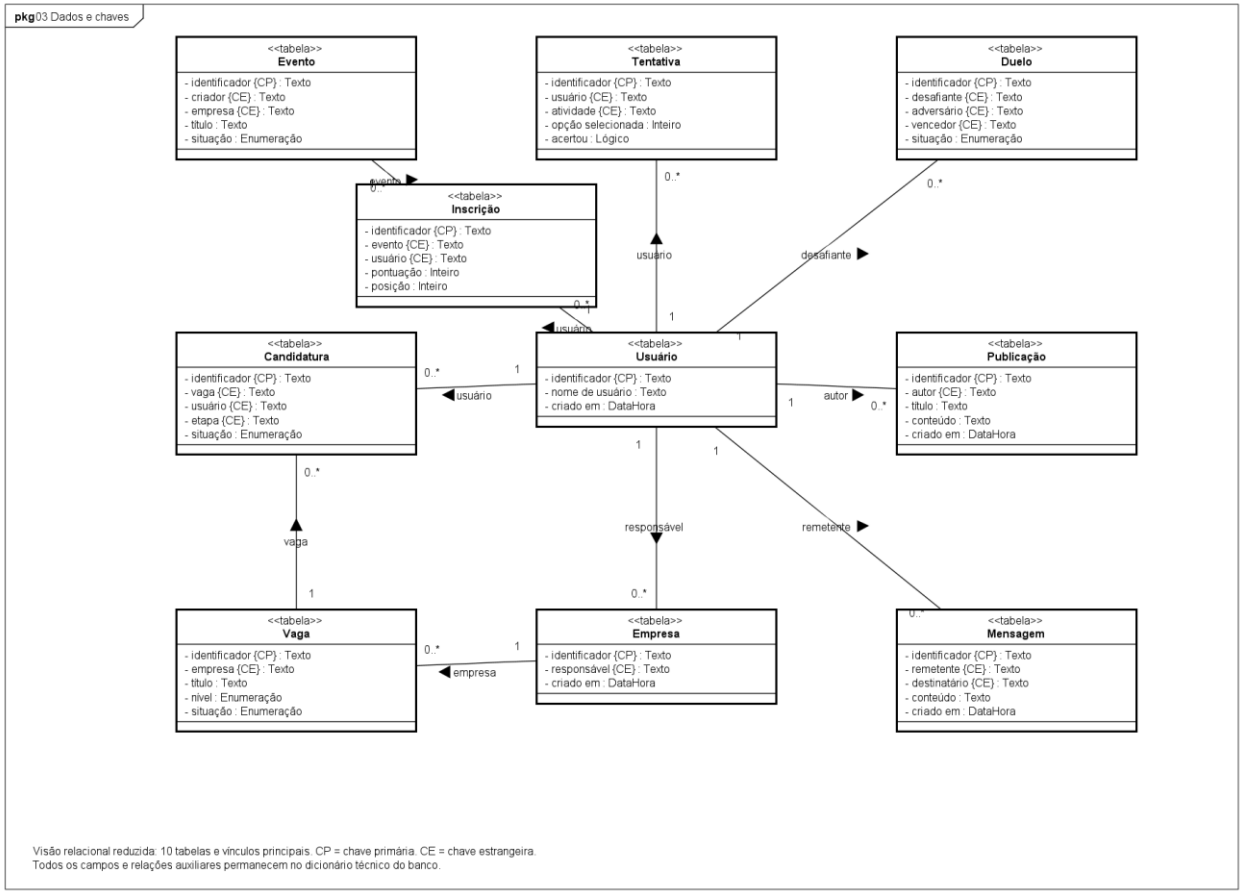
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



CL-GERAL Classes conceituais gerais

Visão relacional geral reduzida

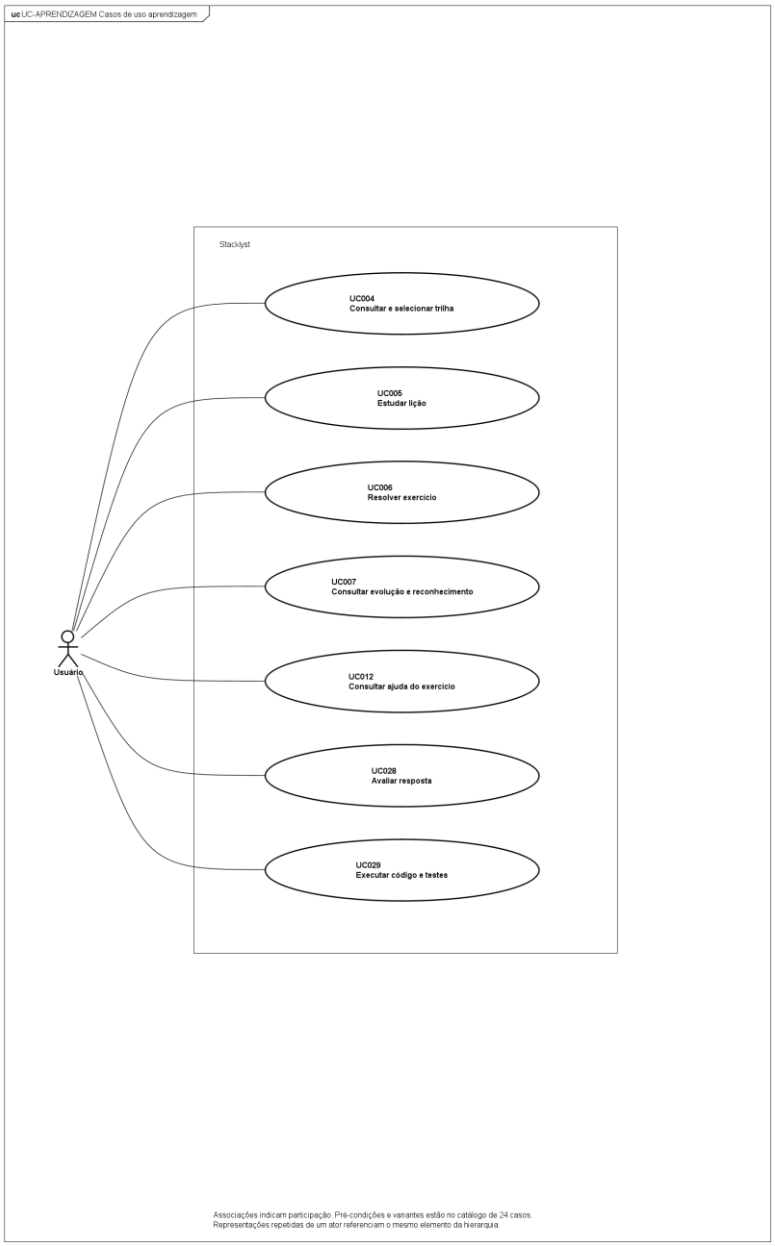
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



DER-GERAL Visão relacional geral reduzida

Casos de uso de aprendizagem

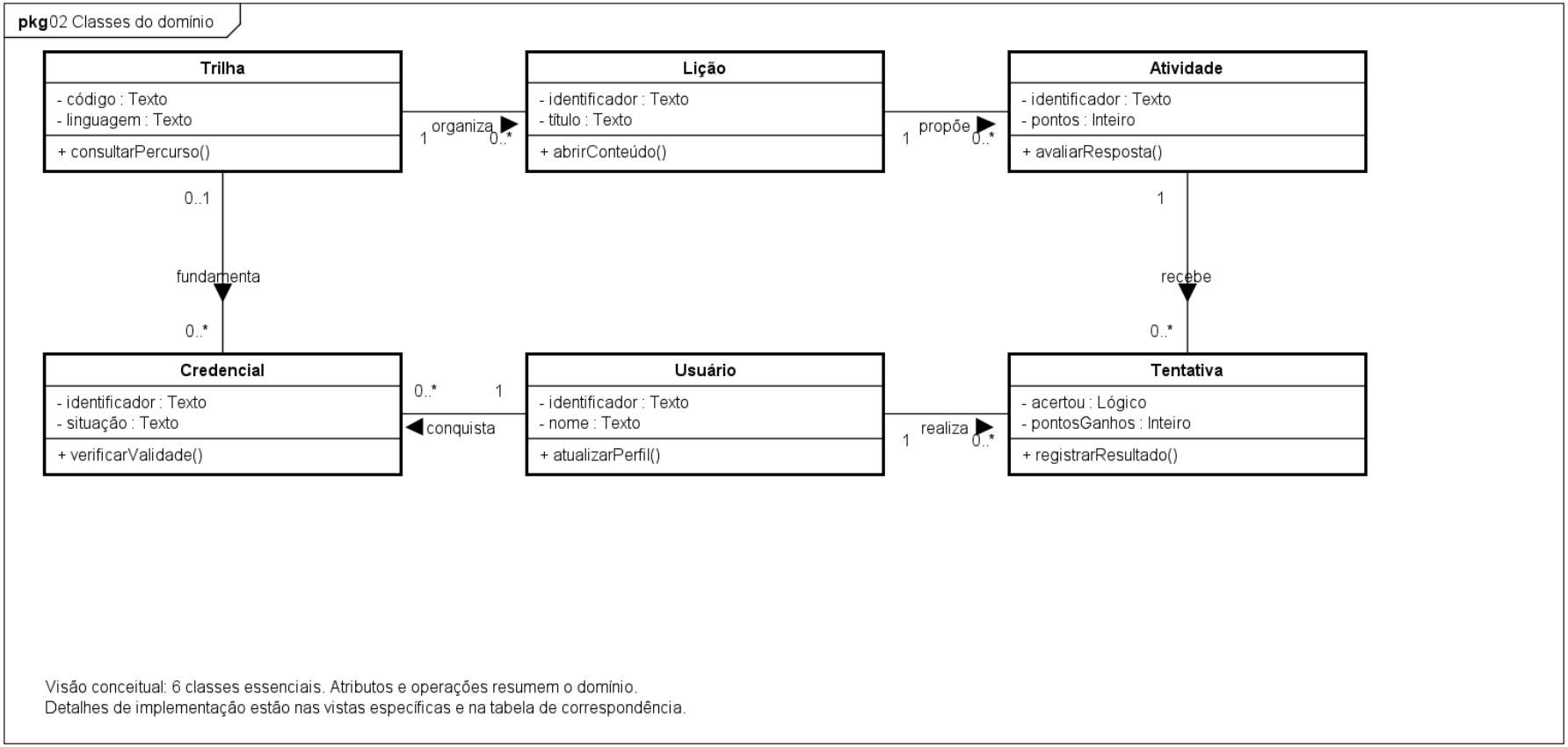
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



UC-APRENDIZAGEM Casos de uso de aprendizagem

Classes de aprendizagem

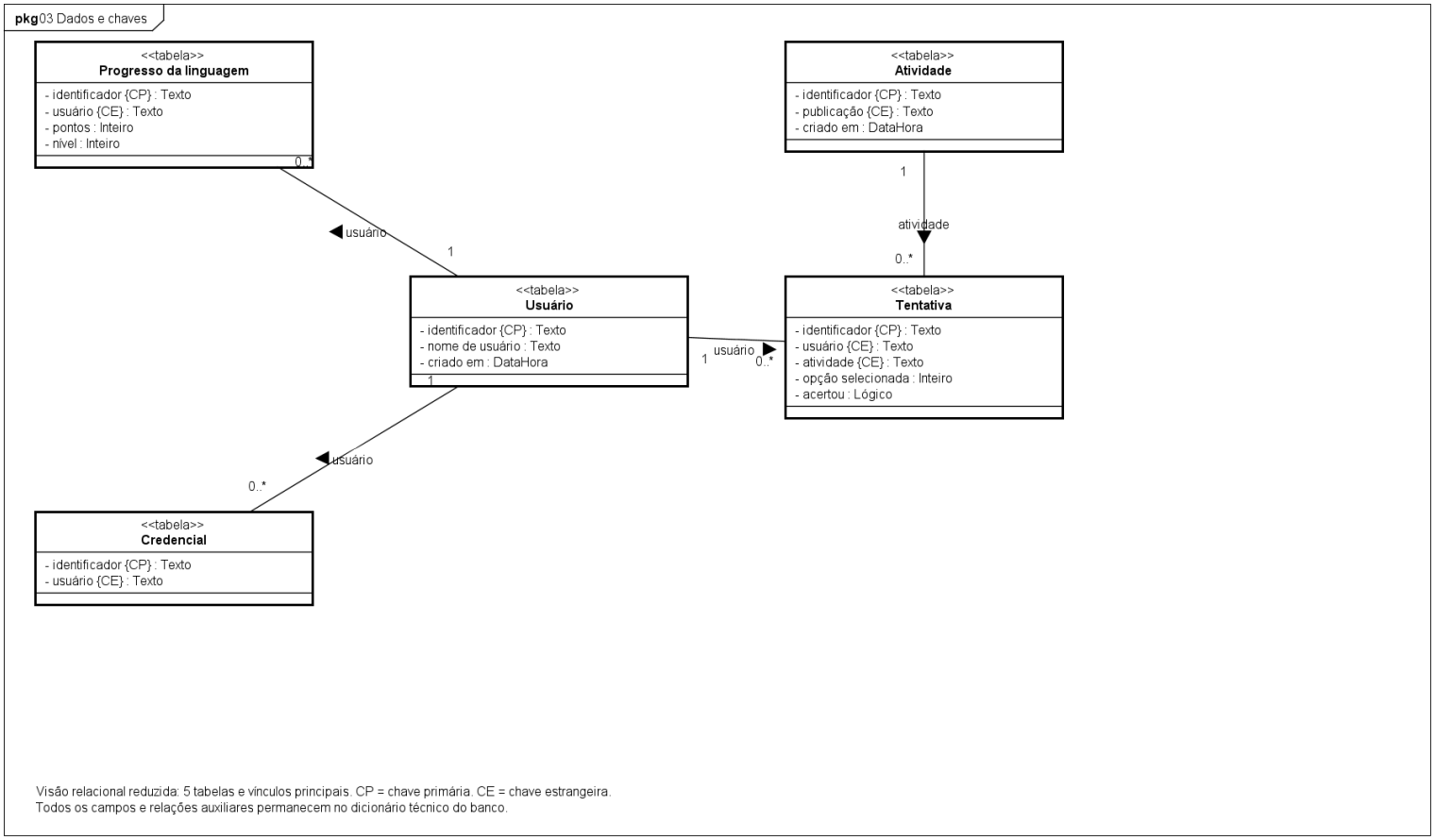
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



CL-APRENDIZAGEM Classes de aprendizagem

Dados de aprendizagem

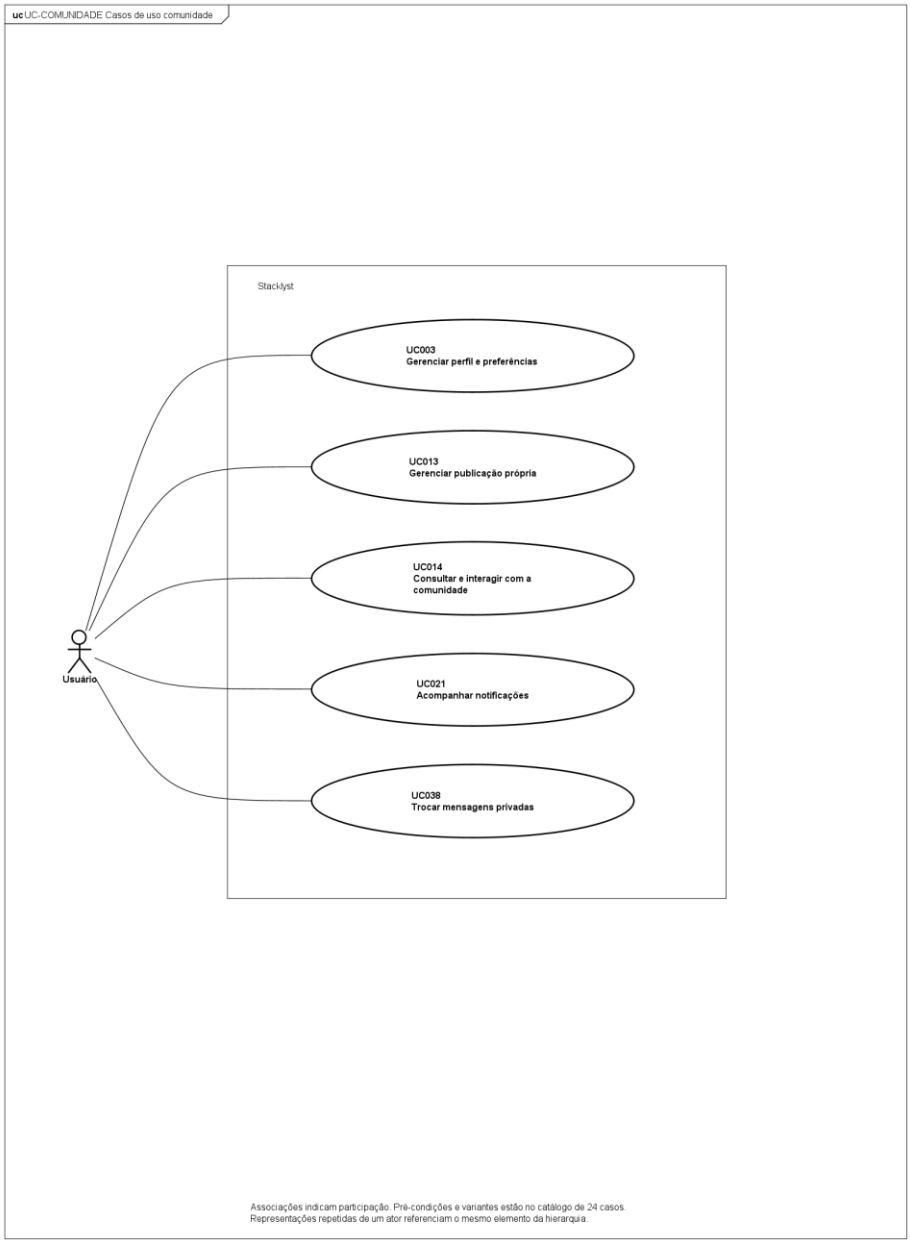
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



DER-APRENDIZAGEM Dados de aprendizagem

Casos de uso de comunidade

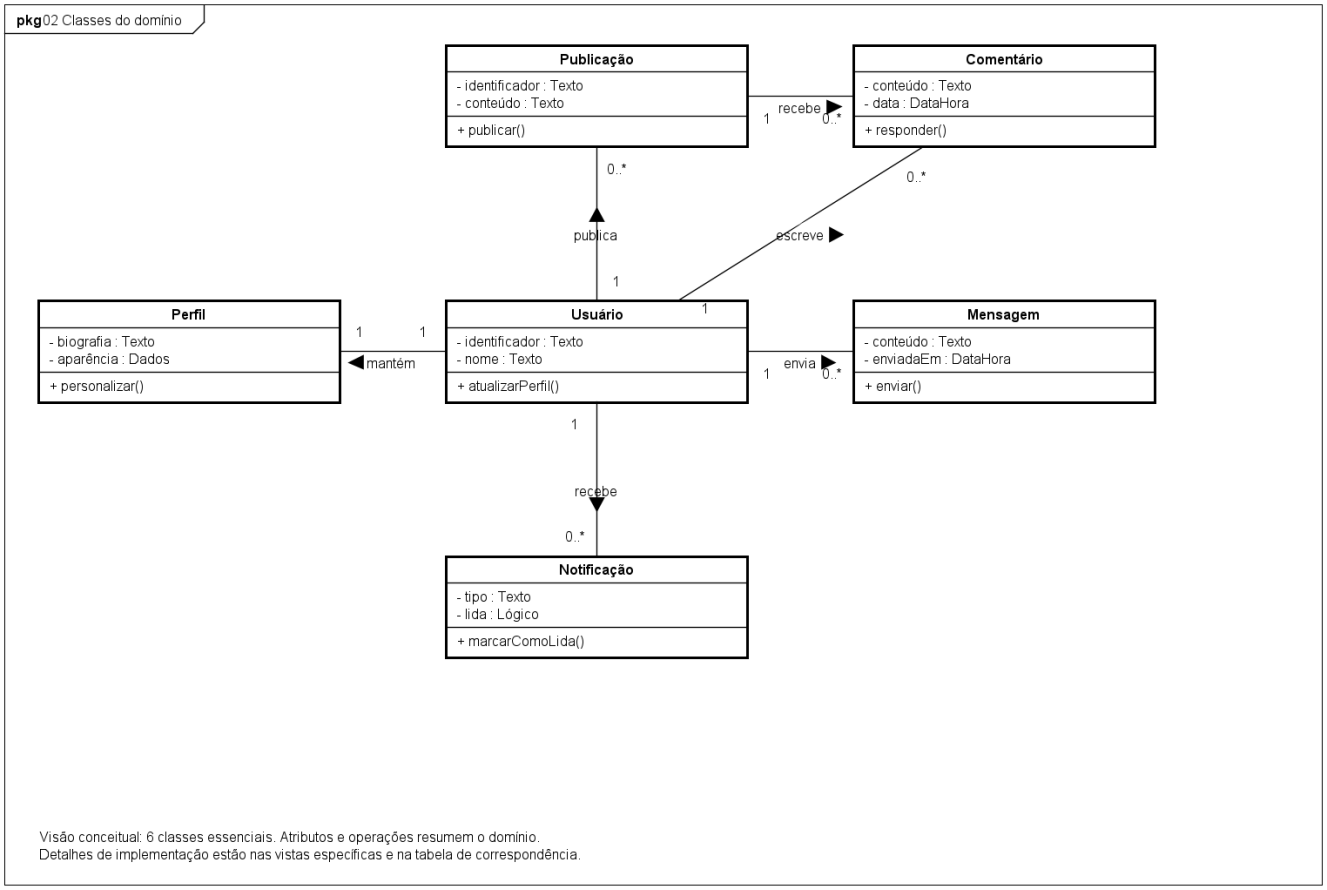
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



UC-COMUNIDADE Casos de uso de comunidade

Classes de comunidade

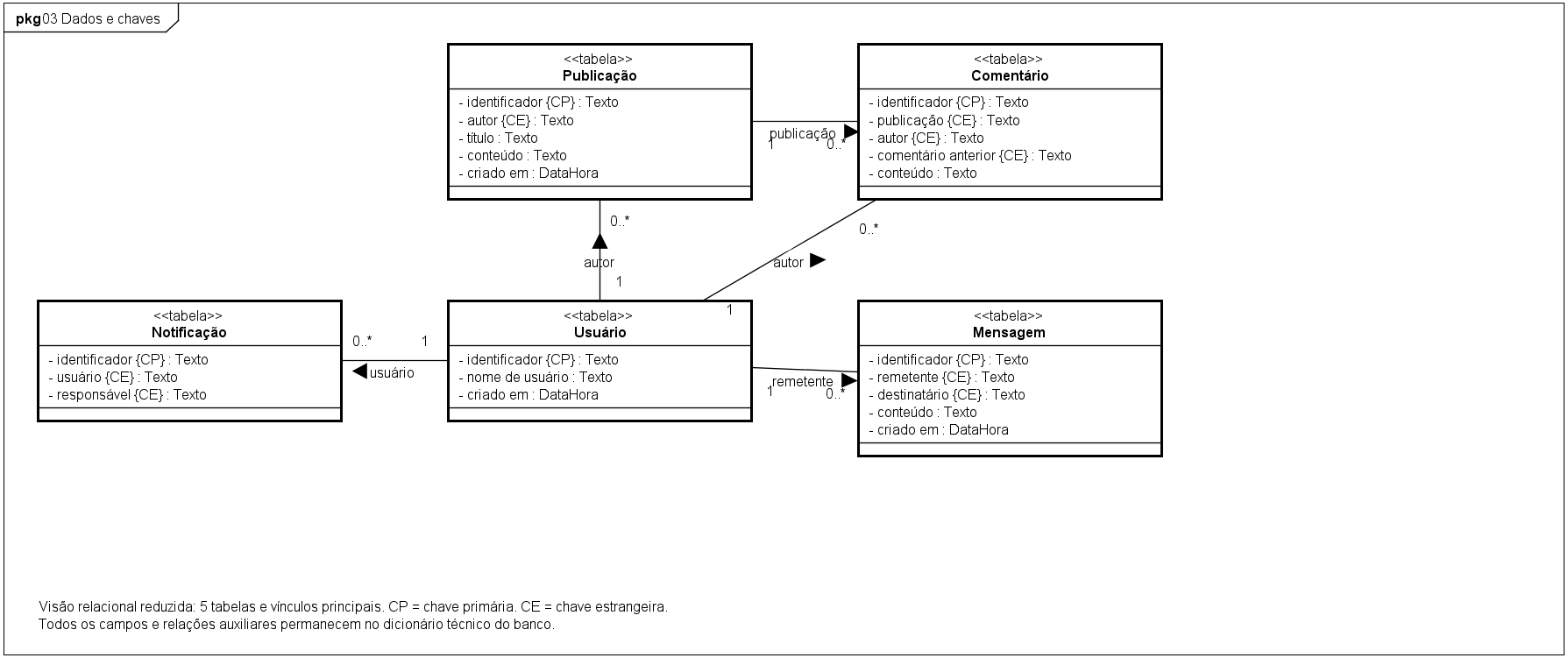
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



CL-COMUNIDADE Classes de comunidade

Dados de comunidade

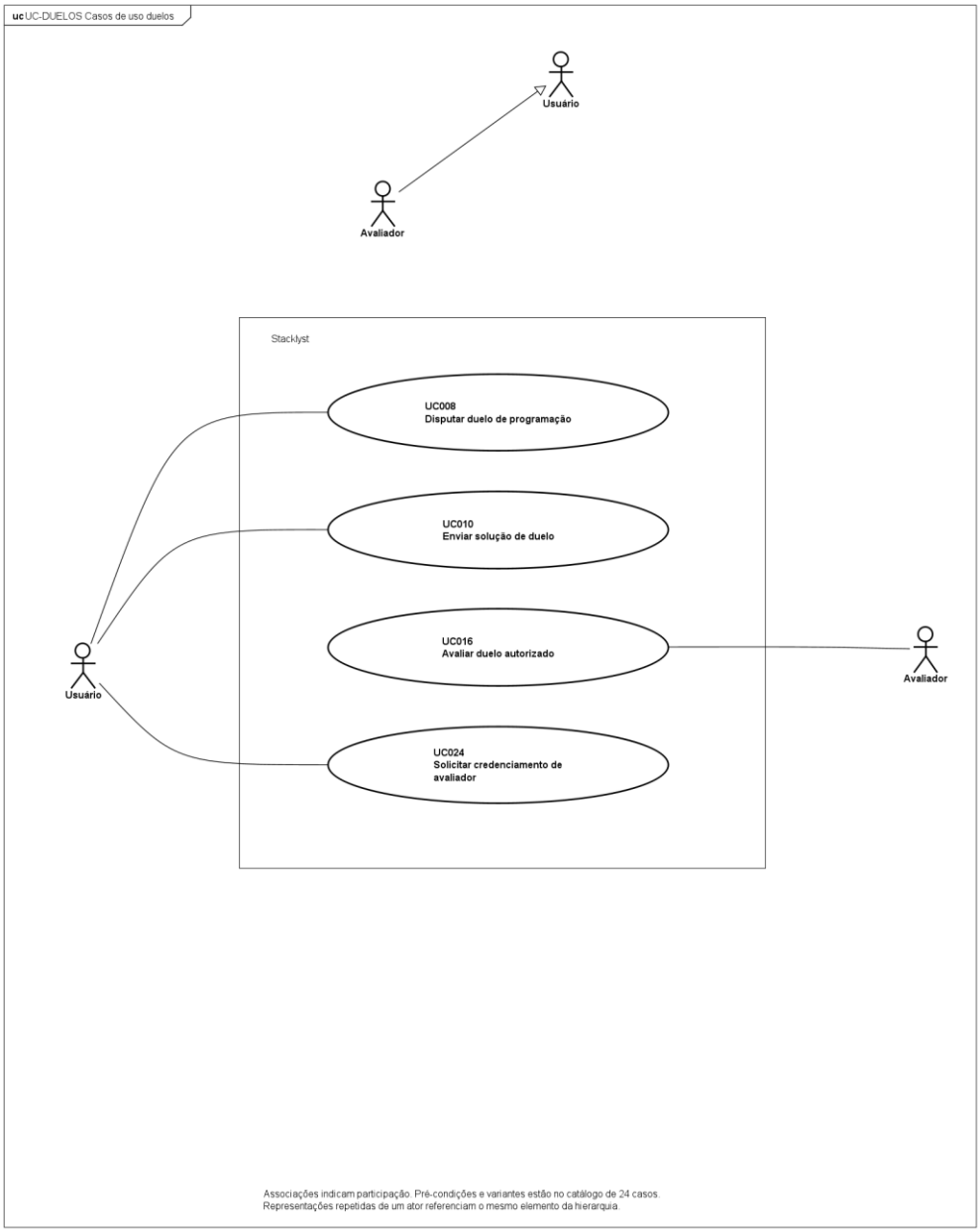
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



DER-COMUNIDADE Dados de comunidade

Casos de uso de duelos

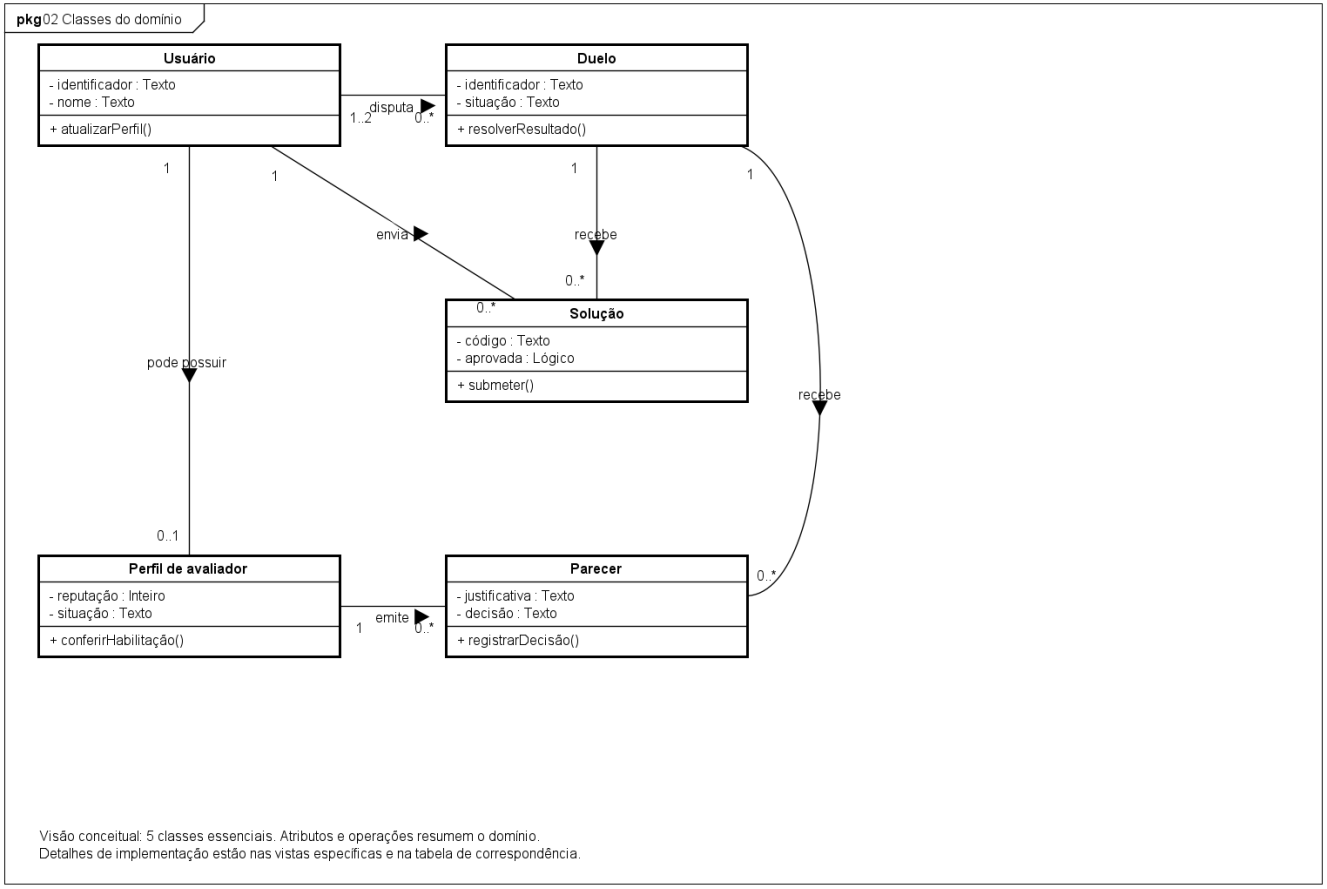
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



UC-DUELOS Casos de uso de duelos

Classes de duelos

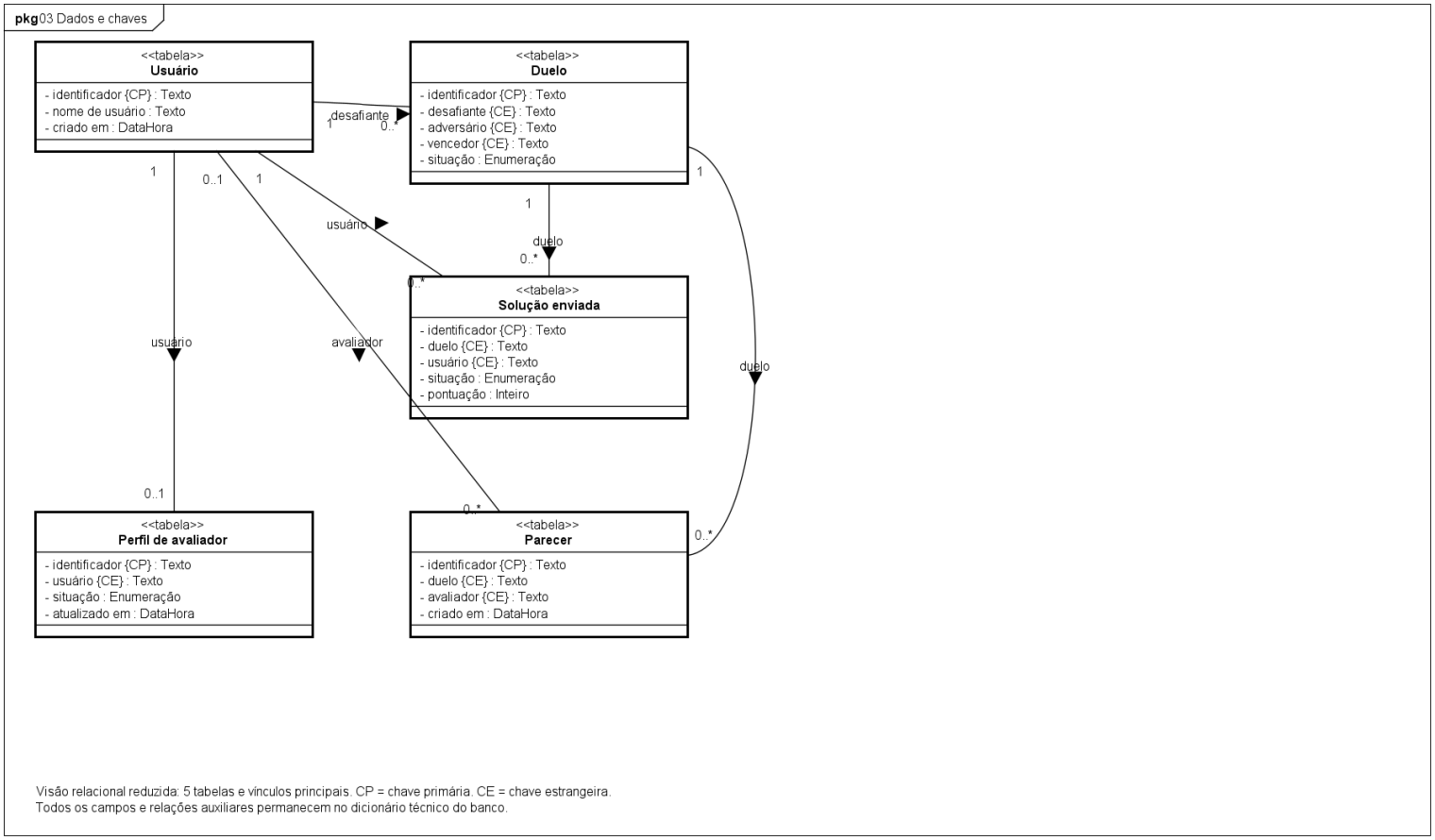
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



CL-DUELOS Classes de duelos

Dados de duelos

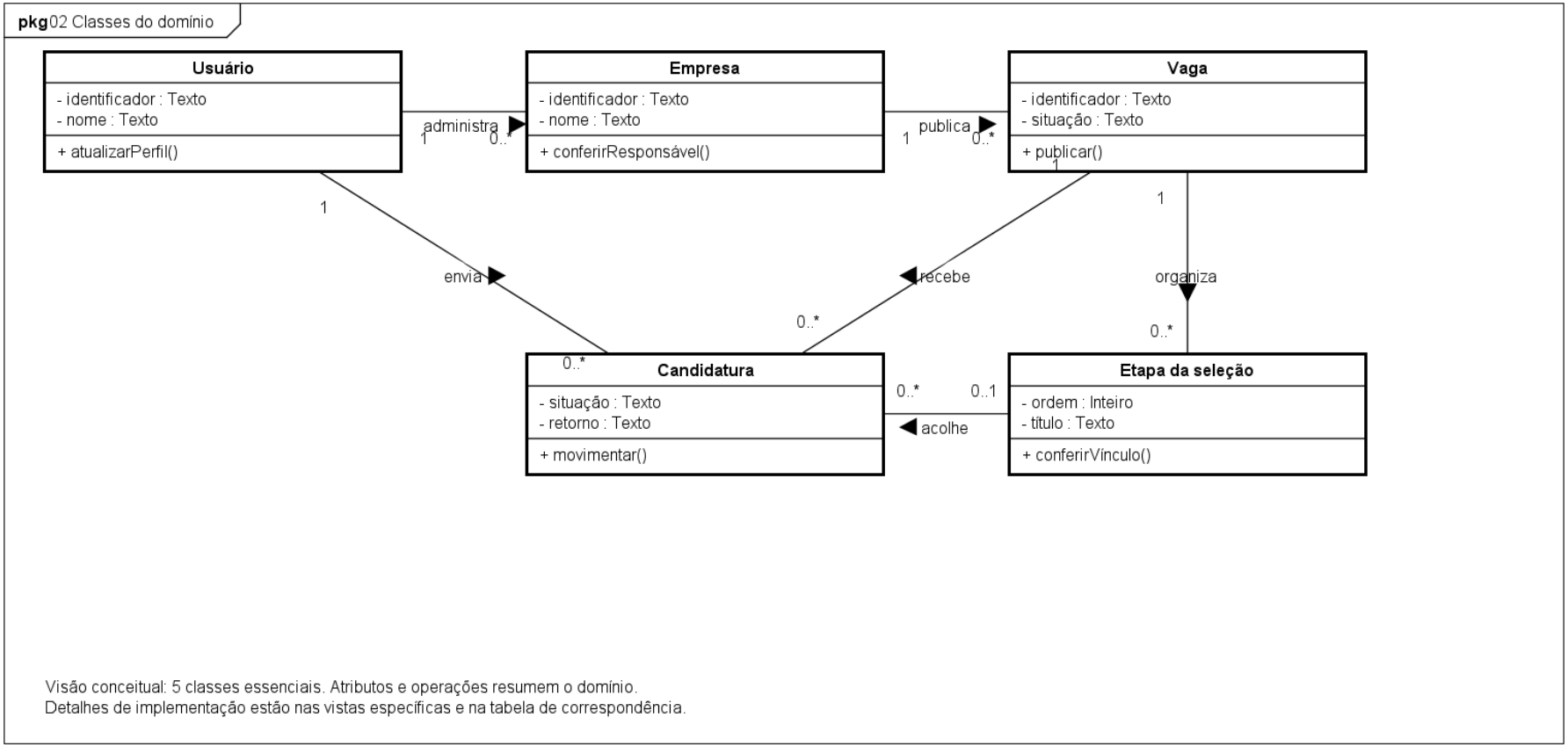
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



DER-DUELOS Dados de duelos

Classes de recrutamento

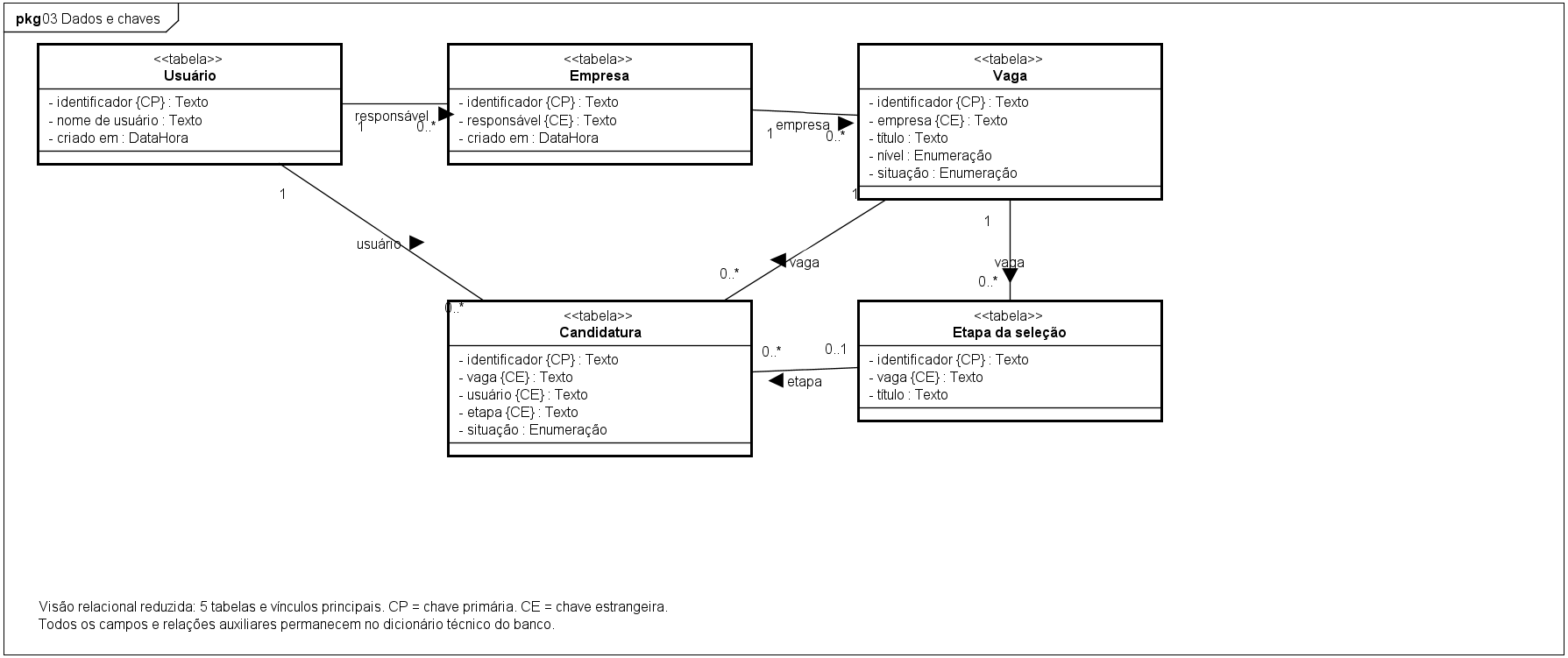
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



CL-RECRUTAMENTO Classes de recrutamento

Dados de recrutamento

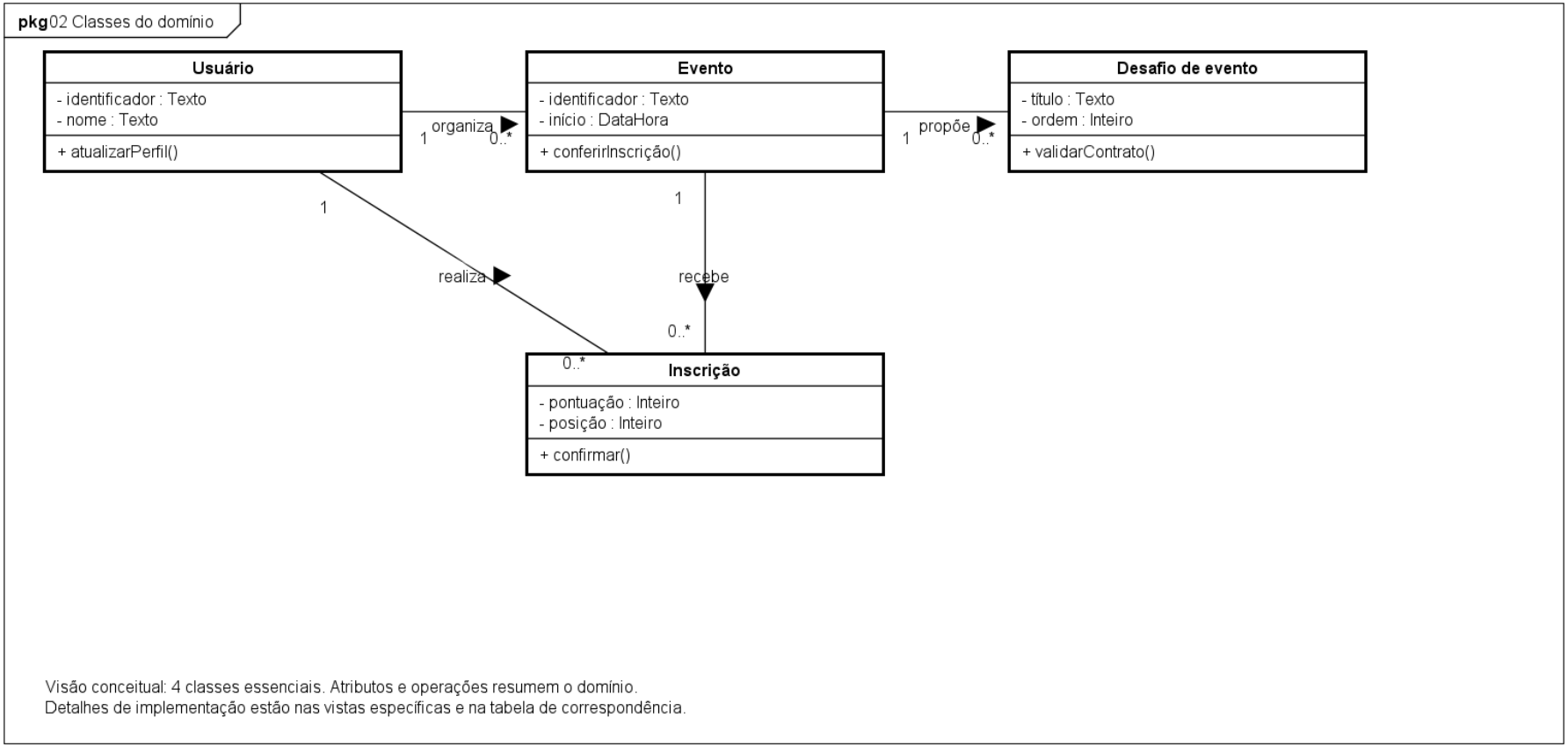
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



DER-RECRUTAMENTO Dados de recrutamento

Classes de eventos

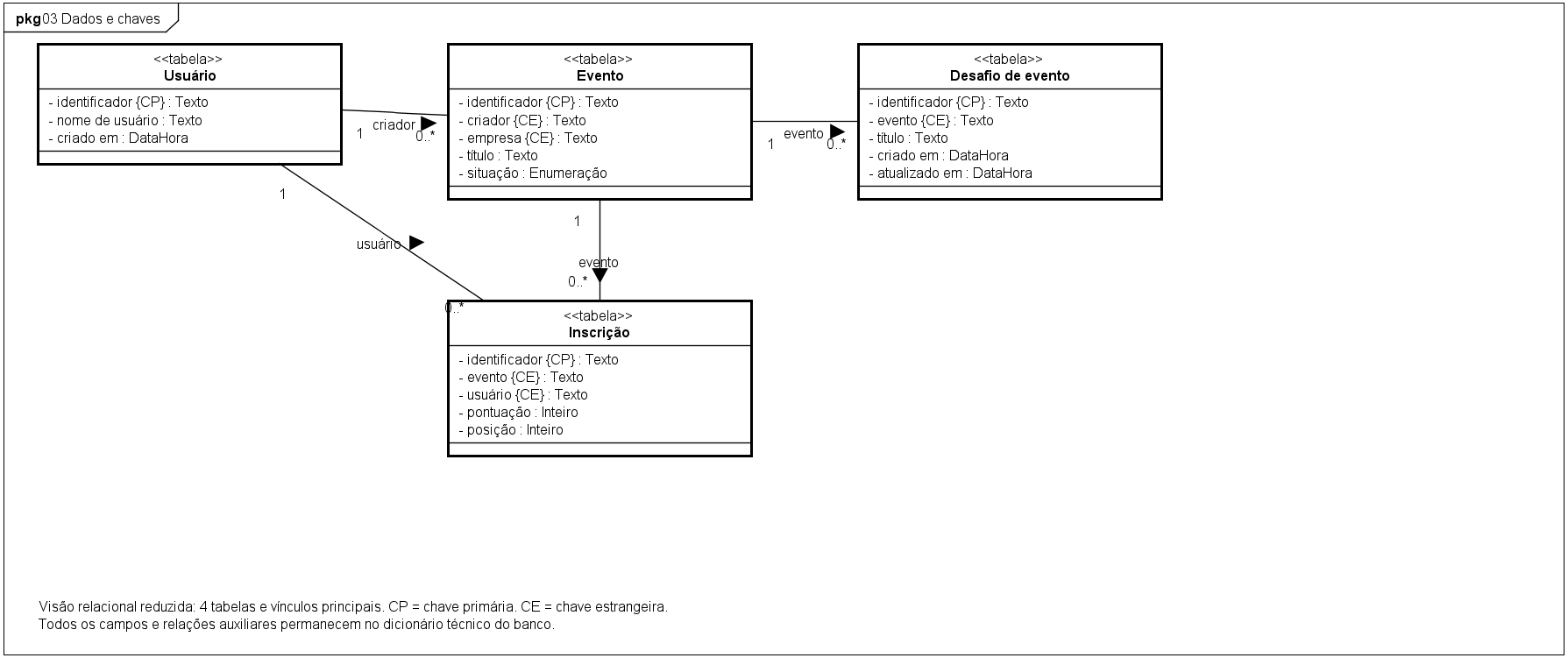
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



CL-EVENTOS Classes de eventos

Dados de eventos

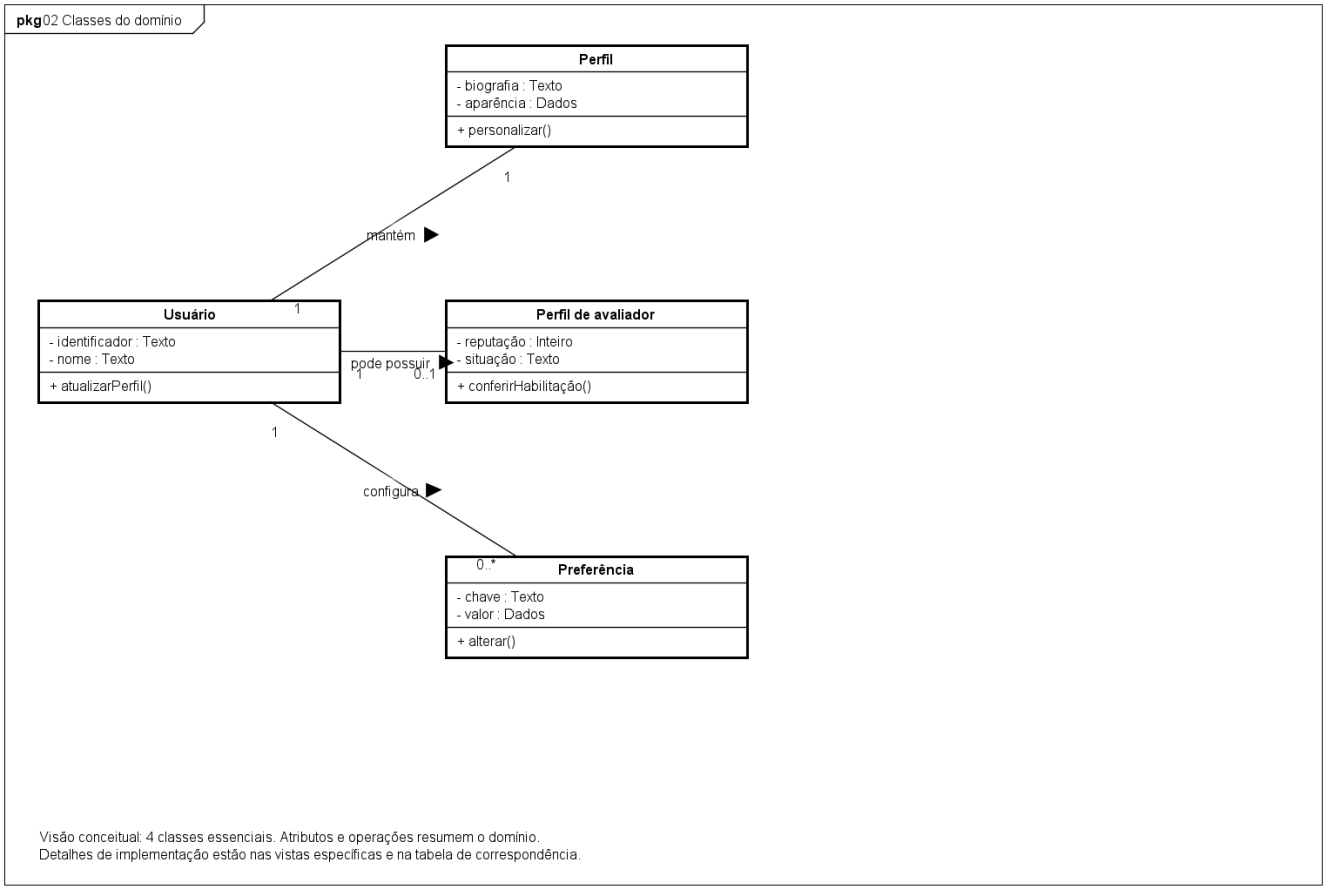
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



DER-EVENTOS Dados de eventos

Classes de identidade

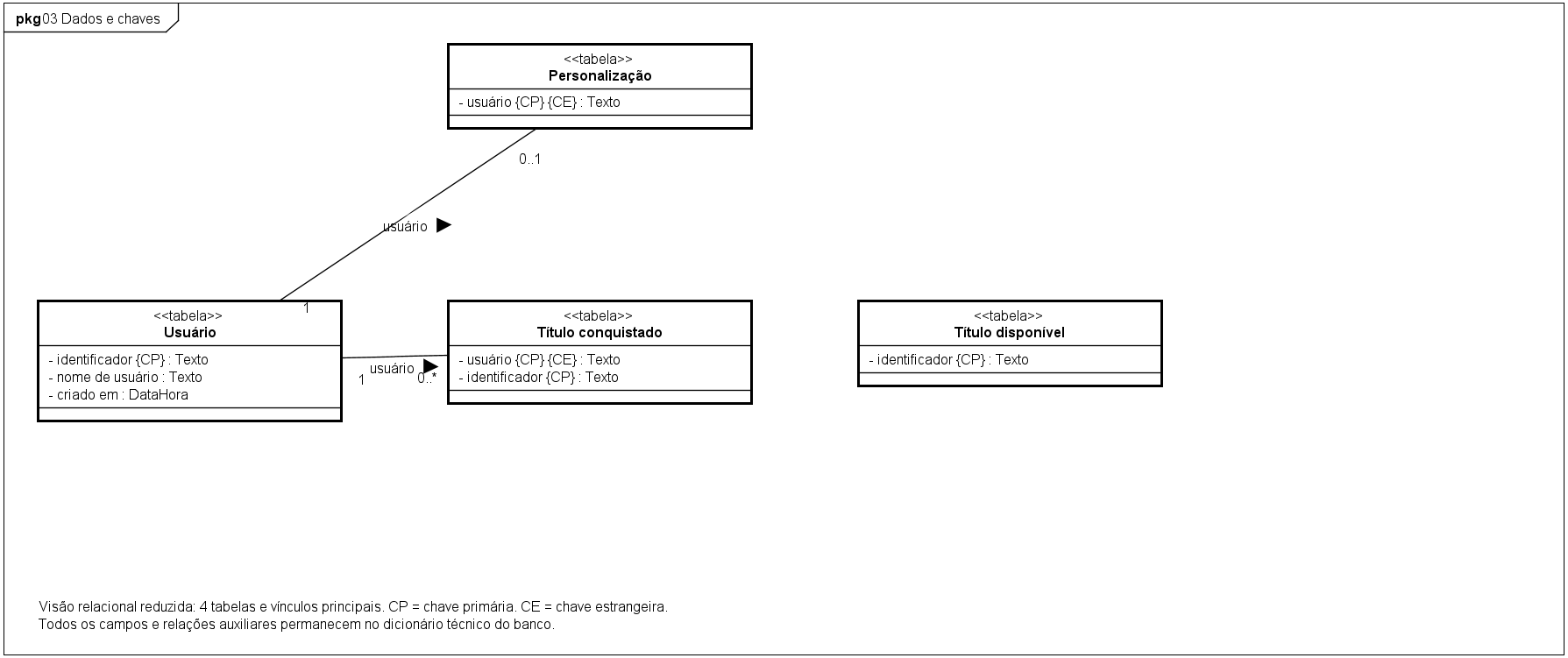
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



CL-IDENTIDADE Classes de identidade

Dados de identidade

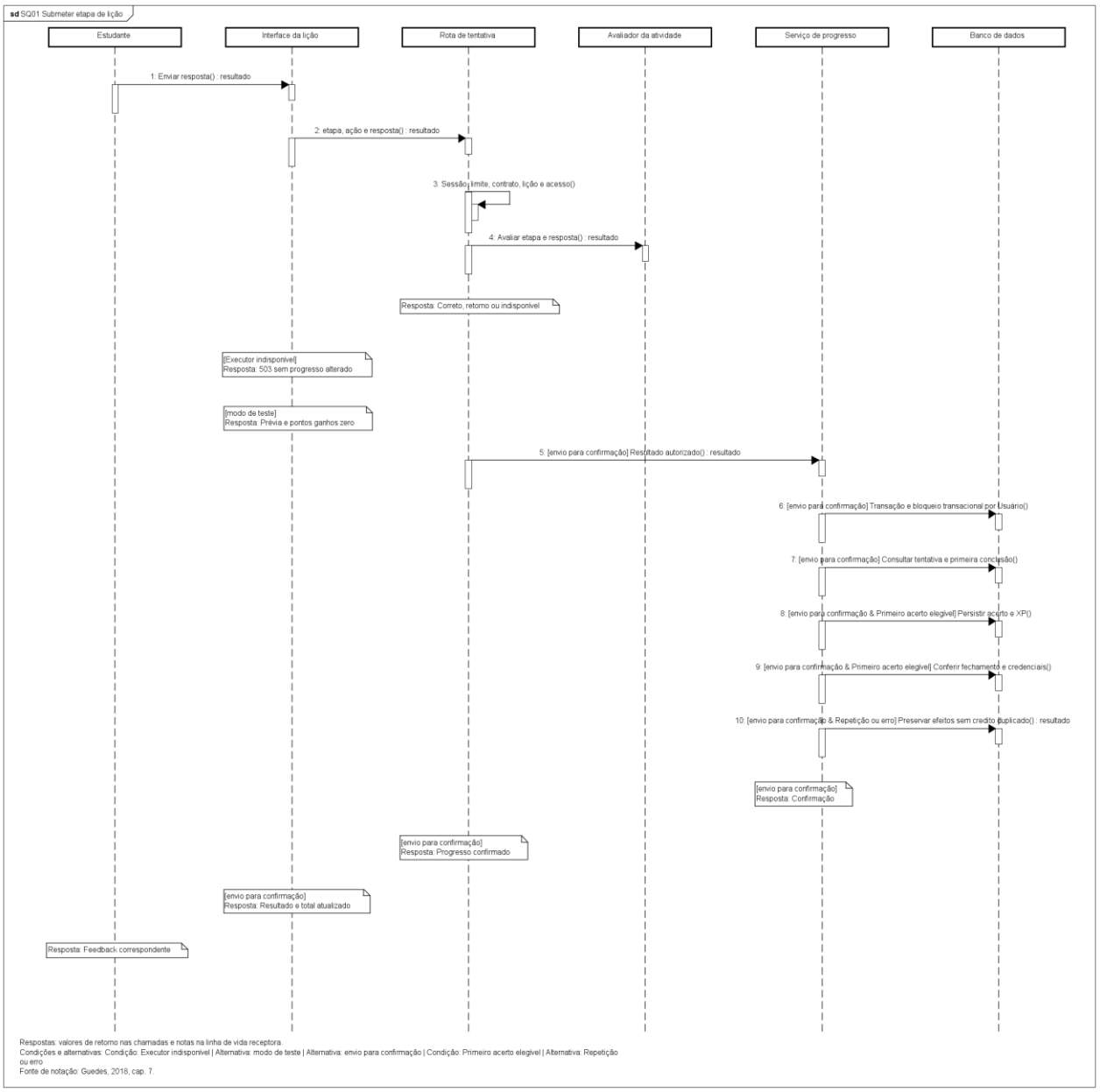
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



DER-IDENTIDADE Dados de identidade

Sequência completa 1

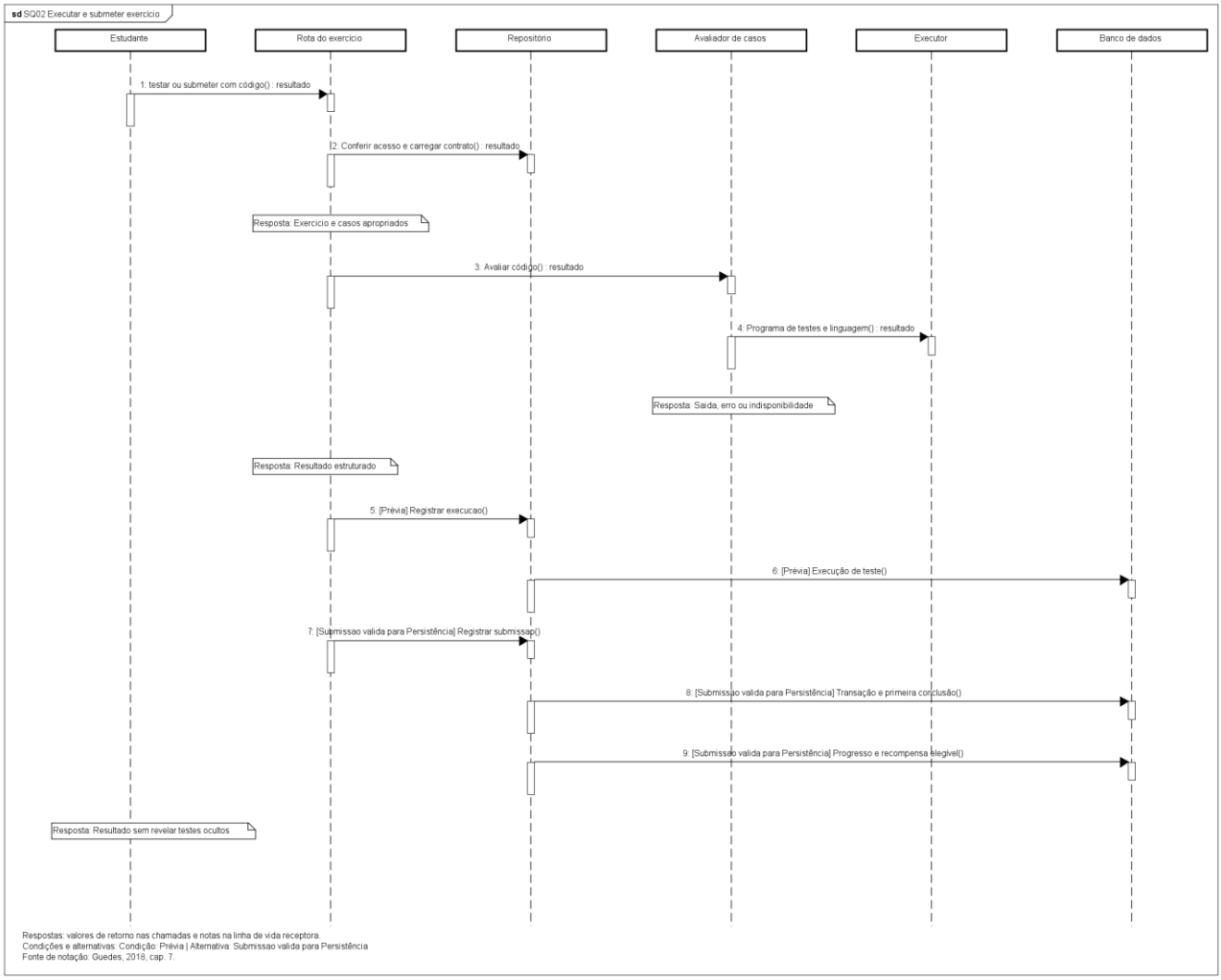
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ01 Sequência completa 1

Sequência completa 2

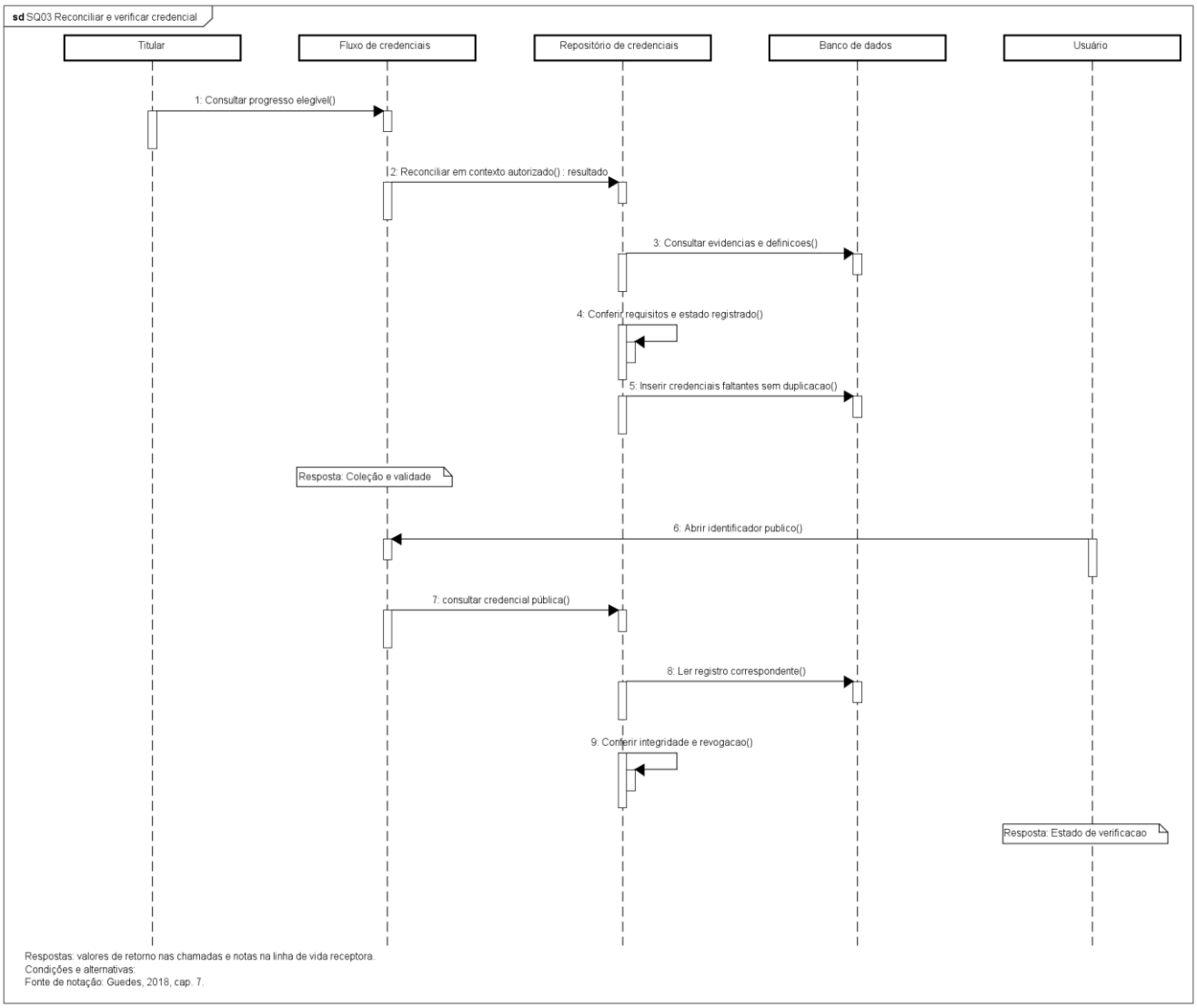
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ02 Sequência completa 2

Sequência completa 3

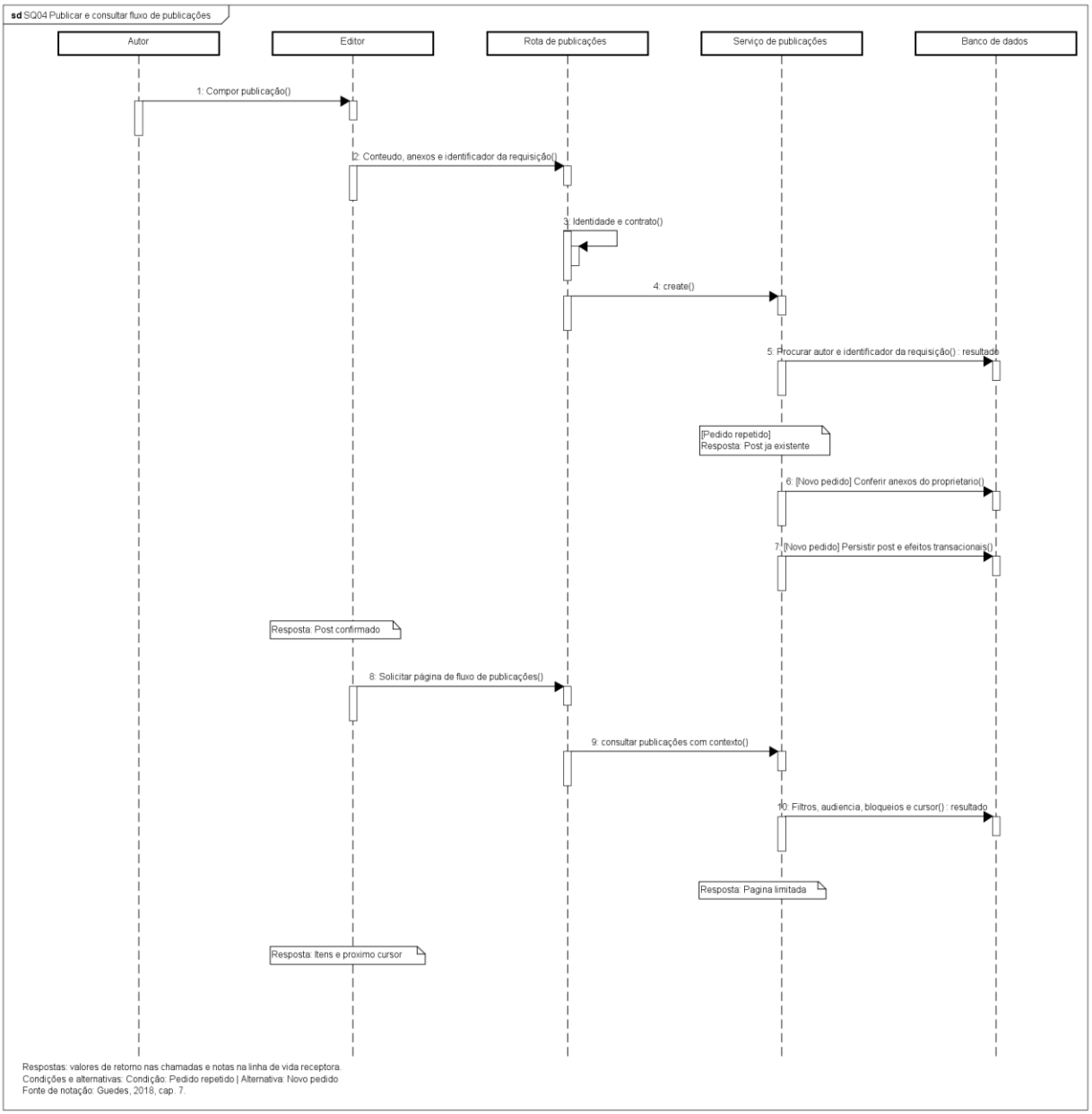
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ03 Sequência completa 3

Sequência completa 4

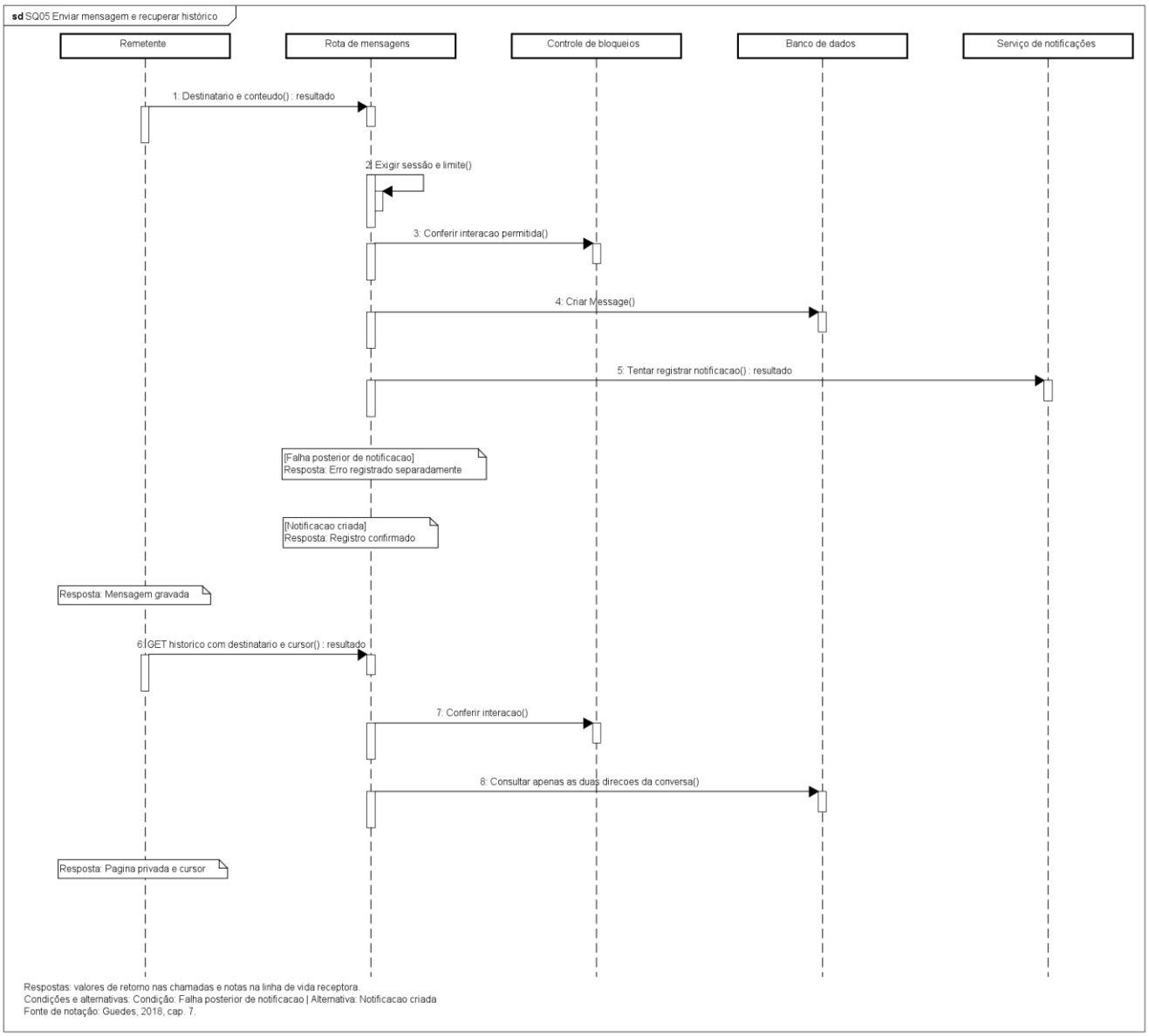
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ04 Sequência completa 4

Sequência completa 5

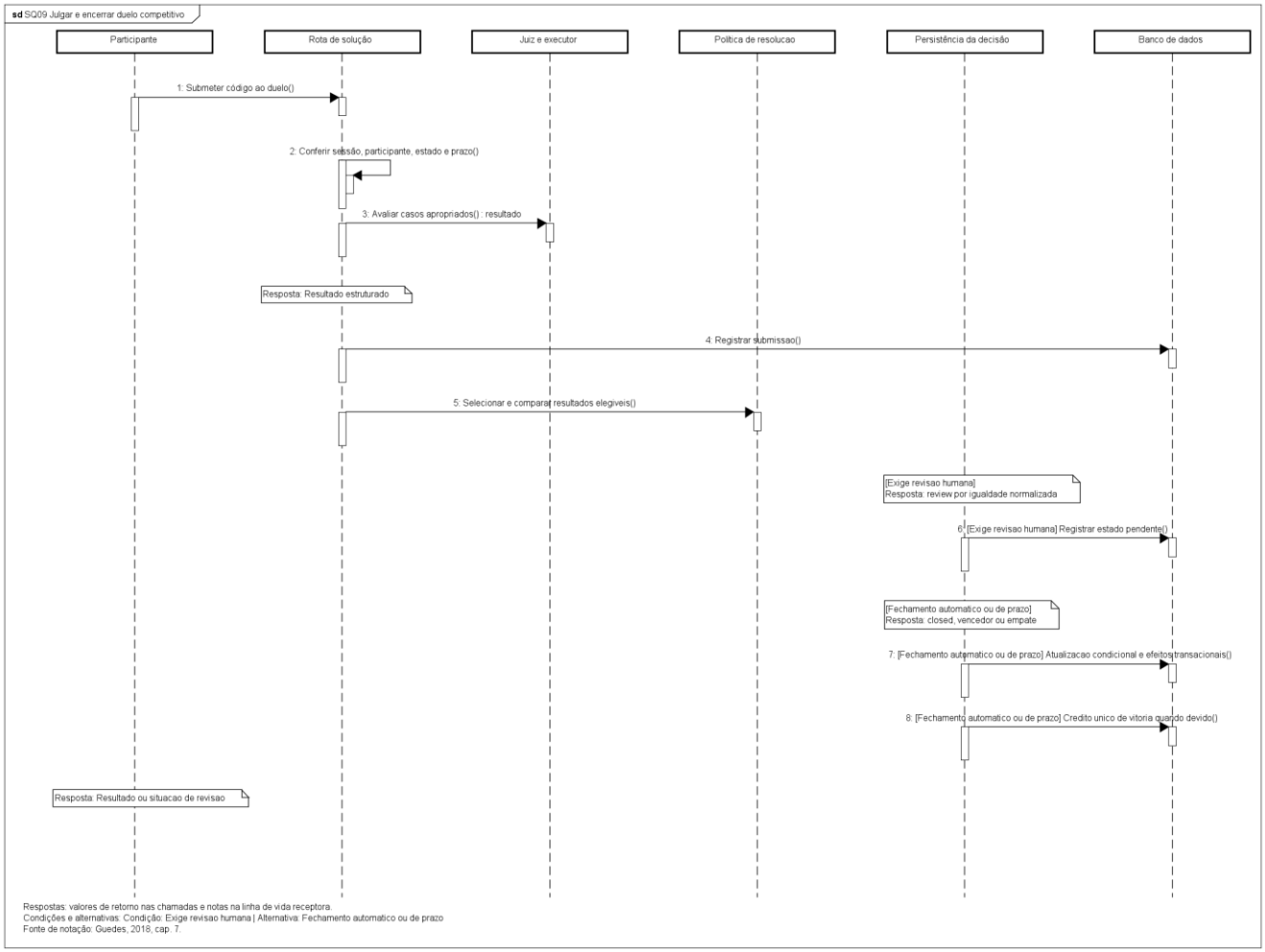
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ05 Sequência completa 5

Sequência completa 9

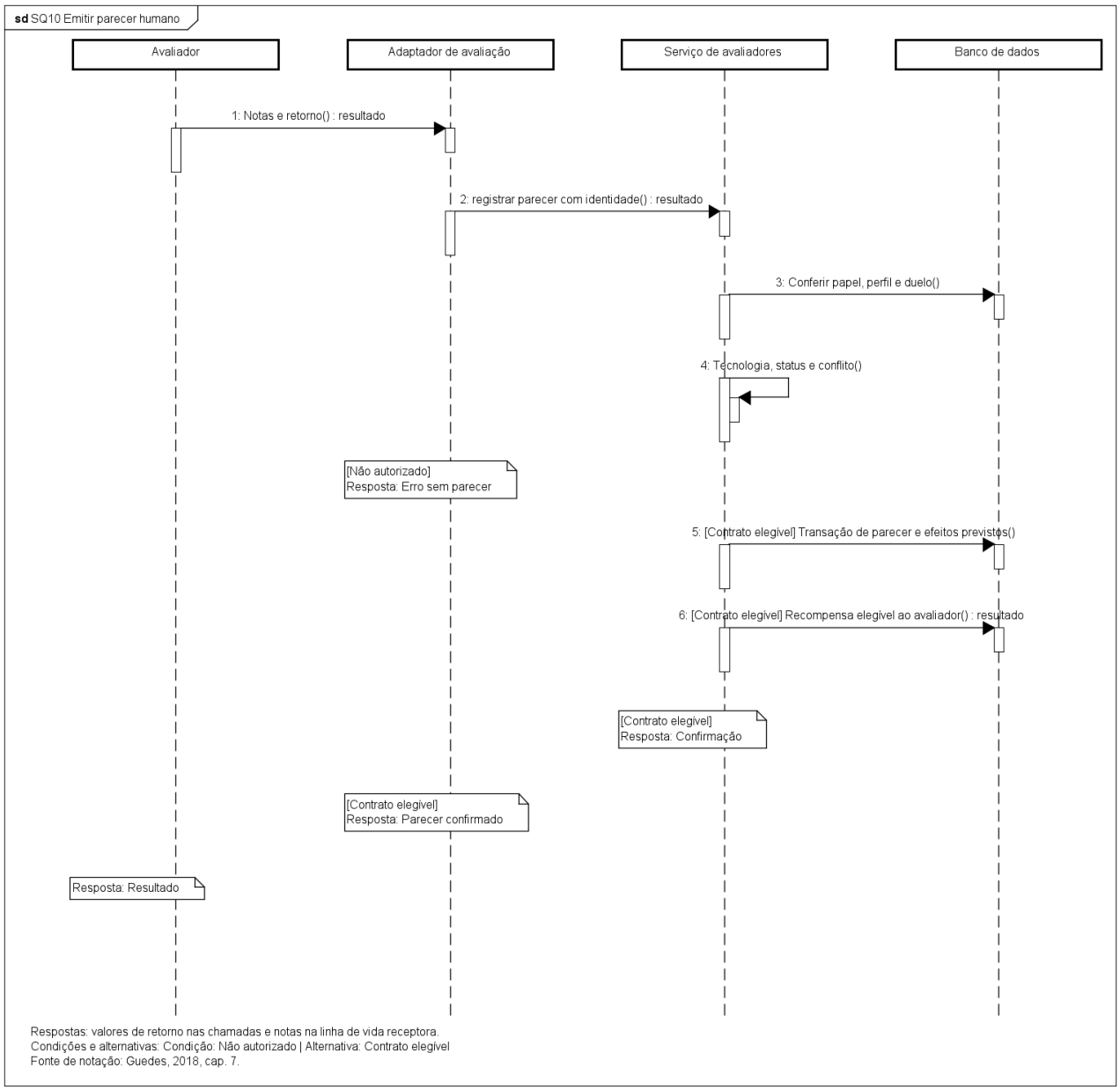
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ09 Sequência completa 9

Sequência completa 10

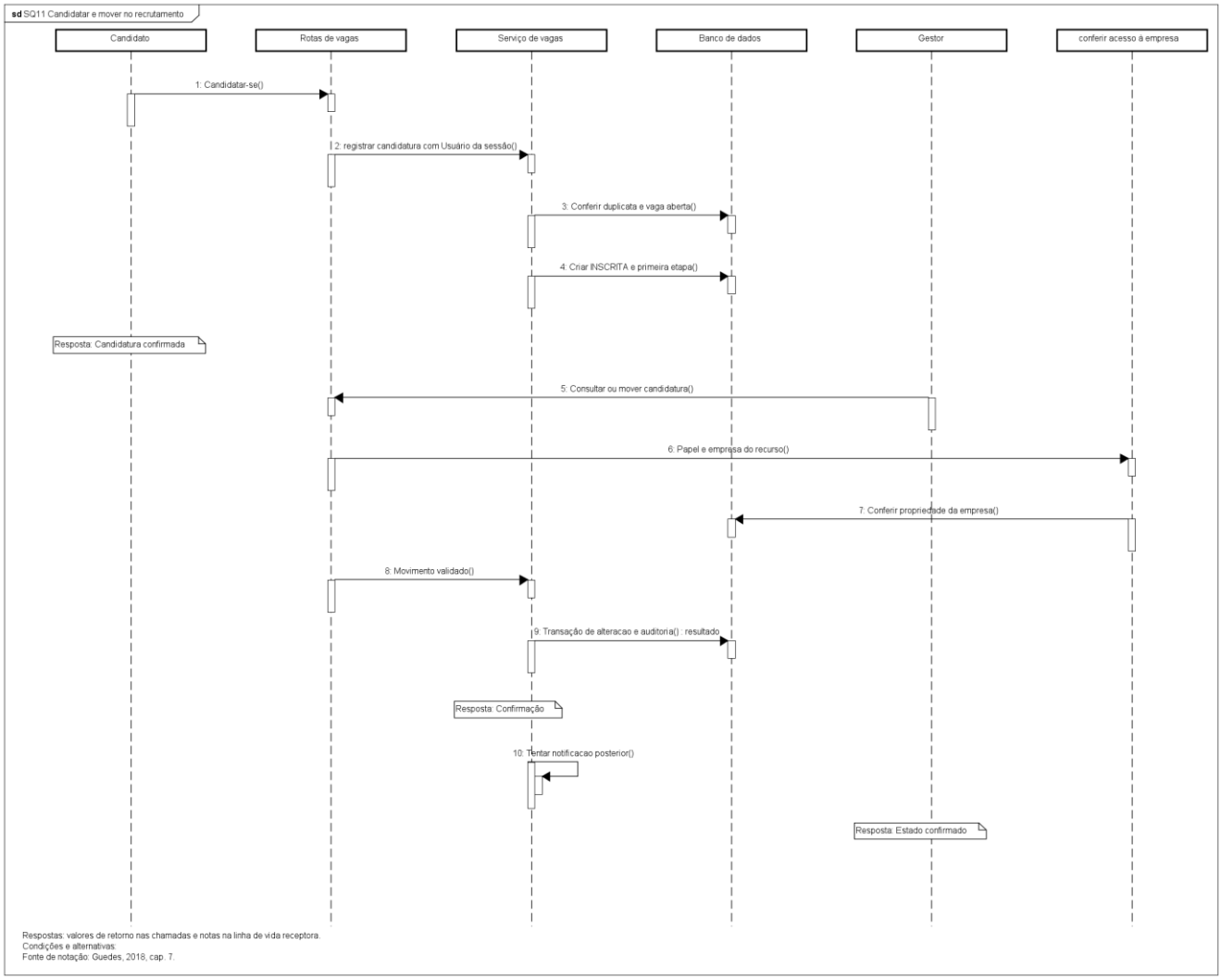
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ10 Sequência completa 10

Sequência completa 11

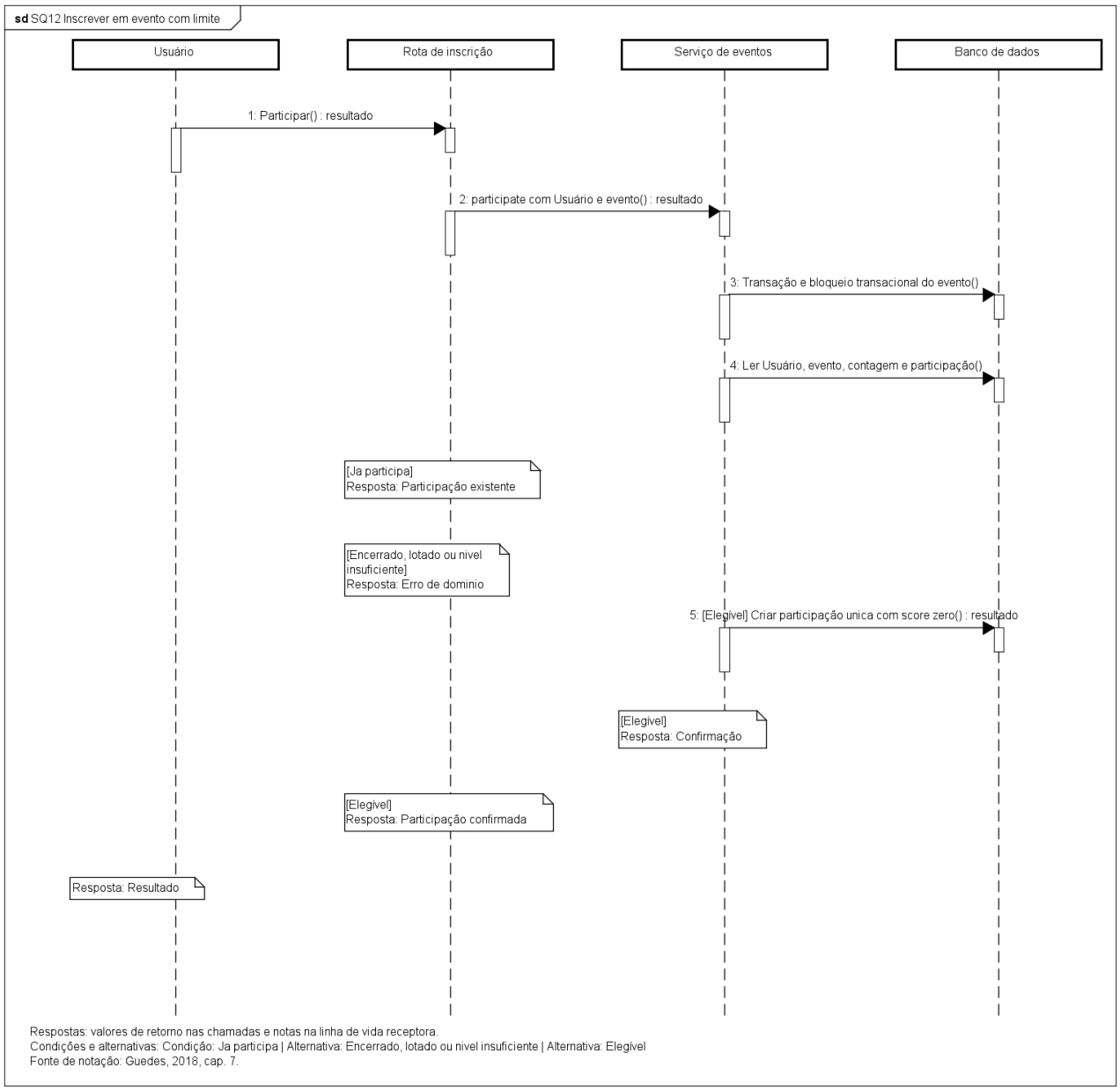
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ11 Sequência completa 11

Sequência completa 12

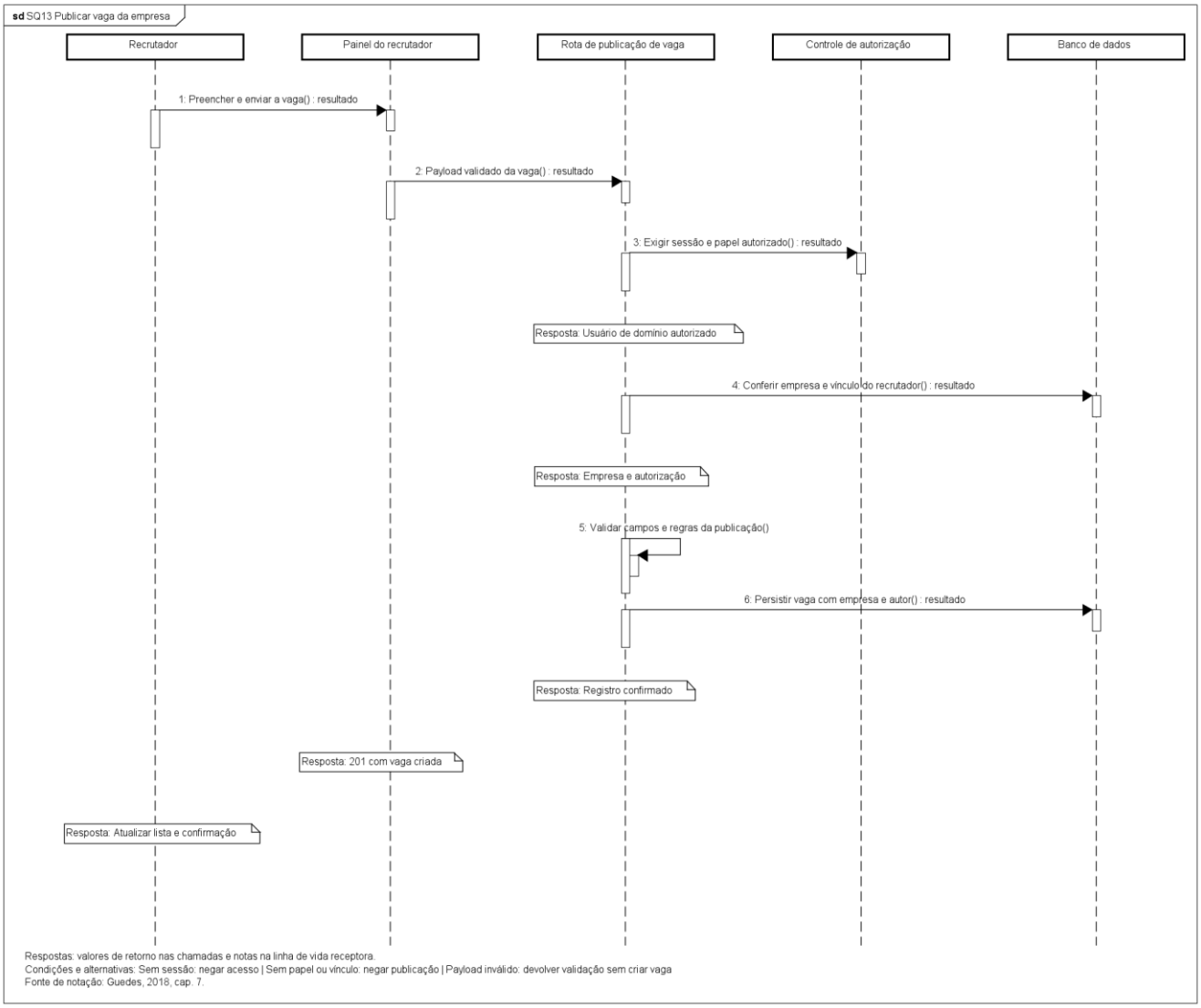
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ12 Sequência completa 12

Sequência completa 13

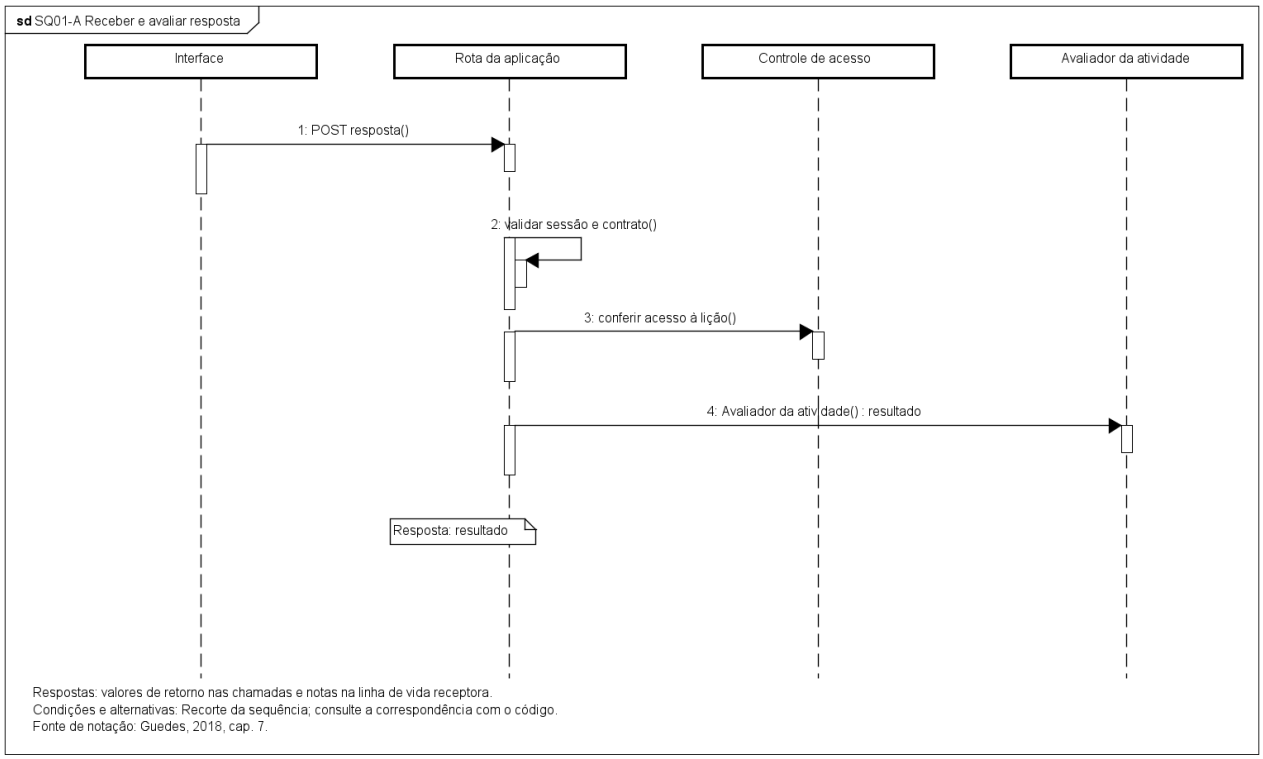
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ13 Sequência completa 13

Receber e avaliar resposta

Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



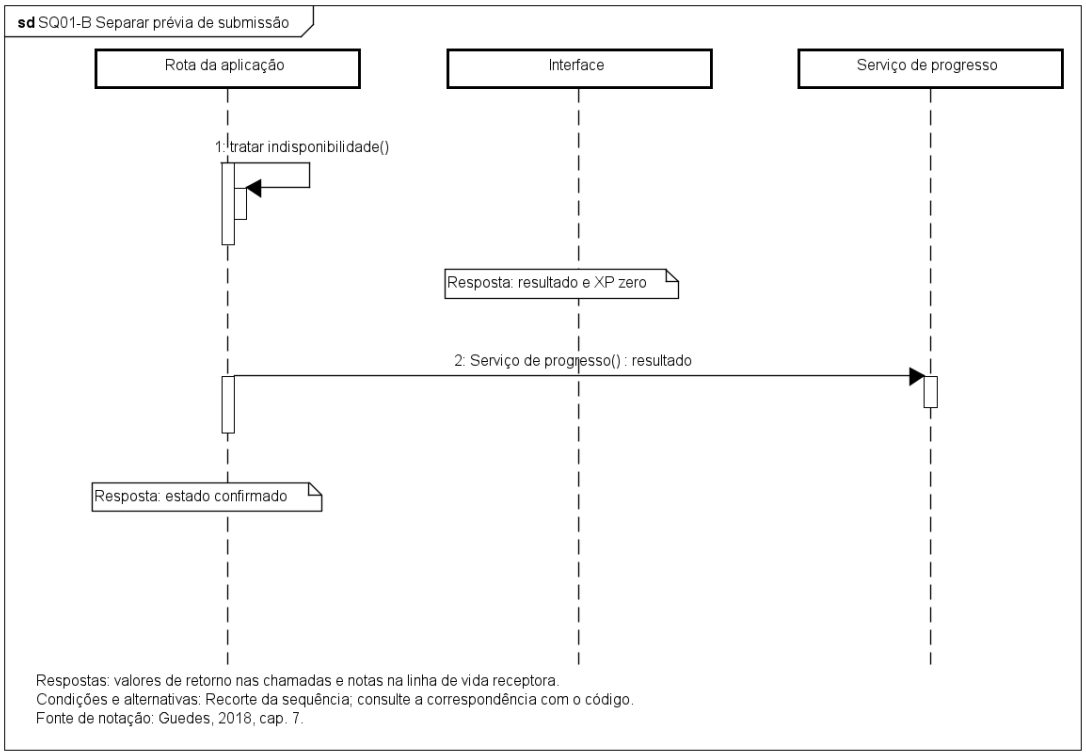
SQ01-A Receber e avaliar resposta

Código real: src/app/api/lessons/[lessonId]/attempt/route.ts, linhas 10 a 31. Contexto e ramos omitidos devem ser consultados no arquivo completo.

```
export const POST = apiHandler(async (request, { params, session: user }) => {
  if (!user) throw new UnauthorizedError();
  const { lessonId } = await params;
  await rateLimit(`lesson:${user.id}`, {
    limit: 30,
    window: '1 m',
    endpoint: '/api/lessons/attempt',
  });
  const parsed = lessonAnswerSchema.safeParse(await request.json().catch(() => null));
  if (!parsed.success)
    throw new ValidationError('INVALID_ANSWER', 'Confira os dados da resposta e tente novamente.');
  const answer = parsed.data;
  const lesson = getLessonById(lessonId);
  const step = lesson?.steps.find((s) => s.id === answer.stepId);
  if (!lesson || !step) throw new AppError('LESSON_NOT_FOUND', 'Atividade não encontrada.', 404);
  await requireLessonAccess(user.id, lesson);
  const outcome = await assessLessonStep(lesson, step, answer);
  if ('unavailable' in outcome && outcome.unavailable)
    throw new AppError(
      'RUNNER_UNAVAILABLE',
      'Serviço de execução indisponível. Seu progresso não foi alterado; tente novamente.',
      503
```

Separar prévia de submissão

Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



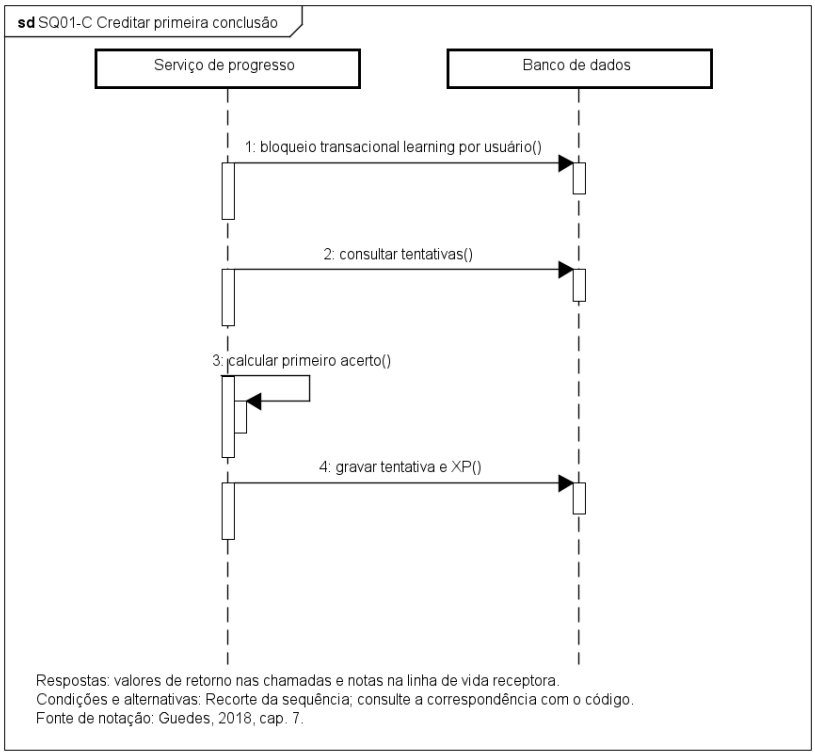
SQ01-B Separar prévia de submissão

Código real: src/app/api/lessons/[lessonId]/attempt/route.ts, linhas 33 a 45. Contexto e ramos omitidos devem ser consultados no arquivo completo.

```
  if (answer.action === 'run') return NextResponse.json({ ...outcome, xpEarned: 0 });
  const progress = await saveLessonAssessment(
    user.id,
    lesson,
    step,
    outcome.isCorrect,
    answer.selectedOption ?? 0
  );
  if (outcome.isCorrect) {
    await syncProfileAchievementsSafely(user.id);
  }
  return NextResponse.json({ ...outcome, ...progress });
});
```

Creditar primeira conclusão

Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



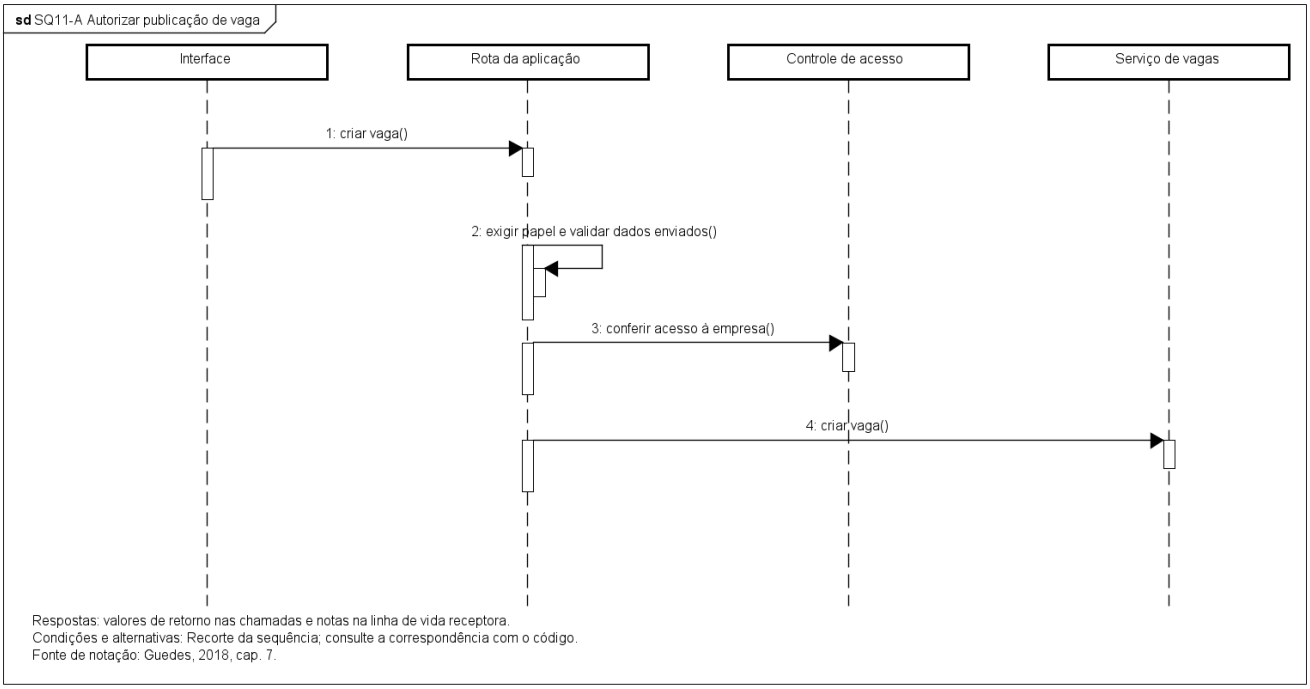
SQ01-C Creditar primeira conclusão

Código real: src/lib/learning/lesson-progress.ts, linhas 62 a 83. Contexto e ramos omitidos devem ser consultados no arquivo completo.

```
const firstCompletion = isCorrect && !previous?.is_correct;
const xpEarned = firstCompletion ? Math.max(0, step.xp) : 0;
if (!previous)
  await tx.quiz.upsert({
    where: { id: step.id },
    create: {
      id: step.id,
      question: step.question ?? step.instruction ?? step.title,
      options: step.options ?? ['Avaliado no servidor'],
      correct_index: step.correctOptionIndex ?? 0,
      is_daily: false,
    },
    update: {},
  });
if (!previous)
  await tx.quizAttempt.create({
    data: {
      user_id: userId,
      quiz_id: step.id,
      selected_index: selectedIndex,
      is_correct: isCorrect,
      xp_earned: xpEarned,
```

Autorizar publicação de vaga

Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ11-A Autorizar publicação de vaga

Código real: src/app/api/jobs/route.ts, linhas 72 a 95. Contexto e ramos omitidos devem ser consultados no arquivo completo.

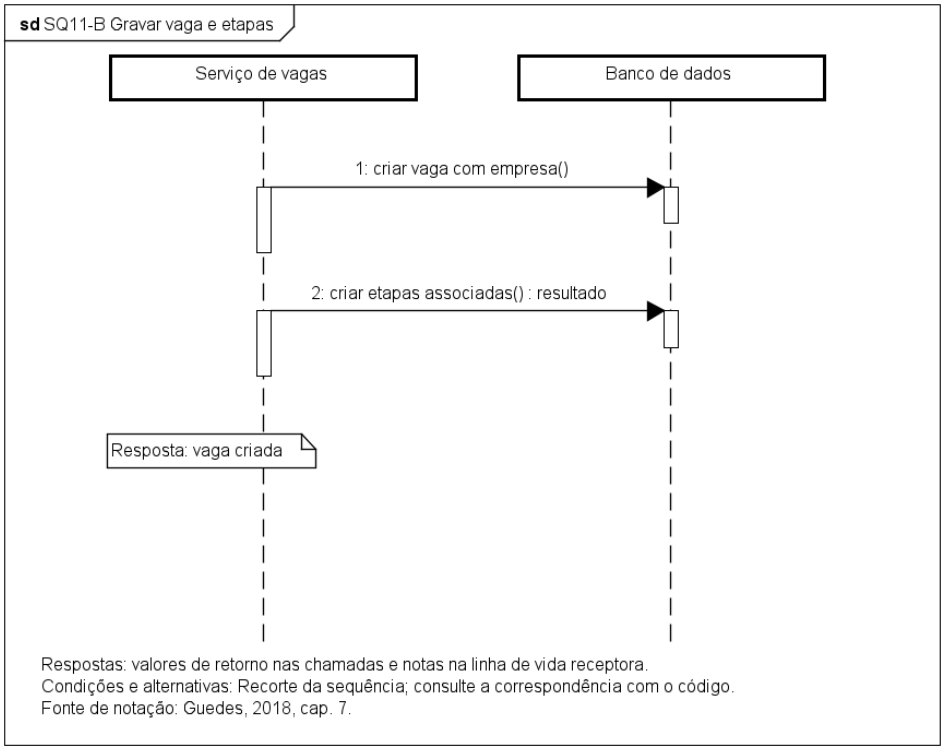
```
export const POST = apiHandler(async (req) => {
  const user = await requireRole(['RECRUITER', 'ADMIN']);
  const body = await req.json();
  const parsed = createJobSchema.parse(body);
  await requireCompanyAccess(parsed.company_id, user);

  const job = await JobService.createJob({
    companyId: parsed.company_id,
    title: parsed.title,
    description: parsed.description,
    level: parsed.level,
    technologies: parsed.technologies,
    modality: parsed.modality,
    location: parsed.location,
    contractType: parsed.contract_type,
    salaryMin: parsed.salary_min,
    salaryMax: parsed.salary_max,
    requirements: parsed.requirements,
    benefits: parsed.benefits,
    stages: parsed.stages,
  });

  return NextResponse.json({
    success: true,
```

Gravar vaga e etapas

Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



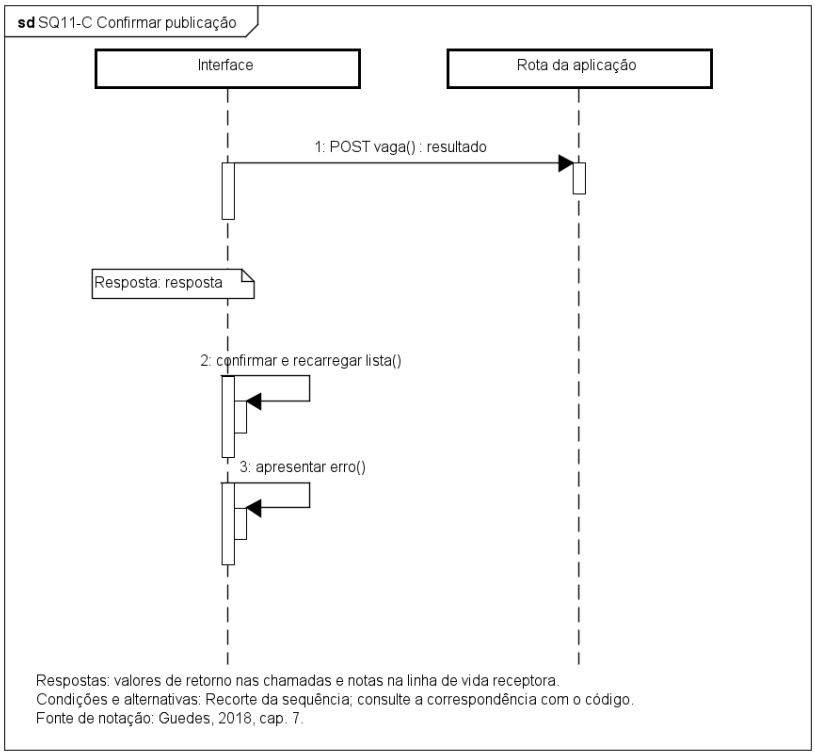
SQ11-B Gravar vaga e etapas

Código real: src/services/job.service.ts, linhas 218 a 247. Contexto e ramos omitidos devem ser consultados no arquivo completo.

```
    const job = await prisma.job.create({
      data: {
        company_id: companyId,
        title,
        description,
        level,
        technologies,
        modality,
        location,
        contract_type: contractType,
        salary_min: salaryMin,
        salary_max: salaryMax,
... (linhas intermediárias omitidas neste extrato) ...
            challenge_id: st.challengeId,
            description: st.description,
          })),
        },
      },
      include: {
        stages: true,
        company: true,
      },
    });
```

Confirmar publicação

Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



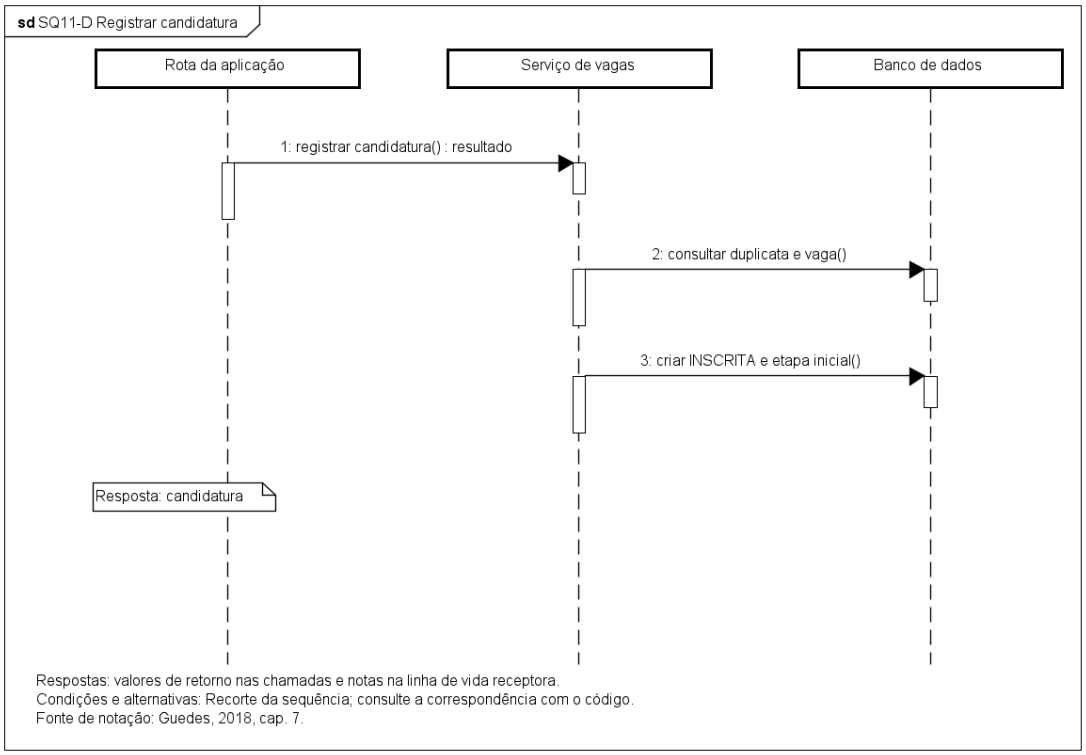
SQ11-C Confirmar publicação

Código real: src/app/recruiter/RecruiterContent.tsx, linhas 378 a 394. Contexto e ramos omitidos devem ser consultados no arquivo completo.

```
    if (res.ok) {
      setSuccessMsg('Vaga publicada com sucesso! As etapas do processo já estão disponíveis.');
      setJobTitle('');
      setJobDescription('');
      loadJobs(); // Instantly update jobs list
      setTimeout(() => setSuccessMsg(null), 4000);
    } else {
      const errData = await res.json().catch(() => null);
      setErrorMsg(errData?.message || errData?.error || 'Erro ao publicar vaga.');
    }
  } catch (err) {
    console.error(err);
    setErrorMsg('Erro de conexão ao publicar vaga.');
  } finally {
    setCreatingJob(false);
  }
};
```

Registrar candidatura

Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ11-D Registrar candidatura

Código real: src/services/job.service.ts, linhas 282 a 304. Contexto e ramos omitidos devem ser consultados no arquivo completo.

```
  const firstStage = job.stages[0];

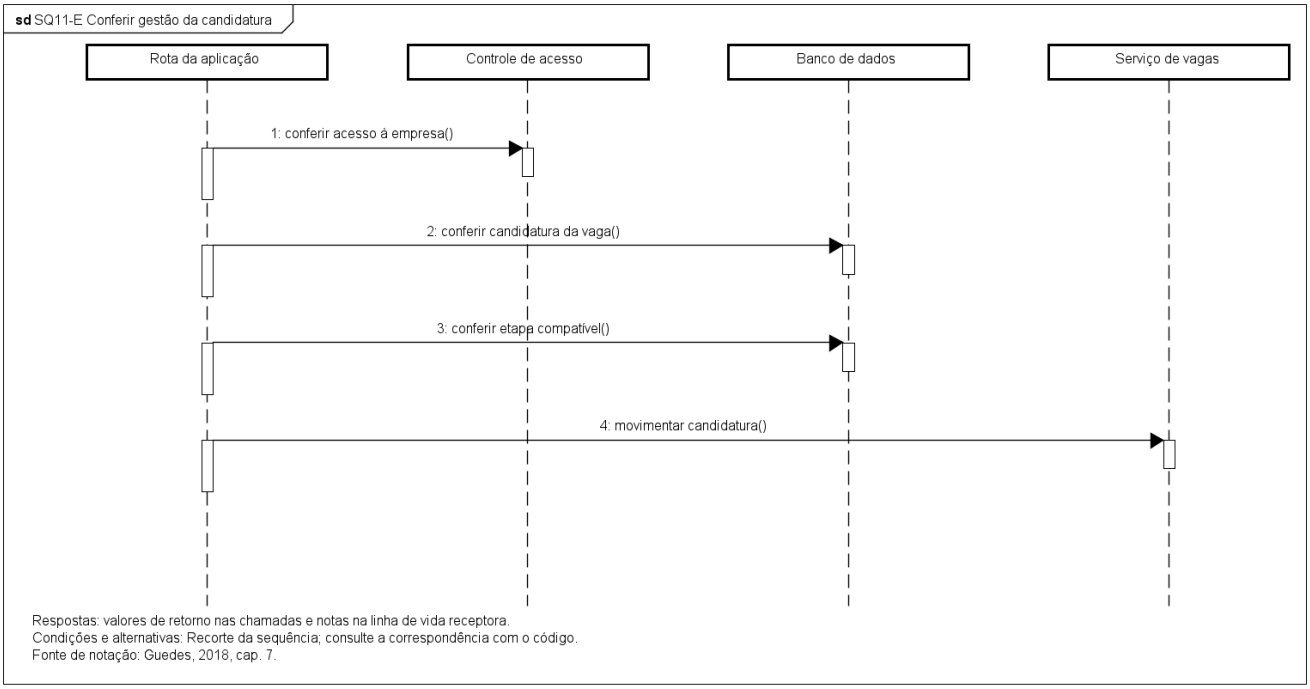
  const application = await prisma.jobApplication.create({
    data: {
      job_id: jobId,
      user_id: userId,
      stage_id: firstStage?.id,
      status: 'APPLIED',
    },
    include: {
      stage: true,
      job: { include: { company: true } },
    },
  });

  logger.info('User applied for job', { userId, jobId, applicationId: application.id });
  return application;
},

/**
 * Advance or update candidate application status in recruitment pipeline.
 */
async updateApplicationStage(params: {
```

Conferir gestão da candidatura

Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ11-E Conferir gestão da candidatura

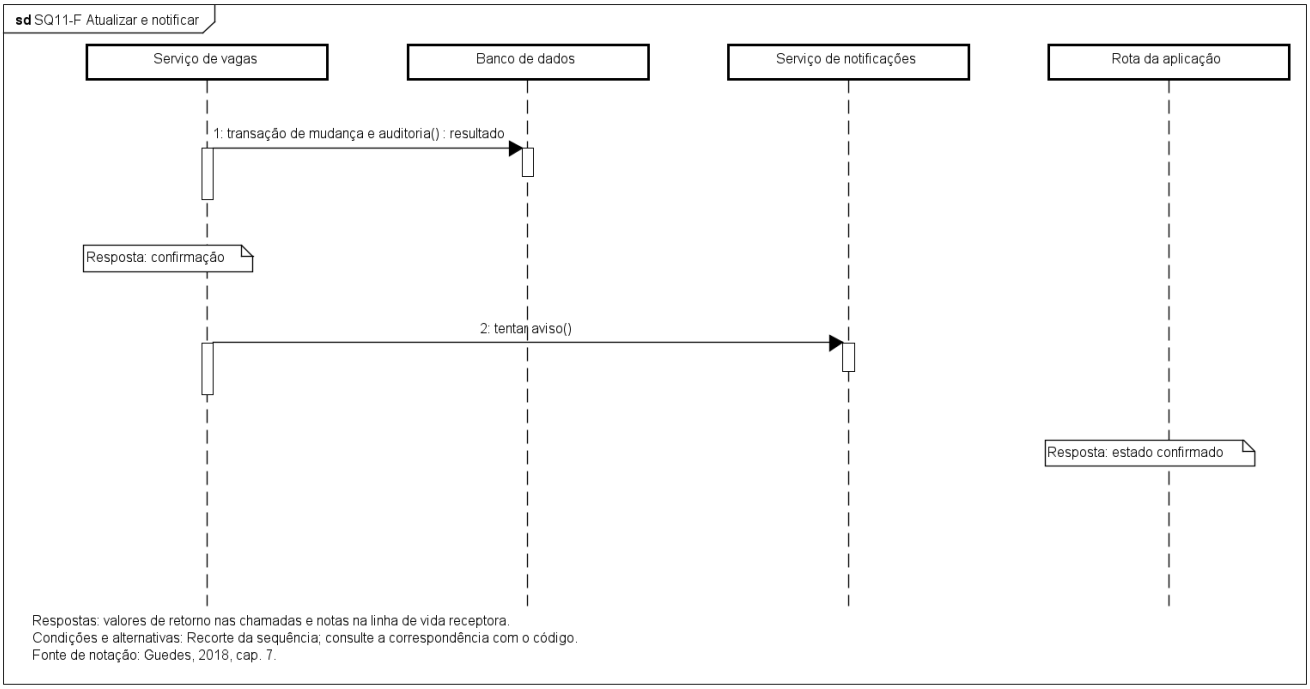
Código real: src/app/api/jobs/[id]/route.ts, linhas 30 a 62. Contexto e ramos omitidos devem ser consultados no arquivo completo.

```
export const PATCH = apiHandler(async (req, { params }) => {
  const user = await requireRole(['RECRUITER', 'ADMIN']);
  const { id } = await params;
  const job = await prisma.job.findUniqueOrThrow({ where: { id } });
  await requireCompanyAccess(job.company_id, user);
  const body = await req.json();
  const parsed = updateStageSchema.parse(body);
  const application = await prisma.jobApplication.findFirst({
    where: { id: parsed.application_id, job_id: id },
  });
  if (!application)
    return NextResponse.json({ error: 'Candidatura não encontrada nesta vaga.' }, { status: 404 });
... (linhas intermediárias omitidas neste extrato) ...
    technicalScore: parsed.technical_score,
    actorId: user.id,
  });

  return NextResponse.json({
    success: true,
    message: 'Etapa do candidato atualizada com sucesso!',
    application: updatedApplication,
  });
});
```

Atualizar e notificar

Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ11-F Atualizar e notificar

Código real: src/services/job.service.ts, linhas 314 a 359. Contexto e ramos omitidos devem ser consultados no arquivo completo.

```
    const application = await prisma.$transaction(async (tx) => {
      const updated = await tx.jobApplication.update({
        where: { id: applicationId },
        data: {
          stage_id: stageId,
          status,
          feedback,
          technical_score: technicalScore,
        },
        include: {
          stage: true,
          user: {
... (linhas intermediárias omitidas neste extrato) ...
        type: 'JOB_APPLICATION_UPDATE',
        resourceId: application.job_id,
        resourceType: 'JOB',
      });
    } catch (err) {
      logger.error('Failed to notify job application update', { error: String(err) });
    }

    return application;
  },
```

Correspondência dos modelos com a implementação

Os nomes em português são conceitos e responsabilidades do domínio. As operações conceituais não afirmam que exista uma função TypeScript com a mesma grafia. Esta tabela localiza os modelos e os contratos reais para a leitura do código.

Classe no diagrama	Referência técnica
Usuário	User
Trilha	src/lib/learning
Lição	src/lib/learning/catalog.ts
Atividade	Quiz / Exercise
Tentativa	QuizAttempt / ExerciseSubmission
Credencial	LearningCredential
Duelo	Duel
Solução	DuelSubmission
Parecer	DuelEvaluation
Perfil de avaliador	EvaluatorProfile
Publicação	Post
Comentário	Answer
Perfil	User / ProfileCustomization
Empresa	Company
Vaga	Job
Etapa da seleção	JobRecruitmentStage
Candidatura	JobApplication
Evento	Event
Inscrição	EventParticipant
Desafio de evento	EventChallenge
Mensagem	Message
Notificação	Notification
Preferência	User metadata / MobileState

Tabelas relacionais

Nome em português	Modelo Prisma
Usuário	User
Progresso da linguagem	LanguageTrail
Atividade	Quiz
Tentativa	QuizAttempt
Credencial	LearningCredential
Publicação	Post
Comentário	Answer
Mensagem	Message
Notificação	Notification
Empresa	Company
Vaga	Job
Etapa da seleção	JobRecruitmentStage
Candidatura	JobApplication
Duelo	Duel
Solução enviada	DuelSubmission
Parecer	DuelEvaluation
Perfil de avaliador	EvaluatorProfile
Evento	Event
Inscrição	EventParticipant
Desafio de evento	EventChallenge
Personalização	ProfileCustomization
Título conquistado	UserProfileAchievement
Título disponível	ProfileAchievementCatalog

Uso da documentação no seminário

O que foi aproveitado do material fornecido

A lousa e a transcrição organizam a apresentação em duas escalas: visão geral uma vez e explicação específica por participante. Cada recorte deve ligar interface, requisitos, regras, modelagem, sequência, código e demonstração. A entrega escrita deve cobrir também funções que não caibam na exposição oral.

A entrega anterior identifica Pedro Antonio Silvestre Tescaro, Guilherme Alves de Campos e Paulo Henrique Leal dos Santos, nessa ordem proposta para aprendizagem, publicação de vaga e candidatura. Esta revisão conserva essa continuidade, com referência à entrega de 20/10/2026; não presume uma nova distribuição comunicada pelo grupo.

Ordem lógica sugerida

Bloco	Objetivo	Material e preparação
Abertura	Explicar problema, atores e jornada	Visão do sistema, atividade do negócio e panorama de casos
Identidade e arquitetura	Explicar como tela chama servidor e quem pode agir	Guia de programação, classes de responsabilidade e DER geral
Aprendizagem	Demonstrar o núcleo da proposta	Curso, lição, etapa, run, submit, progresso e XP
Reconhecimento e comunidade	Mostrar como evidência e identidade aparecem	Perfil, título, credencial, post e notificação
Competição	Diferenciar juiz de duelo e combate RPG	Duelo ou Mundinho/PvP com dois participantes preparados
Oportunidades	Consumir perfis e progresso em outro contexto	Empresa, vaga, candidatura e funil; evento se houver tempo
Fechamento	Mostrar limites e decisões de engenharia	Atomicidade, proteção de dados e evidências de testes

Os blocos podem ser repartidos entre integrantes. A sequência evita demonstrar uma candidatura antes de existir vaga, ou revisão humana antes de existir avaliador e duelo elegível.

Roteiro de cada funcionalidade

1. Abra a tela e explique quem a utiliza e para quê.

2. Aponte os requisitos funcionais, as propriedades de qualidade e as regras relevantes pelos IDs deste manual.

3. Mostre as responsabilidades de código e as tabelas daquele recorte.

4. Percorra a sequência completa sem mergulhar em todas as linhas.

5. Escolha dois ou três trechos reais e explique entrada, validação, autorização, decisão e efeito.

6. Execute a ação demonstrada e confira o resultado persistido visível.

7. Mostre uma alternativa importante: repetição, bloqueio, resposta incorreta ou falta de requisito.

Não apresente somente um arquivo de serviço sem explicar a interface e o objetivo. Também não apresente somente botões sem explicar como o servidor confirma a regra.

Diagramas gerais e específicos

O modelo de responsabilidades [modelagem.md] não pretende fingir que todos os serviços são classes JavaScript: vários são objetos literais e funções. Explique essa correspondência ao apresentar uma visão de classes conceitual.

O DER deriva do schema e contém relações com chaves. MER é o modelo de entidades e relacionamentos; DER é uma representação diagramática. A presença de FK depende do nível conceitual/lógico/físico escolhido, e não é por si uma definição universal que separa as siglas. Para atender à organização da disciplina, use a visão de banco com PK/FK e explique seus vínculos, evitando duplicar slides sem informação nova.

Recortes de sequência e código

Demonstração	Trechos especialmente úteis
Lição	Rota de tentativa, assessLessonStep, saveLessonAssessment
Exercício	Avaliador, execução e recordExerciseSubmission
Baú	dayKey, credit, ramo da ação chest
PvP	chooseArenaAttack, advanceArenaTurn, conferências em arena
Duelo	Política decideDuelResolution e persistência de resultado
Vaga	applyForJob, requireCompanyAccess, updateApplicationStage
Evento	participate e lock de capacidade
Perfil/credencial	Conciliação de conquista versus emissão de credencial

Os recortes gerados [codigo-comentado.md] conservam origem e linhas; o arquivo completo deve estar disponível para responder perguntas.

Dados preparados

Prepare contas de demonstração e papéis adequados. Use atividades e objetos controlados: lição com resposta conhecida, outro usuário para interação, vaga aberta, evento em datas válidas e adversário disponível. Se uma informação não será explicada, pode estar previamente criada, mas diga que é dado preparado.

Não prometa que um compilador externo estará disponível sem plano alternativo. Uma evidência gravada pode apoiar a explicação, desde que seja identificada como evidência de outra execução e não apresentada como resultado ao vivo.

Tempo e divisão

O material fornecido menciona até uma hora. Esse é o contexto informado, não uma regra técnica do produto. Distribua o tempo por complexidade e dependência; reserve espaço para mudança de conta, carregamento e perguntas. Evite um integrante consumir a abertura inteira e outro mostrar apenas uma tela.

Todos precisam conseguir explicar a teoria e o código de seu recorte, inclusive quem trabalhou principalmente na documentação. O objetivo do encadeamento é permitir essa compreensão sem decorar o projeto inteiro linha por linha.

Checklist de entrega

Casos de uso escritos cobrem todos os módulos, não só os escolhidos para os slides.

Diagramas gerais e vistas específicas estão coerentes com a mesma base.

Regras citadas têm fonte e não confundem desejo futuro com implementação.

Sequências terminam na confirmação/erro, não apenas na chamada à API.

Dados de demonstração não envolvem membros reais sem autorização.

Credenciais e capturas não expõem segredos ou dados privados.

Relatórios antigos conservam sua data; esta edição não os declara atualizados automaticamente.

O ensaio permite executar início, resultado e pelo menos uma alternativa importante.

As entregas históricas de Word, PDF, slides e Astah permanecem no diretório da disciplina [../../laboratorio-engenharia-software/README.md]. Esta edição é a documentação Markdown atualizada que pode orientar uma revisão futura desses artefatos.